

Mathematics of Linguistic Reality

An Introduction

Cem Bozşahin

No series description provided

Mathematics of Linguistic Reality

An Introduction

Cem Bozşahin

Cem Bozşahin. 2026. *Mathematics of Linguistic Reality: An Introduction*. Berlin: Language Science Press.

This title can be downloaded at:

`redefine\lsURL`

© 2026, Cem Bozşahin

Published under the Creative Commons Attribution 4.0 Licence (CC BY 4.0):

<http://creativecommons.org/licenses/by/4.0/>

ISBN: `no digital ISBN`

`no print ISBNs!`

`no DOI`

`ID not assigned!`

Cover and concept of design: Ulrike Harbort

Fonts: Libertinus, Arimo, DejaVu Sans Mono

Typesetting software: $\text{\LaTeX}$

`redefine \publisherstreetaddress`

`redefine \publisherurl`

Storage and cataloguing done by `redefine \storageinstitution`

Contents

Preface	v
Abbreviations	xi
1 Basics revisited	1
1.1 Sets	3
1.2 Strings	7
1.3 Relations	9
1.4 Functions: named and nameless	12
1.4.1 Trees as functions	15
1.4.2 Function composition	16
1.4.3 Diagrams of functions	18
1.4.4 Lambda calculus	19
1.4.5 Function composition and application: one connectedness	22
1.5 Recursion: named and nameless	25
1.6 Probability	28
1.7 Logic	31
1.8 Mathematics and computation	34
1.9 Linguistic structures	36
1.10 Linguistics and philosophy	39
1.11 Typology and mathematics	42
1.12 Theories and models: a brief foray into modeling	45
2 Mathematics and philosophy of linguistics	53
2.1 Philosophy of linguistics as philosophy of science	54
2.2 Ontogeny of grammaticality	56
2.3 Some questions that make linguistics a special science	63
2.4 Mathematics in linguistics versus mathematicizing linguistics	65
3 Degrees of structure from string admissibility	67
3.1 Cutting the Chomsky hierarchy at different seams	68
3.2 Structure: admissibility of strings	71
3.2.1 Context-free admissibility	71

Contents

3.2.2	Regular admissibility	73
3.2.3	Linear admissibility	74
3.3	String admissibility: nonreified mathematics with implications for order and structure	75
4	Formal grammars: describing admissible strings	79
4.1	Context-free grammars	85
4.1.1	Definitions	85
4.1.2	Examples	85
4.2	An excursion to linear-indexed grammars	88
4.2.1	Definitions	90
4.2.2	Examples	90
4.3	Regular expressions (grammars)	93
4.3.1	Definitions	95
4.3.2	Examples	96
4.4	An excursion to linear grammars	97
4.4.1	Definitions	99
4.4.2	Examples	100
4.5	Methods for context-free grammars	101
4.5.1	Structure-preserving and stringset-preserving CFGs	101
4.5.2	Normal forms and binary CKY derivation	103
4.6	Methods for regular expressions	108
4.6.1	Some descriptions are stylistically simpler than others	108
4.6.2	Regular relations	109
5	Automata: computing admissible strings	113
5.1	Pushdown automata	113
5.1.1	Definitions	114
5.1.2	Examples	115
5.2	Finite automata	117
5.2.1	Definitions	118
5.2.2	Examples	119
5.3	Methods for pushdown automata	122
5.3.1	Obtaining a PDA from CFG: top-down parsing	122
5.3.2	Obtaining a PDA from CFG: bottom-up parsing	125
5.4	Methods for finite automata	126
5.4.1	Eliminating nondeterminism	126
5.4.2	From regular expressions to automata and back	129
5.4.3	Transducers and regular relations	134

6	Mathematics of linguistic embedding and delineation	137
6.1	Why are syntax and semantics necessarily embedding?	138
6.2	Contingent homomorphisms: putting embeddings together	140
6.3	Contingent homomorphisms and fundamental questions	144
6.4	Why are morphology and phonology sufficiently decomposing and necessarily nonembedding?	146
6.5	Mathematics of delineation	147
6.5.1	The idea of system as an abstraction	149
6.5.2	Open systems, semantics, and choice	153
6.5.3	Closed systems, semantics, and decision-making	154
	References	157
	Index	165
	Name index	165
	Language index	165
	Subject index	165

Preface

The book addresses the next generation of linguists, who have to cope with integrating various fields into their education to thrive both theoretically and practically in the current technologically advanced world. It promotes mathematical thinking for sustained active involvement in the process. Its motivation is to bring together wide-ranging concerns for the new linguists, which happen to widen as technology advances, for a self-contained introduction.

The book assumes prior exposure to first courses on morphology, syntax and semantics; syntax, semantics, and eleventh-grade or freshman mathematics being the minimal set. There is a complementary companion book for exercises cross-indexed by the section numbers of this book.

Linguistic and philosophical reassessment of some well-known mathematical methods and a gentle introduction to some understudied and underrepresented mathematical concepts concerning the basics of knowledge of linguistics are the main emphases. Mathematics does not float above human practices in a timeless and ahistorical realm. I believe that current mathematical practices and their relation to older ones require for the next generation of linguists a new exposure to some well-known and venerable ideas in mathematics to face new challenges.

The ideas behind linguistically relevant mathematical methods have become just as important as the methods themselves for the students because to them the methods seem detached from current issues to be of interest in focused and urgent problems of today. Some critical assessment of the current state of affairs in linguistic theorizing, which I know is a bit unexpected in an introductory book, is offered with exactly that in mind, for a refocus in addition to taking stock. I redraw the picture providing historical and philosophical background, thereby revealing the ideas behind linguistic methods and changing the feeling of being out of the loop in discussions of natural language in a digital life. I hope to instill the joy of abstraction to reconsider the contribution of some well-known methods. Foundational questions such as language acquisition and linguistic diversity appear to be the new frontiers, and active involvement in them requires knowing more than the methods.

The coverage of the book is not encyclopedic. It is an introductory book in textbook style on selected topics and subtopics, a focused introduction, a primer

or companion on selected topics. Its goal is not to overwhelm the reader with references to complete or must-read reading lists in every major topic. It focuses on rethinking some particular well-known mathematical ideas for making further discoveries in linguistics.

There are excellent textbooks for mathematical methods in linguistics, for example Dowty et al. (1981), Keenan & Faltz (1985), Bach (1989), Partee et al. (1990), L.T.F. Gamut (1990), Chierchia & McConnell-Ginet (1990), Cann (1993), Heim & Kratzer (1998), Morrill (2011), Jacobson (2014), Winter (2016). This book encourages mathematical thinking in linguistics to reconsider the methods in a new light and to reconnect with advances in science and technology, at the same time keeping philosophical implications in our minds. It is the idea that mathematics is the art of abstraction laid over objects of study to see how (and whether) we can reach an invariant. This ties with Poincaré's depiction of invariants as admissibility conditions for the variants, the most important of them for linguistics being good descriptions, modeling and computation, with an understanding of the connection between the three. The book hopes to do so without mathematizing linguistics. It is not a book of "mathematics for linguists." It is about the **representation of structure**, and mathematics as its language to compare different kinds of structure to see their limits, with philosophical implications.

For linguistics in particular, mathematics is not only a posthoc means to introduce some rigor to linguistic work. It seeks to unearth new problems and understand the standing ones from a broader view. From this perspective, invariants are not mental objects or universals per se; they are assumed to be necessary to do some work; and, in an empirical domain, they can be insufficient for that work. Variants are consequences of the invariants; they are considered to be sufficient if well-developed but they are not necessary. The interplay of necessity and sufficiency is philosophically nuanced. Here the beauty is in the detail, which is emphasized.

The book empirically motivates every mathematical abstraction covered, with examples from morphology, phonology, syntax, semantics, and typology, rather than from either an empiricist or a rationalist standpoint. The study of decision (truth) has been covered well in mathematical methods of linguistics, for example logical methods, model-theoretic semantics and automata-theoretic computational methods. The book promotes for the future linguists adding to this list the study of choice and the study of putting things together, in particular putting together of choice, decision, and structure. Choice can be conceived in many ways; here, it means the subject's deliberate act of choosing a category for an expression to make it meaningful. It will be seen that it is a reworking of what Husserl called categorial intuition, which is leveraged to mathematical intuition

for studying grammaticality and its consequent sense of meaningfulness. Well-known material such as formal language theory, automata theory, model theory, systems theory, and lambda calculi are resituated in linguistics with these additions in mind.

The book promotes the realist position in linguistics. Other schools of philosophy are also covered in some detail. Linguistic reality is in part established by the community for the child and by the child, and by the adult to the extent of affecting grammaticality. Obviously it has an individual dimension (it is not obvious at all that this is an exclusively psychological dimension), and it has a social dimension. Like [Katz \(1985a\)](#), the book assumes that elements of grammar embodying all of this are abstract objects, but objects nevertheless. The subject builds that knowledge. How this process may be abstracted over is our main concern. At certain points in the book the defence of realism may seem opinionated. My concern is how to advance the debate so that we can assess understanding of the problems rather than just their solutions in the complex world we live in and build on ourselves. It may be proven wrong one day, but first let us see what it puts on the table for reasoned interaction.

This may be considered first-order realism, that things may exist independent of our experience of them, and everything else would be up for grabs. It does not follow that everything we create or work with needs us individually to continue to exist. Our knowledge of them and of other things also depends on community experience and intuition. This is where realism and conventionalism meet rather than contradict each other. In that sense the two extreme positions of realism and formalism beg a better understanding. Mathematics is one way to clarify that. The primal assumption is that a mathematical *language* was first invented by humans, not discovered, in the practice of mathematical thinking, then led to discoveries and further inventions. (Perhaps humans rediscovered through their linguistic invention what is already out there mathematically such as $2+2=4$; this is a recurring discussion in the book; see [Linnebo 2017](#) for extended discussion of mathematics and philosophy.)

Unlike Katz however, linguistics is not considered a branch of mathematics. It is not considered a branch of psychology, logic or biology either. Linguistics has independently motivated fundamental questions that cannot be reduced to studies in these fields. These questions are emphasized.

The book points out that reductive attempts and reified attempts in linguistics have been diversions from the fundamental questions. It is suggested that mathematical thinking may help us avoid further diversions and reassess the older ones. In particular, putting things together, the broader perspective, requires mathematical understanding without falling into the temptation of committing

to separate, componential (and, in many cases, nonconstitutive, derivable) micro theories first. An abstract grammar reified is not abstract anymore. Further commitments down the road only exacerbate patch management for invariants and variants.

Contemporary syntactic theories tend to narrowly focus on for example, case, voice, agreement, extraction, lexical insertion, numeration, variation, module interaction etc., with certain reified roles ascribed to morphology in fulfilling these functions related to form, from competition with syntax to subservience or encapsulation in the lexicon. Most of this discussion takes place in a reified terminology. If linguists are right in suggesting that linguistic levels that are postulated are independently motivated empirically, then we have to rise above unnecessary prioritization between the levels.

Mathematical thinking—not just its use—suggests that we can do this without compartmentalizing the levels, if every aspect of form is there to do something about meaning, to adapt an adage from one of the brilliant semanticists of the 20th century, Dwight Bolinger. Micro theories tend to create a comfort zone by causing rapid advances and focused discussion, in many cases at the expense of eschewing repercussions and connections outside that zone. Putting things together project combined with mathematical thinking which is promoted in the book attempts to avert this trend to maintain the idea of grammar as the locus of connectedness of everything related to grammatical structures. Arguments against premature reification of a linguistic theory will be frequent. In fact the book is an introduction for not going down that path. In this regard the reader of the book is exposed to a critical review of philosophy of theoretical linguistics.

The most revealing descriptions of structures for studying linguistics' fundamental questions are suggested to be mathematical, in the form of invariants (admissibility) and variants (methods). Linguistic objects are by their nature variants, and their abstraction (grammar) speaks of a need to identify their invariant. Mathematics is our best tool for studying the invariants in any field. Understanding and explaining the fundamental linguistic problems this way means turning mathematical abstraction to a thinking tool for linguistic exploration, not just rigorous posthoc capture of linguistic ideas.

Mathematical methods may provide the best description of linguistic structures; however, they are necessary but not sufficient to understand and explain language. This is a recurrent theme in the book. Mathematics is one way to understand the difference and the connection between (i) mechanical aspects (i.e. how 'how' relates to 'what') and (ii) subjective aspects of grammaticality, which is a central concept in any modern theory of grammar. Neither aspect is sufficient by itself to understand grammaticality, and both of them seem to be necessary

for studying natural language grammars without making a preemptive jump to mental grammars. This is emphasized in the book.

Mathematics is also one way to break the pendulum swing between ahistoricism and historicism in science making by providing an abstract reflection of current science, especially when it is not tied to methods dryly. It is inevitable for an introductory book to provide some methods in detail. The goal of the book is to convey a sense of philosophical, theoretical and empirical purpose for them.

Mathematical thinking for linguistics that is suggested in the book promotes connectedness as the main concept behind invariants for linguistic universals to avoid divergent commitments to universal ontologies for psychological, biological, computational or physical objects. After all any native speaker-signer has the understanding of connectedness of her knowledge in the grammar, no matter what the categories and mechanisms are presumed to be. Consequent variants need an empirical theory, from morphology to typology, the need which is clarified throughout the book. I hope that the book's attempt to do more than using mathematics in linguistics for rigor is appreciated by the reader.

Below is a roadmap to navigate through the book and to connect the ideas and methods in it. They are too much spread over the chapters to give a chapter map.

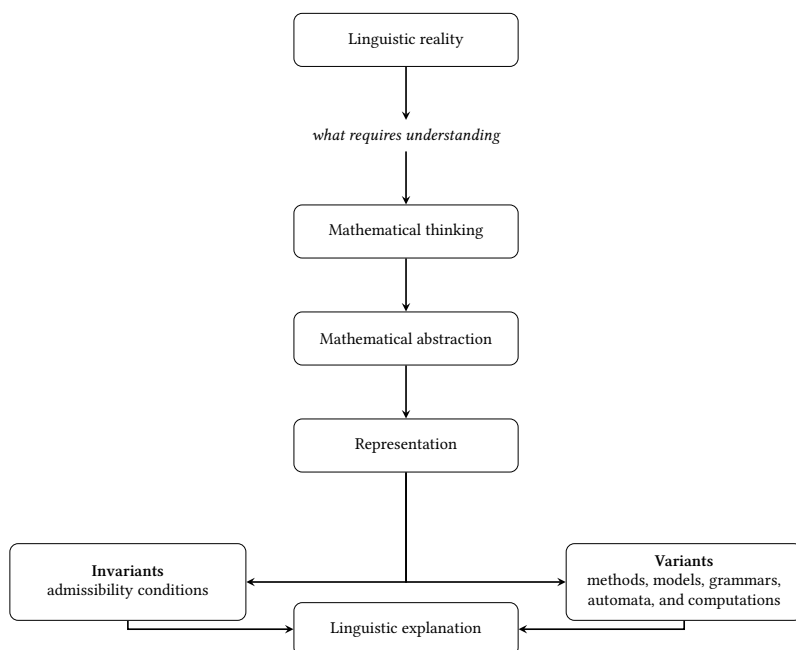


Figure 1: Roadmap of the book.

Abbreviations

Abbreviations used in glosses

Grammatical abbreviations follow the Leipzig Glossing Rules.

1,2,3 first, second, third person

A agent

ABL ablative

ABS absolutive

ACC accusative

ACT active

ALL allative

ASP aspect

AUX auxiliary

CAUS causative

CL classifier

COM comitative

COMP complementizer

COND conditional

COP copula

DAT dative

DEF definite

DEM demonstrative

ERG ergative

F feminine

FOC focus

FUT future

GEN genitive

IMP imperative

Abbreviations

INF	infinitive
INS	instrumental
LOC	locative
M	masculine
NEG	negation
NMLZ	nominalizer
NOM	nominative
PASS	passive
PERF	perfective
PLU	plural
POSS	possessive
PRES	present
PROG	progressive
PAST	past
REFL	reflexive
REL	relativizer
SG	singular

Common acronyms and abbreviations

CG	Categorial Grammar
CCG	Combinatory Categorial Grammar
CFG	Context-Free Grammar
CNF	Chomsky Normal Form
CKY	Cocke–Kasami–Younger algorithm (also, CYK)
DFA	Deterministic Finite Automaton
DS	Deep Structure
ECP	Empty Category Principle
FA	Finite Automaton
FST	Finite-State Transducer
GB	Government and Binding
GPSG	Generalized Phrase Structure Grammar
HPSG	Head-driven Phrase Structure Grammar
INUS	Insufficient but Necessary part of an Unnecessary but Sufficient condition
LIG	Linear Indexed Grammar
LG	Linear Grammar
LF	Logical Form
LFG	Lexical-Functional Grammar
LTAG	Lexicalized Tree Adjoining Grammar
MG	Minimalist Grammar
MP	Minimalist Program
NFA	Nondeterministic Finite Automaton
PDA	Pushdown Automaton
PTQ	“The Proper Treatment of Quantification in Ordinary English”
PF	Phonological Form
REG	Regular Grammar
RE	Regular Expression
regex	Regular Expression
SPE	<i>The Sound Pattern of English</i>
SS	Surface Structure
TAG	Tree Adjoining Grammar
UG	Universal Grammar
X-bar	X-bar Theory

Common mathematical symbols

$\emptyset$	empty set
ϵ	empty string
$\in$	set membership
$\notin$	nonmembership
$\subseteq$	subset
$\subset$	proper subset
$\cup$	union
$\cap$	intersection
$\times$	Cartesian product
$\mathcal{P}(A)$	powerset of set A
A^*	Kleene closure of set A
A^+	positive closure of set A
$ A $	cardinality of set A
$ x $	length of the string x
$\mathbb{N}$	natural numbers
$\mathbb{R}$	real numbers
$f : A \mapsto B$	function f with domain A and codomain (range) B
$x \mapsto y$	mapping, from x to y
$f \circ g$	function composition, composing function $f : B \mapsto C$ with function $g : A \mapsto B$, to obtain $f \circ g : A \mapsto C$
λ	lambda abstraction
$\vdash$	derives (logic), one-step computation (automata)
$\models$	satisfies/models
$\equiv$	equivalence
$\approx$	approximation
$\Rightarrow$	implication (logic), derives (formal grammar)
$\rightarrow$	implication (logic), rewrite (formal grammar)
$\Leftrightarrow$	equivalence (logic)

1 Basics revisited

Mathematics is the art of **abstraction**. More specifically, it is about the amount and kind of abstractions we need to arrive at an invariant.

Invariants do not compute. They admit. Variants do not admit. They compute. Maybe. A gentle introduction to the relevance of the distinction to linguistics—and the caveat—is the subject matter of the book. If it gives the feeling of wanting to read more, I would consider it mission accomplished.

Invariants are what Poincaré called **singulars**, structural relations that are preserved under mathematical transformations. **Variants** are his **particulars**; contingent, empirical instantiations. **Science**, then, does not discover facts; it discovers stable relations that remain invariant across facts. This distinction is the organizing principle of the entire book.

Most of these “facts” may exist independent of our experience; but, scientific discovery is a human practice, and some of these facts are *constructed* by us. This is a recognition of **physical reality** and **mental reality** as loci of the variants, and **mathematical reality** as the locus of invariance. Thus a part of the invariant *may* be about the facts and abstractions invented by us. The book adopts an unapologetically old-fashioned realist stance on these aspects, and its philosophical angle on everything linguistic may perhaps be excused because of such deliberations.

Linguistics asks **fundamental questions** about the nature of the natural language phenomenon, starting with the following:

- (1) a. Why are children able to **acquire** language? (**ontogeny**)
- b. What is the relation of **adult grammars** to **child grammars**? (**continuity**)
- c. Why are there so many different languages when **dependencies** they manifest seem to be limited? (**diversity**)

We need a mathematical roadmap to go from **admissibility** to **empirical theories** about **natural language**, and from that to **modelability**, transparently, so that theoretical claims about (1) can be assessed by believers and nonbelievers alike. Some mathematical concepts are preliminary in this way of studying. Invariants and variants are good places to reach, starting with the basics. We revisit some well-known mathematical basics in this chapter, with linguistics in mind.

If you are a student of linguistics, the distinctions made above may sound like schisms about abstraction and the world. They would probably make you wonder what they would mean in the study of the world's languages, past, present and future, which seems very empirical, as in (1).

The book is an attempt to distill the notion that they are not schisms; but, first things first. One answer to your understandable concern has been that mathematical methods are our most precise ways to *solve* problems and to convey that knowledge to others. This book suggests that, although this has been a very venerable practice, it has not made it easier to see the thinking and ideas behind these methods so that we can understand the problems to be more than just a collection of solutions.¹

This book is based on the premise that we need mathematical thinking to be able to arrive at this level, one that does not just live in a comfort zone or rigor zone but reveal connections to problems themselves as well as the methods. Mathematical thinking in any domain is a disciplined search for an invariant.

An invariant is not a reification of anything, reducing some idea say to mental objects or to some kind of ontology. It is just abstraction of an intuition to put it under discipline. It simply means what can remain unchanged in studying objects and processes that may vary. In the case of linguistics, an invariant would not be an abstraction for abstraction's sake. (It is not true for mathematics either.) It would not be a psychological universal, properties of immanent logic or self-evident sense/feature geometry. These properties do not *necessarily* follow, but they may contingently do so.

If an invariant shapes the variants by providing admissibility, then these variants must be some kind of abstract elements and processes that can be made concrete if one wishes to do so, as we do in modeling and problem-solving. This is going to be the nowadays missing connection to well-known mathematical methods that are covered in the book.

They are based on some classical abstractions in mathematics: sets, strings, relations, functions and recursion. They are covered first from the perspective of the book's premises with linguistics in mind for their particular relevance. More is added on top of these abstractions, which are covered subsequently in the chapter. We assume that the sections of the chapter are read in sequence.

¹Marr (1977) was one attempt to do that for what he called **information processing problems**.

1.1 Sets

A set is a **collection** of elements. As such it is an abstraction, implicit or explicit, because there must be a reason to put the collection together; things in nature by themselves do not need it. Whether these elements are **objects** or some other kind of thing, whether they exist, relate to each other, does not matter. What matters is that the act of collecting itself is an abstraction.

Set abstraction is by convention denoted using curly braces:

(2) $\{ \}$

For example, a set with four elements a , b , c and d can be written with comma as the element separator:

(3) $\{a, b, c, d\}$

This notation is convenient for **finite sets** but problematic for **arbitrarily large sets**. Readers will notice that we are trying to avoid the term **infinity**, as in would-be actually infinite sets, a concept which is also problematic.²

Conventional linguistic wisdom about sets is that anything finite is inherently boring no matter how large; just list the elements and be done with it. Modern science and technology prove that this may not be the whole story. Just think about searching a set for a certain value. Even if the set is very very large, brute-force search would work, but not to everyone's satisfaction. If the set itself is structured, for example if it contains chemical compounds, all and only them, we must have come up with a theory for them to understand their structure and now designing an algorithm for everyone to search for them. Why would we design an algorithm to search for them if the set is finite? After all we could have gone over them by brute force if we can afford a nuclear power plant. We do so because we have the urge to manifest our understanding. Enumerating all compounds would not count as a theory in chemistry, no more than listing all the

²There is less talk of actual infinity even in physics nowadays than used to be. There is more talk of cycles of collapses and bangs, for example Penrose (2013), which bring eternal properties to the fore than discussions of actual infinity. We are told by physicists that the universe is big, very big, probably nonarbitrarily large. Even in physics, "very large" and "unbounded" are often more informative than assuming actual infinity. Incidentally, Penrose's (2006) book can be read as a gentle introduction to mathematics which is not covered in this book, as well as an introduction to modern physics and mathematical thinking in it. In terms of philosophy of science however, we can take it with a grain of salt. Here is the opening paragraph of the book [my emphasis added]: "What laws govern the universe? How shall we know them? How may this knowledge help us to comprehend the world *and hence guide its actions to our advantage?*" The part that I emphasize is not everyone's cup of tea in science.

particles in the universe would count as a theory in physics. In case of linguistics, the number of well-formed sentences in any language is potentially infinite, in one Humboldtian idealization, but we would not be content with brute-force search to understand a language; we would write a grammar. Infinity is not the interesting property here, but structured arbitrary largeness would be. It is not the method but the idea that makes the difference.

Arbitrarily large qualification in abstractions means if someone has an enormous size in mind, say in terms of number of elements in the set, we can up the ante by thinking something even larger and still be in the set. For example, the set of even nonnegative counting numbers is arbitrarily large because there is no end to evenness. If someone claims to know the biggest even number, we can think of a bigger even number. Cantor proved to us that the notion of infinity is actually—pardon the pun—more problematic than we thought: Some things are more infinite than others even when we idealize them as sets. For example, *given* two natural numbers, there are only countable and finite number of natural numbers between them; but, given two real numbers, there aren't countable and finite number of real numbers between them. Surely this is a **mathematical reality**, but not necessarily **physical reality** or **mental reality**. In 'arbitrarily large' terminology, natural numbers can be called arbitrarily large and countable, and real numbers can be called arbitrarily large and not countable.

A set with finite elements means that the set has elements *up to* a definite counting number, not arbitrary number. These are shown in the following (4a), and (4b).

- (4) a. $\{0, 2, 4, 126, 8, 1040, 0\}$
- b. $\{0, 1.2, a, \textit{Picasso}\}$
- c. $\{0, 2, 4, \dots\}$

Arbitrariness in (4c) is informally shown for the time being with dots. The size of the set in (4c) is arbitrarily large, not infinite; (4a) has six elements, not seven—notice two 0s that might suggest seven. **Set membership** as an abstract concept assumes by convention that if something is a member of the set, we better not count duplicates in finding its size because otherwise even a very simple and finite set may then look arbitrarily large. For example, if the set in (5a) is meant to be *the* set containing the element called 1 only, if duplicates were to be counted for size, what would stop us from writing it as an arbitrarily large set where every element is 1, as in (5b)?

- (5) a. $\{1\}$
- b. $\{1, 1, \dots, 1\}$

The **size** (or **cardinality**) of a set, its number of elements, is therefore an abstract property as well because it is based on this understanding of membership.

The common convention is that there is no ordering relation between the members of the set for comparing them. If we want comparison, we would need another abstraction. An ordered set is therefore meaningless, because there is no notion of **next** in a set (next element, next object, etc.). If we want order in a collection, we can use **lists** and **strings**, which we will cover later in the chapter.

There is however the notion of *next size* for sets, from null to one, two, etc., but not say 1.2809. This notion allows us to distinguish arbitrarily large sets which are **countable** and **uncountable**. Countability is defined over a well-known set which is ordained with a name, the set of natural numbers, informally shown for the time being in (6a) using the dots notation. Uncountability is defined over another well-known set, the set of real numbers in (6b). Another special name for a special set is (6c) for the empty set.

- (6) a. $\mathbb{N} = \{0, 1, 2, 3, \dots\}$
- b. $\mathbb{R} = \{0.01, 3.141519, 1.2, 1.200000002, \dots\}$
- c. $\emptyset = \{\}$
- d. $\{\emptyset\} \neq \emptyset$

To summarize, here is the take-home idea about **countability**:

- (7) Countability is not about being large. It is about admitting a notion of *next*.

Notice the levels distinction in set abstraction. (6d) is not the empty set because it contains an element in a set although the element is the empty set. Sets can have sets as elements, in fact anything as elements. This will also prove crucial for abstracting over **strings**.

Using the notion of *next* is natural for $\mathbb{N}$: For any member of $\mathbb{N}$ say n , denoted $n \in \mathbb{N}$, the next of n , $n + 1$, is also in $\mathbb{N}$. This is meaningless for $\mathbb{R}$ because, given any two elements say x and y in $\mathbb{R}$, there are arbitrary number of elements between them which are also in $\mathbb{R}$, whereas given any two elements x and y in $\mathbb{N}$, there are finite number of elements of $\mathbb{N}$ between them.

It is also common to use the **set generator** notation for abstracting over elements. For example, (6a) can be written as (8a). We shall see later yet another way to write it using recursion. Here we note the superficial similarity of defining $\mathbb{R}$ in a similar way in (8b).

- (8) a. $\{n \mid n \in \mathbb{N}\}$
- b. $\{n \mid n \in \mathbb{R}\}$

Recursion will be more explicit about the difference. It will turn out that using the set generator notation for $\mathbb{R}$ would be misleading because although $\mathbb{N}$ can be thought to be generated by a recursive definition, there is no such generator for $\mathbb{R}$. This is an early reminder in the book that once we are able to read between the lines in representations our ideas and arguments may clarify, sometimes without us noticing the change.

This is important for understanding the set operations giving more **set objects**. There are five operations that are quite common.³

- (9) a. Set union: $A \cup B = \{x \mid x \in A \vee x \in B\}$
- b. Set intersection: $A \cap B = \{x \mid x \in A \wedge x \in B\}$
- c. Set difference: $A - B = \{x \mid x \in A \wedge x \notin B\}$
- d. Cartesian product: $A \times B = \{(x, y) \mid x \in A, y \in B\}$
- e. Power set: $\mathcal{P}(A) = \{\text{the set of all subsets of } A, \text{ including } A \text{ itself and } \emptyset\}$

It looks like an evenhanded innocent terminology but if one of the sets A or B is something like $\mathbb{R}$, then there is no known way to check whether the value of the placeholder we called x in (9) would be in that set—we know no way of saying that two arbitrarily chosen real numbers are the same. Apparently, what we understand from the notion of grammaticality in linguistics is not something like this if that amounts to being in a set of grammatical sentences; we always know whether something is grammatical or not.⁴

Notice also the subtle distinction in potential values for the placeholder x : In (9a–d) it corresponds to a value from the sets, whereas in (9e) its value is necessarily a set; for example:

$$(10) \quad \mathcal{P}(\{a, b, 0\}) = \{\emptyset, \{a\}, \{b\}, \{0\}, \{a, b\}, \{a, 0\}, \{b, 0\}, \{a, b, 0\}\}$$

The set $\mathbb{N}$ is the very definition of countability. It does not follow that anything related to $\mathbb{N}$ would be countable. For example, $\mathcal{P}(\mathbb{N})$ is uncountable. Although we can think of $\mathbb{N}$ as being generated using the notion of *next*, nothing can generate $\mathcal{P}(\mathbb{N})$; what we can do at best is a characterization of it, as Cantor did.

³We are assuming $\vee$: logical or, $\wedge$: logical and; $\subseteq$ subset relation; $\subset$: proper subset relation.

⁴Generative grammar, which started this way of thinking, has lost its appetite for evaluating sets of expressions for explaining linguistic constraints, concentrating instead on feature geometry and what are called derivational constraints. Once more we see that instrumental use of mathematical methods may be here now and gone tomorrow, but identifying admissibility conditions as a mathematical thinking tool for new problems over and above methods must still be there. That is what invariants are for.

We can relate the distinction of $\mathbb{N}$ and $\mathbb{R}$ to structures in natural language. Imagine putting sentences of any natural language in equivalence sets depending on the number of words in them, whatever the notion of wordhood is in the language (or the morphemes, phonemes). We would have *the* sets containing all one-word sentences, two-word sentences, etc. We know of no language where there are such sets of size n and m , but no set of sentences of some size in between. Linguistic structures appear to be inherently countable because there is always the notion of ‘next’. That is relevant to any kind of linguistic theorizing, not as a method but as a necessary property in understanding of the admissibility conditions. However, uncountability is still a relevant abstraction, as we shall see later.

Sets provide abstraction over collections. Yet, a **grammar** is not a collection. A grammar is an abstraction that constrains collections. Let us see what these collections can be so that we can understand the constraints.

1.2 Strings

It is tempting in mathematical methods to define strings over the set operation of Cartesian product because it is an ordered notation, just like strings. However, strings are based on another set operation—for reasons we shall come to after (15), called the **concatenation of sets**. This clarification has empirical linguistic consequences.

Let us define the operation first. Set concatenation is set of strings formed by concatenating the elements of the set involved:

- (11) Set concatenation of sets A and B , denoted $AB = \{xy \mid x \in A, y \in B\}$

What is xy ? If A and B are **stringsets**, that is, sets whose elements are strings not something else, it is the concatenation of the string x with string y , which is ordered, that is, it is not necessarily the case that $xy = yx$. From the identity element we know that $A\{\epsilon\} = A$, and $\{\epsilon\}A = A$. Note the curly braces, because what we concatenate is sets. If A and B are not stringsets, then xy is **set concatenation** once more, and here too, the order makes a difference. For example, just as we have $\{a, b\}\{0, 1\} = \{a0, a1, b0, b1\} \neq \{0, 1\}\{a, b\} = \{0a, 0b, 1a, 1b\}$, we have $\{\{a, b\}\}\{0, 1\} \neq \{\{0, 1\}\}\{a, b\}$.

We now have to define strings. Here it is important to note that order and repetition are significant. That is, **strings** are *not* sets. Since repetition is significant for length, we start with the simplest string, the empty string (12), of length zero, which is the identity element of set concatenation in which the notion of power

of a set is based. It is the only string whose repetition does not accumulate, for example, $\epsilon\epsilon\epsilon = \epsilon$. Nothing else is special about it besides being the only string of length 0.

(12) The empty string: ϵ

For any set A , we have the following property:

(13) $A^0 = \{\epsilon\}$. Notice that $\{\epsilon\} \neq \emptyset$

Null strings are common in linguistics, sometimes called **empty categories**. Caution is in order here. It is one thing to say that a string is of length zero, which we can check against the admissibility of strings, that is, against their invariant, and another thing to say that a category based on it is empty. This must be a variant if we have no notion of category admissibility.⁵

We can see the connection (and distinction) of set concatenation to/from **power of a set** (which is distinct from the powerset of a set), which is its size (i.e. **cardinality**):

(14) A^n : for set $A = \{a_1, a_2, \dots, a_m\}$, for some $m, n \in \mathbb{N}$, $A^n = \overbrace{AA \cdots A}^n$
The **power (size, cardinality)** of A is m , denoted $|A| = m$.

Consider the set built from A : $A' = \overbrace{\{a_1\}\{a_2\} \cdots \{a_m\}}^m$. Then $|A'| = 1$, and the size of its element does not depend on m .

The power of A is not about the size of its atoms but number of atoms. That is, every a_i counts as one for size, including the empty string as a member. They are atomic. If we want structure (i.e. nonatomism) in an element in the world of sets, we must cast it as a set object; see (9) and (11).

From here we can reach **arbitrarily long** (not large) strings, relating set concatenation to **string concatenation**:

(15) u^* , for $u \in A$, is a **string** from A^* . This is the **Kleene star**.

$$A^* = \bigcup_{i \in \mathbb{N}} A^i.$$

By definition, $\epsilon^i = \epsilon, i \in \mathbb{N}$.

The **length of a string** u is denoted as $|u|$ when no confusion arises.

⁵**Categorical grammars** have that notion. An empty category is meaningless in that way of thinking, but an empty string is not.

For example $\{a, b\}^3 = \{a, b\}\{a, b\}\{a, b\} = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$. Therefore **sets** can be **arbitrarily large**, and **strings** can be **arbitrarily long**.

If we were to construct strings out of the Cartesian product, the zero product for any set would be a **singleton set** by convention, just to keep products going uniformly over $\mathbb{N}$, the very definition of countability. A singleton is a set with one element. It being a set, it would have no notion of order whether it has one element, many, or no elements. That is why string concatenation is not defined over the Cartesian product. We want our strings to be ordered because that matters in natural language. It is necessary, not just convenient: ab and ba may not be the same objects. **String admissibility** (Chapter 3) for natural language strings crucially depends on this mathematical property.

Notice the exception clause following (11) concerning stringsets, arising from notational peculiarities, from our tendency to use the same notation for set concatenation and string concatenation. What is so special about strings? If we are concerned with linguistic invariants, that would be equivalent to studying which objects count as admissible. These objects in linguistics are expressions, strings with structure, various kinds of structure, for example phonological, morphological and syntactic structure. Therefore we must make up an abstraction which can help us study such objects, their length and their makeup. How we deal with these objects is the point of contention, not the fact that they are ordered. *Generating* a string would therefore be a concern for the **variants**, that is, for methods, not the **invariant**.

Strings introduce order. Grammar studies the admissibility of ordered structures.

1.3 Relations

It is common practice in emphasizing mathematical methods to think of relations as a species of relaxed functions, relaxed not to insist on totality and uniqueness of a mapping over sets. This is only half the story. **Relations** are constitutive and functions are derivative concepts.⁶ For example:

(16) A **binary relation** on A and B is a subset of the Cartesian product $A \times B$.

No more no less. This is not a mapping (i.e. not another abstraction), it is a constraint; notice the word **subset**. We see in the next section that functions are not

⁶We say more about functions in the next section. Reordering of the common practice of defining functions first relations later—with exceptions of course—is reversed because of this reason.

constraints but mappings. A ternary relation on the sets A , B and C is a subset of $A \times B \times C$, and so on. Notice that although $A \cup B = B \cup A$ and $A \cap B = B \cap A$, it is not always the case that $A \times B = B \times A$. This ordering constraint on the product is one source of confusion thinking about strings as based on products. As we have seen in the previous section, they are not.

If relations are constraints, can they have discernible properties? There are well known relation types whose properties would look awkward for functions. They are:

- (17) Let R be the name of the subset of $A \times B$, A and B not necessarily distinct. It is common to write relations as infix operators on values from A and B ; A -value on the left of R and B -value on the right, for example aRb , $a \in A$, $b \in B$. R is then
- a **reflexive** relation if aRa holds, that is, the tuple $(a, a) \in R$, for all values a . The implication here is that this R is a subset of $A \times A$.
 - an **irreflexive** relation if aRa does not hold for any a .
 - a **symmetric** relation if both aRb and bRa holds, for all a, b .
 - an **asymmetric** relation if whenever aRb holds, bRa does not.
 - an **antisymmetric** relation if aRb and bRa implies $a = b$ for all a, b .
 - a **transitive** relation if aRb and bRc implies that aRc holds, for all a, b, c .

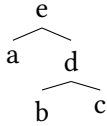
Some everyday examples are:

- (18) a. R ='has the same birthday as' is reflexive.
 b. R ='is brother of' is irreflexive.
 c. R ='is married to' is symmetric (at least in the way marriage is understood in most of the world; recall that relations are constraints).
 d. R ='is mother of' is an asymmetric relation.
 e. R ='is subordinate of' is antisymmetric. If a is a subordinate of b and b is a subordinate of a , then a and b must be the same persons (at least in the world we live in; recall that relations are constraints).
 f. R ='is older than' is transitive.

Another well-known relation is from linguistics, the **c-command**:

- (19) In a tree, x c-commands y if x does not dominate y and the first branching node that dominates x also dominates y .

Dominance itself is antisymmetric and transitive; c-command is neither antisymmetric nor transitive. These properties can be seen from the following tree:

- (20) a.  D: Dominance relation C: c-command relation
- b. aDa bDb cDc eDa eDd eDb eDc dDb dDc
- c. aCd dCa aCb aCc bCc cCb

We shall see in the next section that ‘**first-branching**’ must be a function not a relation, otherwise this definition would not work.

At this point it is not at all clear that something belongs to an invariant if it is relational and variant if functional. This is not what singulars and particulars are about, as we shall see subsequently.

So what exactly is a **tree**, the symbol of dominance? One is tempted to give a definition, for example being a collection of composable, that is, dependent, functions. We leave that to next section. It is important at this point in the book to start delineating invariants from variants. Invariants are **admissibility** properties which really do not need a formalism but intuition so that anyone with interest can take an object and assess whether it is admitted.⁷ For trees it can be as informal as:

- (21) A tree has the following properties.
- There are nodes, and links indicating parenthood (ontology).
 - Every nonroot node has exactly one parent (dominance).
 - There is one overall root (uniqueness of hierarchy).
 - Every internal node must have a child or children and a parent (connectedness).
 - Leaves have no children (terminus).

This is it. Sister relations, whether we read dominance as a relation looking top-down or function looking bottom-up, c-command, labeling relations on nodes and links, etc. are all **variants** *because* they presume admissibility. Notice that none of these properties presupposes a particular notation.

Can dominance be a function then? Hint: Look at dominance from bottom-up, say as *child-of*; for example, a is a child of e in (20a). It will turn out to be a function, also giving grandchildren etc.

⁷In fact, history of science is full of early failures partly arising from overformalization of invariants.

Why do we consider relations to be more primitive than functions? Because there is absolutely no necessary property of a relation. (17) is just a list of common patterns that deserve a name. Not so for functions. Notice also another lesson learned, that being more primitive does not *ipso facto* make something **invariant**. Dominance is a relation, so is c-command; dominance is a candidate for invariance (21b), c-command is not. These are not formalisms or formal concepts; they are first and foremost intuitive concepts.

Relations express constraints. Functions express mappings. Linguistics needs both.

1.4 Functions: named and nameless

Functions are relations under discipline. The discipline is introduced by what is called a **mapping** in mathematics. However, treating relations as defective functions obscures the role of constraints and admissibility in linguistic theory.

A mapping is a systematic association between objects of one kind and objects of another. As such it is abstraction. Therefore relations are *not* mappings, because they are constraints, not associations. Hence functions cannot be a species of relations. Certain kinds of mappings are called **morphisms**, which are mappings that preserve structure up to a desired level (identical structure, called **isomorphisms**, or similar structure, called **homomorphisms**).

Isomorphism is a much stronger discipline than homomorphism, not only in terms of computation but also from the perspective of knowledge. Two structures are **isomorphic** if there is a one-to-one and onto mapping from one to the other and vice versa. (We get to these concepts next; see (22).) In other words, we must know *two* structures well enough to establish an isomorphism. One example is odd and even natural numbers. They can be isomorphic because we can define a one-to-one and onto function from odd numbers to even numbers, for example $f_o(2n + 1) = 2n, \forall n \in \mathbb{N}$, and a one-to-one and onto function from even numbers to odd numbers, for example $f_e(2n) = 2n + 1, \forall n \in \mathbb{N}$.⁸ There is some subtlety here: Notice that f_o is not defined over all natural numbers but odd natural numbers (say $\mathcal{O}$), and f_e is defined over even natural numbers (say $\mathcal{E}$). Hence $f_o : \mathcal{O} \mapsto \mathcal{E}$ and $f_e : \mathcal{E} \mapsto \mathcal{O}$. Then they are isomorphic. If they were

⁸Pythagoreans would disagree with this isomorphy. They have a very geometric interpretation of numbers. Oddness means harmony therefore goodness because with odd number of basic obs they can build perfect squares, a symbol of harmony, whereas with even number of obs they cannot, indicating disharmony. It just shows once more that morphism, therefore isomorphism, therefore abstraction, is in the eye of the beholders in its relation to their invariant (harmony).

defined over all natural numbers, they would not be isomorphic but homomorphic because they would not be total and onto functions. We are nitpicking about these aspects because in many cases in linguistics, misunderstandings arise from overinterpretations.

A **homomorphism** is a mapping that preserves structure. It can be one-to-one, onto, etc. but it does not have to be. It is a *one-way* preservation of structure. It just preserves the operations on the input in the output structure. For example, let $g(n)=n \bmod 3, \forall n \in \mathbb{N}$. Addition is homomorphically preserved in this structure; doing the operation before or after it makes no difference: $g(a+b)=(a+b) \bmod 3=(g(a)+g(b)) \bmod 3$. The range of g is $\{0,1,2\}$, what remains when a natural number is divided by 3. Function g is clearly not isomorphic because many numbers are mapped to 0, e.g. 6, 9, 12, so it is not one-to-one, and for example 0 backmapped to natural numbers (to find out what divides 3 to no remainder) is not a function.

We are usually told that this means that in a homomorphism doing the operations after the mapping is same as doing the operations before the mapping, but something more important than such liberties is at stake for **empirical sciences**: We do not need to know two structures in a homomorphism. If we know one structure and guess the other one from that structure in a way we can assume to be structure-preserving, we do not need to know the other structure.⁹ Identity necessarily implies similarity, but similarity does not necessarily imply identity.

Implications for example for study of **language acquisition**, which is the most **fundamental question** in linguistics, are profound. If we assume that all human languages are acquirable because they arise from structures which are sufficiently homomorphic to an admissible semantic-conceptual structure that is available to every child at least during the **critical period** (Lenneberg 1967, Gleitman & Gleitman 1997), then we do not have to know that structure but we can refer to it by postulating a presumably structure-preserving mapping *to it*; see for example Gleitman & Gleitman (1997) for psycholinguistic models for that and Abend et al. (2017) for computational models. Finding an isomorphism for this problem would be a dream come true, but we are far away from that. A homomorphism would do to support the idea of a common semantic-conceptual base.

Back to basics. We start with the classical view of functions with names, say f, g, h etc., which is a common naming convention:¹⁰

⁹That said, there isn't magic in every homomorphism. Take for example $g(n)=0, \forall n \in \mathbb{N}$. It preserves addition: $g(a+b)=0$, and $g(a)+g(b)=0+0=0$, therefore $g(a+b)=g(a)+g(b)$, and it is homomorphic. But it completely eliminates structure. Good homomorphisms need good designs—just like abstractions; there is no intrinsic value in them.

¹⁰It is just a convention. We could name them $\%\$$ or $a271$ if we liked, as long as we remember

- (22) Let A and B two set expressions, not necessarily distinct. They can be simple sets or sets formed by a combination of the set operations.

A mapping named f from A to B denoted as the following is a **function** if any element of A is mapped to at most one element of B :

- a. $f : A \mapsto B$. (Using ' $\rightarrow$ ' for ' $\mapsto$ ' is also common.)
- b. A is called the **domain** of f . More accurately the domain is the set of things in A for which f is defined. B is called the **codomain** of f . The **range** of f is the set of things in B for which f is defined, that is, $range(f) = \{f(x) \mid x \in domain(f)\}$.
- c. Function f is **total** if every element in A is mapped, **partial** otherwise. Most mathematical functions are total unless specified otherwise. It is same as saying that f in (22a) covers A entirely.
- d. A and B can be simple sets or Cartesian products. Recall that the elements of an n -product are single values, an ordered **tuple** with n elements.
- e. Function f of (22a) is **one-to-one** if every element in its domain is mapped to a distinct element in its range.
- f. Function f of (22a) is **onto** if its range is same as B .

For example, $f(x) = x + 1, x \in \mathbb{N}$ is a function with one argument from $\mathbb{N}$ to $\mathbb{N}$, $g(x, y) = x + y, x \in \mathbb{N}$ is a function with two arguments from $\mathbb{N} \times \mathbb{N}$ to $\mathbb{N}$, and $h(x, y) = (x^2 + y, y^3 + x^4), x, y \in \mathbb{N}$ is a function with two arguments from $\mathbb{N} \times \mathbb{N}$ to $\mathbb{N} \times \mathbb{N}$. The last one does *not* yield two values, it yields a single ordered tuple with two values. There is a tendency to think of functions as computations but they are actually characterizations of set properties selected by the mapping. For example, $f(2n) = n, n \in \mathbb{N}$ is a function mapping even natural numbers to natural numbers. Therefore arguments can be expressions too, characterizing the domain. We emphasize these aspects to encourage thinking of functions not as methods but mappings. When we get the right idea about a mapping, methods follow naturally.

One more concept—not notation—about functions proves very relevant to linguistics. (23a) uses the **exponential notation** to focus on a set of functions, that is, focus collectively. It is equivalent to (23b), not (22a).

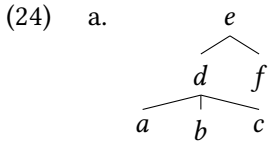
how to refer to them. If you think this is a cumbersome way trying to think of arbitrarily many functions, think about naming them $f_i, i \in \mathbb{N}$, which is also common. We shall see soon that we can live without names so as not to worry about this kind of stuff. How could we then refer to them? Enter another abstraction. This is quite Fregean (see sections 1.5 and 1.7), but it had to wait Moses Schönfinkel and Alonzo Church to materialize.

- (23) a. B^A denotes the set of **total functions** from set A to set B .
b. $B^A = \{f \mid f : A \mapsto B\}$.

For example, $\{0, 1\}^{\{a,b,c\}}$ is the set of functions mapping a, b, c s to 0s and 1s. As a preview of linguistic aspects to come later in the book, we can conceive relative clauses of the world as functions from say S^{NP} to NP^{NP} , which is to say functions from functions of nounphrases to clauses to functions of NP modifiers. It may sit well as a typological invariant.¹¹ Then its variants may be where S^{NP} is realized as syntactically extracting out of a clause to share arguments (e.g. English, Turkish), morphological co-indexing of the arguments in clauses without extraction (e.g. Navajo), and extraction or resumption via a pronoun for co-indexed arguments (e.g. Chinese).

1.4.1 Trees as functions

When we look at **tree** structures in linguistics as manifestations of a hierarchy of some kind, for example syntactic structure or syllable structure, we may get the feeling that functions and relations are two different ways of looking at the same thing. We shall see that they are not; we would not be looking at the same thing. Take the following tree and view it with two relations and one function.



- b. C (child-of relation): aCd bCd cCd dCe fCe
c. S (sister-of relation): aSb aSc bSc bSa cSa cSb dSf fSd aSa bSb cSc dSd eSe fSf
d. p (parent-of function): $p(a) = d, p(b) = d, p(c) = d, p(d) = e, p(f) = e$

From (24d) we also get the following because it is a function. We cannot do the same kind of chaining with (24b) because it is a relation.

- (25) $p(p(a)) = e, p(p(b)) = e, p(p(c)) = e$

We are able to do this because p is a composable function; $p(p(\cdot))$ is the grand-parent function, which is partial, and $p(p(p(\cdot)))$ is the great grandparent function, which is also partial.¹² It may be tempting to think that maybe we cannot do this

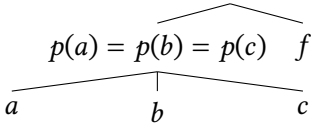
¹¹Variants are good food for work. Invariants are good food for thought. We can wonder why we have participant-sharing in languages of the world but not clause-sharing.

¹²Every node has a parent, but not every node has a grandparent. This is not real life.

with (24b) because C is not a transitive relation. R is transitive (24c), but we still cannot do it. However, we can infer (24b) and (24c) from (24d) alone, precisely because p is the constitutive function of the tree. If $p(x) = p(y)$ then x and y are sisters. If $p(x) = y$ then x is a child of y . Notice that for example $p(a) \neq e$.

We can see the constitutive role of p by redrawing all the nonleaf nodes of (24a) by equivalences on p (26). Now, e and d are just names respectively for $p(f)=p(p(a))=p(p(b))=p(p(c))$ and $p(a)=p(b)=p(c)$; cf. (24a) and (26).

$$(26) \quad p(f) = p(p(a)) = p(p(b)) = p(p(c))$$



In other words, connectedness of the tree is uniquely captured by p ; recall that admissibility condition on trees (21d). Other admissibility conditions are manifested in p too: (21a) amounts to saying that p is definable for any tree. (21b) amounts to saying that p is a function. (21c) amounts to saying that any such p is defined for nodes only, that is, its domain must contain the set of leaves. (21e) amounts to saying that its range does not contain leaves.

1.4.2 Function composition

What is then function composition such as $p(p(a))$, and what is its relation to **function application** such as $p(a)$? In principle function composition shows **connectedness**¹³ of any two functions; note in the following the common set B :

(27) a. Let $f : B \mapsto C$ and $g : A \mapsto B$ be two functions, not necessarily distinct.

b. **Composition** of f and g , denoted $f \circ g$, is the function $f \circ g : A \mapsto C$.

We can read (27b) as follows to understand the notation:¹⁴ Function f needs B to reveal C , and g provides B by mapping A to it. In other words, f **depends** on g , not the other way around. For example, let us assume

¹³The term comes from the **domain condition** of connectability of arrows (functions) in **category theory**: For two functions to be composable as $f \circ g$, the domain of f must be the same as the codomain (range) of g (Mac Lane 1971:1).

¹⁴Writing the result function first in $f \circ g$ has been the standard convention in mathematics since Mac Lane (1971) in standard textbooks (e.g. Galovich 1989). There is an alternative notation which is more common among algebraists and functional programmers, where they write g first and f next to get the result of f ; that is, they would write $g \circ f$. Gallier (2011):118 explains the rationale for this notation.

$$(28) \quad f(y) = y^2 \text{ and } g(z) = z + 1.$$

Then $f \circ g(x) = f(g(x)) = f(x + 1) = (x + 1)^2$. Here, f provides the square function, and g provides the plus-one function.

What about our tree admissibility function p (24d)? We have to assume that such hierarchies have a **fixpoint**, for example the fixpoint of p is a special value which we can call *root*, that is, $p(\text{root}) = \text{root}$. This is where the function stops. The concept is relevant to functions stopping to yield more values or more functions:

- (29) a. The **fixpoint** of a function f is x such that $f(x) = x$.
 b. The fixpoint of a higher-order function H (i.e. a **functional** H), is function f such that $H(f) = f$.
 c. There is a function that can find the fixpoint of any function in (b) if there is one.

Now we can define any constitutive tree admissibility function as follows:

- (30) a. Let $p : N \mapsto N$ where N is a set of nodes.
 b. Function p as tree admissibility is an endofunction with a fixpoint.
 c. An endofunction has domain and range as the same set. The p function has no cycles, that is, no node gives $p(p(\dots p(x))) = x$ except for the special element *root*.

With such a definition p is a composable function: $p \circ p(x) = p(p(x))$. Also, $(p \circ p) \circ p(x) = p \circ (p \circ p(x))$ by **associativity of composition**,¹⁵ which is equivalent to $p(p(p(x)))$, and so on. At the innermost parentheses it may appear that we must have function application (i.e. $p(x)$), in addition to composition up to that point. We shall see soon at §1.4.5 that that too is function composition supplied with function-argument orientation.

In other words, function application is a derivative concept. Then, function composition can be uniquely constitutive to become a good mathematical candidate for the formal specification of tree invariance in (21). Good candidates for invariance not only exhibit chicness, rigor and beauty, they reveal hitherto unnoticed connections.

In order to see the asymmetry of dependence of application on composition for any function, we need to extend our coverage of functions to arbitrarily many functions. The best way to do that is making them nameless but enumerable. Lambda calculus is perfect for this purpose.

¹⁵This associativity is related to **admissibility of composition**. The particulars, for example the verbs as functions, do not have to be associative *because* of this assumption. We may call that **empirical associativity**.

1.4.3 Diagrams of functions

One clarification before we cover lambda calculus. Three notions in the mathematics of functions confuse the newcomer more than anything else: **associativity**, **commuting** and **commutativity**. Associativity concerns grouping of an operation. Commutativity concerns ordering of arguments. **Commuting diagrams** concern comparability of several functions. They are all distinct notions addressing different concerns.

Examples from string functions may be easiest for linguists to see their differences. String concatenation is associative: $x(yz) = (xy)z$ for any string x, y, z . It is not commutative: xy is not necessarily the same as yx . For commuting and associativity, take three string functions that do the same thing, prefixing: $f(w) = aw; h(w) = bw; k(w) = cw$; for some constant strings a, b, c . We have $h(f(w)) = baw$ and $f(h(w)) = abw$, therefore they do not commute: $h \circ f \neq f \circ h$. However, they are associative: $(k \circ h) \circ f = k \circ (h \circ f) = cbaw$. To see that $(k \circ h) \circ f = cbaw$, notice that $((k \circ h) \circ f)(w) = (k \circ h)(f(w)) = (k \circ h)(aw) = k(baw) = cbaw$.

Commuting here may look like the same idea as commutativity, that from $h \circ f = f \circ h$ we must infer both commutativity and commuting. After all, functions are values themselves, as we shall see more clearly in lambda calculus, so that the equation above would be equality arising from commutativity. But, commuting is concerned with different ways of reaching the same result, not just different orders of arguments. Take four functions: $f : B \mapsto C; g : A \mapsto B; h : D \mapsto C; k : A \mapsto D$; such that $f \circ g : A \mapsto C, h \circ k : A \mapsto C$, and $f \circ g = h \circ k$. They commute. For the running example, the diagram is:¹⁶

(31)

$$\begin{array}{ccc} A & \xrightarrow{g} & B \\ k \downarrow & & \downarrow f \\ D & \xrightarrow{h} & C \end{array}$$

The idea generalizes to a set of functions. It is worth remembering:

- (32) A **diagram of functions** is a set of functions that can be studied under composition. A diagram **commutes** if any two **paths** with the same start and end points give the same result. Paths are formed by sequences of **function composition**.

¹⁶Gallier (2011):118 points out that writing $f \circ g$ in text for the result function f and drawing the diagrams from g to f is a bit perplexing. This is another source of confusion in mathematics and computer science.

Philosophically speaking, these different ways of reaching the same result forms the conceptual basis of **model selection** (§1.12).

1.4.4 Lambda calculus

Although its full coverage is not in our scope, the spirit of lambda calculus reverberates in everything linguistic therefore we look at it in some detail; see for example [Barendregt \(1981\)](#) for all its mathematics.

It is indeed a lingua franca for linguistics, as can be seen from its coverage as a mathematical method in many textbooks (especially the lambda operator), for example [Dowty et al. \(1981\)](#), [Keenan & Faltz \(1985\)](#), [Bach \(1989\)](#), [Partee et al. \(1990\)](#), [Chierchia & McConnell-Ginet \(1990\)](#), [Cann \(1993\)](#), [Heim & Kratzer \(1998\)](#), [Winter \(2016\)](#). It is covered as an instrumental mathematical method for linguistics in general.

We shall see that it arises from a conceptual need already mentioned, related to the invariants: It is necessary to address arbitrarily many functions to see the asymmetry of composition and application, to see that application can be reconstructed as composition given function-argument orientation, whereas application alone does not supply the general **connectedness** condition expressed by composition. If that is the case, then the only invariant for connectedness would be function composition.¹⁷

Let us start with a named function, for example $f(x) = x + 1$, where x is the placeholder, a local name. Lambda calculus assumes that what a function does is what matters, therefore the ‘body’ of the function is the focal point. The function says that, given x , the result is $x + 1$. The **lambda operator** can do that without a name: $\lambda x.x + 1$. The material before the dot corresponds to ‘given x ’, and the material after, to the body, viz. $x + 1$. Now let us consider a two-argument function, say $f(x, y) = x + y$. We can think of two ways to namelessly represent what this f does, one taking x first y next, the other the other way around. It is a mathematical way to say that there are two ways to conceive f , none of which is preferable, so we must be able to do both: $\lambda x.\lambda y.x + y$, and $\lambda y.\lambda x.x + y$. The dot notation is assumed to be right-associative¹⁸ by convention, therefore if we want to write the order more explicitly, they are $\lambda x(\lambda y(x + y))$ and

¹⁷There is an empirical linguistic dividend coming from access to arbitrarily many functions as well. The verb can be thought of as the linguistic function par excellence, because given a verb and its participants, we have a good understanding of the predicate and the event indicated by the verb. Namelessness makes functions an open repository for verbs. For them too composition with function-argument orientation would suffice, if the premise of lambda calculus is right.

¹⁸**Associativity of an operator** is exploited in displays of chains of the same operator without

$\lambda y(\lambda x(x + y))$. What is not conventional is that these are two different functions that are necessary to capture what this f does. There is a price to pay for avoiding names and still making explicit what a function does, rather than leave it implicit as in the classical function notation.

This explicitness pays off when we realize that with it we can represent any n -ary function as a set of one-argument-at-a-time functions to make what is intended *very* explicit:

- (33) a. Let $f(x_1, x_2, \dots, x_n) = b$ be a function with n arguments and body b containing zero or more occurrences of these argument placeholders as an expression.
- b. If b has more function specifications in it, the simplest case can be reached by placeholdering them until no more placeholdering abstraction is left in b . Let us assume without loss of generality that f in (a) is such a function.
- c. $\lambda x_1. \lambda x_2. \dots \lambda x_n. b$ and any permutation of all λx_i constitute a set of functions making explicit what f does, without a name. This is called **Currying** the function f , at the same time making it nameless.¹⁹
- d. This result holds because $\lambda x_1. \lambda x_2. \dots \lambda x_n. b$ is same as $\lambda x_1(\lambda x_2. \dots \lambda x_n. b)$, where there is only one-argument, x_1 , and the result is the partial function in parenthesis, to which we apply the same idea repeatedly until the end, that is, until x_n .

Currying can be given a bird's eye view using the exponential notation (23a) for a less procedural understanding. A two-argument function (34a), when Curried on the second argument, is another function (34b).

- (34) a. Let $f : A \times B \mapsto C$.
- b. When Curried on B , f is the function $f : A \mapsto C^B$.
- c. When Curried on A , f is the function $f : B \mapsto C^A$.

There must be *empirical* reasons to prefer one of these functions in the set over the other. Clearly that is an issue for the variants. The invariant (33) gives the space of admissibility.

In a simple description **lambda terms** can be shown to be as follows.

parentheses therefore it is worth defining: An operator say $\bullet$ is right-associative if $a \bullet b \bullet c$ means $(a \bullet (b \bullet c))$. It is left associative if it means $((a \bullet b) \bullet c)$.

¹⁹If we want to be historically more accurate and fair, we can call it **Schönfinkeling**, after the mathematician who invented it, **Schönfinkel** (1920/1924).

- (35)
- a. Assume $x_i, i \in \mathbb{N}$ to be placeholders. They are lambda terms.
 - b. Assume constants such as 1, a etc. as simplest bodies. They are lambda terms.
 - c. If M is a lambda term and x_k is a placeholder, than $\lambda x_k.M$ is also a lambda term.
 - d. If M and N are lambda terms, than MN is also a lambda term, for application of M to N . If M in its outermost structure starts with a lambda, say $M = \lambda x_j.M_b$, then MN can be **beta-reduced** to the lambda term $M_b[x_j/N]$, the square bracket meaning substitution of free occurrences of x_j in M_b by N , reaching $M_b[x_j/N]$. In the other direction, we can **beta-abstract** $M_b[x_j/N]$ to $MN = (\lambda x_j.M_b)N$.
 - e. The notation for the beta in (d) is: $MN = (\lambda x_j.M_b)N =_\beta M_b[x_j/N]$.
 - f. Beta-reduction is what is commonly associated with function application. Actually, it is simplification of lambda terms. Beta-abstraction is commonly associated with function abstraction. Actually it is enlargement of lambda terms. Altogether it is **beta-conversion**. From this perspective, (35c) is **lambda conversion**.
 - g. There are other conversions, called alpha, xi and eta. We leave them to fuller treatments. They behave compositionally, just like beta.

As can be seen, lambda calculus is a theory of structured substitution, not a theory of functions. It could not care less about what $(\lambda x.\$ \# @ ! * x)3$ would mean; it would just say it is beta-equivalent to $\$ \# @ ! * 3$. It is up to the variant, that is, its computation, to worry about what $\$ \# @ ! * 3$ means.²⁰ In lambda calculus, everything is a value, including functions. (In category theory, everything is a function, including values; see for example [Simmons 2011](#). The change of perspective is not merely notational; abstractions and concerns are different.)

Let us look at some examples. $(\lambda x.\lambda y.x + y)3$ beta-reduces to $\lambda y.3 + y$. How do we know that 3 eliminates λx and substitutes for x in free occurrences of x in $\lambda y.x + y$? Because the structure of the lambda term is, without dots and with parentheses, $(\lambda x(\lambda y(x + y)))3$. That is, λx and 3 are at the top level, as lambda configuration of beta, $MN = (\lambda x_j.M_b)N$, assume them to be; λy which is deeper inside is not ‘visible’ to 3. Since there can only be one such leveling relation, beta is deterministic.

²⁰In the terminology of the classic in the field, [Barendregt \(1981\)](#), lambda calculus is “logic-free.” It is an equational theory.

(36) $(\lambda x_j. M_b)N$ is called a **reducible expression**, or **redex** for short.²¹

What about the condition on substituting for free occurrences? This is where lambda calculus shows its structure- and meaning-preserving (i.e. functional) character. For example in $(\lambda x. (\lambda x. x^2)x)3$, the inner lambda abstraction implies that that x is another x . By beta we get $((\lambda x. x^2)x)[x/3] = (\lambda x. x^2)3$, not $\lambda x. (3^2)3$.

1.4.5 Function composition and application: one connectedness

Given these properties, and assuming function bodies for lambda terms, we can have a closer look at composition and application. Named and nameless composition can be contrasted as follows, respectively (a) and (b):

- (37) a. Given f and g where f is the primary (i.e. result-defining) function, their composition is $f \circ g = \lambda x. f(g(x))$.
 b. $(\lambda f. \lambda g. \lambda x. f(g(x)))(F)(G)$ is composition in lambda calculus, if the first function to reduce (F) is the primary one. Otherwise, we can suitably rearrange the lambda terms. We can do this if we know which one is primary.

When two functions F and G are supplied with a value for x , they yield the same functional dependence as $(F \circ G)(x)$. Now we can make “supplied with a value for x ” explicit in the lambda notation; it is $\lambda x. F(G(x))$. If the primary function (F) is a multiargument function, composition composes on its first argument. For example, the following f composed with g , equivalently $f \circ g$, can be revealed in steps:

- (38) a. Let $f = \lambda x. \lambda y. x + y$ and $g = \lambda y. y^2$.
 b. Their composition is $(\lambda f. \lambda g. \lambda x. f(g(x)))(f)(g) =_{\beta} \lambda x. f(gx) =_{\beta}$
 c. $\lambda x. (\lambda x. \lambda y. x + y)((\lambda y. y^2)x) =_{\beta} \lambda x. (\lambda x. \lambda y. x + y)(x^2) =_{\beta}$
 d. $\lambda x. \lambda y. x^2 + y$

²¹There can be more than one redex in a lambda term. In such cases we either consistently reduce the outermost leftmost redex or innermost leftmost redex. These are the only possibilities in lambda terms defined by (35d). If we do them consistently, Church-Rosser theorem says that they reduce to same form if there is one. Take for example $(\lambda x. x + 1)((\lambda y. y^2)(3))$. There are two redexes in it, the whole lambda term itself and $(\lambda y. y^2)(3)$ in it, because their lambdas have arguments at their top level. Reducing the first is outermost leftmost reduction, also known as **normal-order evaluation**, giving $(\lambda y. y^2)(3) + 1$. Notice that the argument (placeholder x) is passed unreduced. Reducing the second is innermost leftmost reduction, also known as **applicative order evaluation**, giving $(\lambda x. x + 1)(3^2)$. Notice that $(\lambda y. y^2)(3)$ is reduced first. Continuing either way consistently, we reach the same lambda term: $3^2 + 1$.

If the secondary function (G) is multiargument, then something has to give in before composition makes sense. This is not a peculiarity of the notation; it is about the nature of connectedness. First, let us see what happens if we try it anyway without doing anything first:

- (39) a. Let $f = \lambda y.y^2$ and $g = \lambda x.\lambda y.x - y$. Let us assume f is the primary (i.e. result-defining) function.
- b. Their composition would be $(\lambda f.\lambda g.\lambda x.f(g(x)))(f)(g) =_{\beta} \lambda x.f(gx) =_{\beta}$
- c. $\lambda x.(\lambda y.y^2)((\lambda x.\lambda y.x - y)x) =_{\beta} \lambda x.(\lambda y.y^2)(\lambda y.x - y) =_{\beta}$
- d. $\lambda x.(\lambda y.x - y)^2 = \lambda x.(\lambda y.x - y) \times (\lambda y.x - y)$

Clearly, that is not what is meant by squaring the same argument; to wit, we have two purported abstractions for the second argument in (39d). The problem can be seen informally as follows. Take $f(z)$ and $g(x, y)$ as two functions as candidates for being the primary (i.e. outermost, result-determining) function. Then $f(g(x, y))$ makes sense because f requires one argument; however, $g(f(z))$ is odd because g requires two arguments.

Although we can compose into fewer arguments naturally, we cannot compose *into* more arguments without extra structure. What would that structure be? In the case of these two particular functions, it would require fixing either the first or second argument of g to make it a function with lesser arguments. For example, this g is namelessly $\lambda x \lambda y. \dots$ for some $\dots$ as the body. Either $\lambda x. \dots$ or $\lambda y. \dots$ with $\dots$ as bodies would make it a function with one less argument. It is easier with $\lambda y. \dots$ to see how that could be obtained, which would just need one supplied argument for the outermost lambda, for example $(\lambda x.\lambda y.x - y)2 = \lambda y.2 - y$. What about $\lambda x. \dots$, where the second lambda abstraction (λy) would be reduced?

One answer may be our first insight into conceiving function application as a form of function composition, although representing composition by applications does not thereby make application constitutive of composition. Let us assume that we are going to fix the second argument of g in (a).

- (40) a. Let $g = \lambda x.\lambda y.x - y$. We want to get say $\lambda x.x - 2$, supplying the second argument as 2.
- b. **Raise 2 to a function:** $\lambda p.p\ 2$. **Raising** here simply means ensuring something to be functional. It will be functional if it weren't, higher-order functional if it were.
- c. Compose the raised function with g : $\lambda z.(\lambda p.p\ 2)(gz)$:
- d. $\lambda z.(\lambda p.p\ 2)(gz) =_{\beta} \lambda z.(\lambda p.p\ 2)(\lambda y.z - y) =_{\beta}$
 $\lambda z.((\lambda y.z - y)\ 2) =_{\beta} \lambda z.z - 2$

In other words, if we want to apply the function to its second argument, we raise that argument and compose it with the function, the argument-taker. Although we would not recommend doing this at home, we have achieved something insightful.

We can compose over second, third and other arguments as well. The extension of the *same* operation, composition, is called the **power** of composition. This power is distinct from power of a set; at least on surface. More advanced textbooks explain it in detail. Overall it means that nameless functions can compose transparently across any number of arguments.

Composition works when the range of one mapping matches the domain of another. That is the mathematical understanding of connectedness. When it does not match, extra structure must be supplied, and that does not necessarily mean computed. This is a property of the invariant. Arity is structure, not bookkeeping. The extra structure arises from knowing that arguments can be functions that connect with functions that look for the argument. Incidentally, that is closest to a mathematical definition of the linguistic notion of **case**. We can do this with functions, not relations. That is because it requires an abstraction, a mapping, not a constraint.

Where does that take us in conceiving function application as a derivative concept? The key insight from preceding material is that **connectedness** is a two-party affair. If functions symbolize connectedness, then arguments can ensure that they are parties by being raised to functions that connect to such functions.

Application then arises as a special case where the function too has to raise to stay as the primary function to face the raised argument. To control the function-argument orientation with raised types we can use the following lambda term:²²

$$(41) \quad \text{Let } T = \lambda x_1. \lambda x_2. x_2 x_1.$$

Function application is in the nameless notation $(\lambda f \lambda x. f(x))(F)(A)$, where F is a function and A is an argument for it. For example applying $\lambda y. y^2$ to 4:^{23,24}

$$(42) \quad \begin{array}{l} \text{a. Let } f = \lambda y. y^2 \text{ and } a = 4. \\ \text{b. Applying function } f \text{ to argument } a: (\lambda f. \lambda x. f(x)) f a =_{\beta} \\ \quad (\lambda x. (\lambda y. y^2)(x)) 4 =_{\beta} (\lambda y. y^2)(4) =_{\beta} 4^2 \end{array}$$

²²This term goes back to Schönfinkel (1920/1924). It has gone through some name changes until T has become the standard name.

²³The result is 4^2 , not 16. Lambda calculus is not an interpreter of functions; it is a theory of substitution. A variant may then compute $4^2 = 16$.

²⁴In some sources the verb ‘apply’ swaps the role of argument and function; for example, it is not unusual to hear 4 applies to $\lambda y. y^2$.

- c. Doing the same via composing raised function and argument:
- d. Raise f to $\lambda p.p f$, which is the same as Tf . Raise a to $\lambda q.q a$, which is the same as Ta . Compose them and apply T :
 $((Tf) \circ (Ta))T$
- e. We get $f(a)$:
 $[\lambda x.(\lambda p.p f)((\lambda q.q a)x)]T =_{\beta} (\lambda p.p f)((\lambda q.q a)T) =_{\beta} (\lambda p.p f)(Ta) =_{\beta} Ta f =_{\beta} f a = f(a)$.
- f. For example $(\lambda p.p(\lambda y.y^2))((\lambda q.q 4)T) = T(4)(\lambda y.y^2) = 4^2$. Same as (b).

This is true for any applicative structure, that is, for any function and its argument to apply. If f is any function and a is its argument to apply to give $f(a)$, then composing $\lambda p.p f$ with $\lambda q.q a$ by keeping the function-argument distinction (by T) has the same functional dependence:

- (43) a. Raise function f to $\lambda p.p f$. It is beta-equivalent to Tf .
b. Raise argument a to $\lambda q.q a$. It is beta-equivalent to Ta .
c. Then compose raised f with raised a and apply T :
 $((Tf) \circ (Ta))T =_{\beta} f a = f(a)$

In summary, although relations are more primitive than functions, functions are indispensable for understanding **connectedness** as an abstract dependency.

Functions are not just constraints. They are abstractions. Composition with function-argument awareness is the only thing connectedness needs as an invariant. Consequent variants in linguistics may be computable functions that use application as a derivative concept, for example verbs, quantifiers and properties.

Connectedness, we shall see, is the glue to making all linguistic abstractions real to the extent that the work they do becomes empirically assessable. That is because **function application** can participate in **diagrams of functions** too such as (31), not only at the end point of a **diagram path** but anywhere, making itself available to **model selection** (32).

1.5 Recursion: named and nameless

One of the appeals of **recursion** in linguistics is the observation that it is natural for many languages²⁵ to have syntactic clauses that contain syntactic clauses of the same kind almost endlessly:

- (44) I think you think she thinks I like candies. English

²⁵But not all, for example Pirahã, if Everett (2005) is right, which is still in contention.

To discuss the issue to understand whether we are looking at an invariant or a variant, we may start with the following two recursive definitions in mathematical abstraction:

(45) For $n \in \mathbb{N}$

a. $n! = \text{if } n \text{ eq } 0 \text{ then } 1 \text{ else } n \times (n - 1)!$ Factorial

b. $\text{fib}(n) = \text{if } n < 2 \text{ then } 1 \text{ else } \text{fib}(n - 1) + \text{fib}(n - 2)$ Fibonacci number

For example, $3! = 3 \times 2! = 3 \times 2 \times 1! = 3 \times 2 \times 1 \times 0! = 3 \times 2 \times 1 \times 1 = 6$. For the Fibonacci number, $\text{fib}(2) = \text{fib}(1) + \text{fib}(0) = 1 + 1 = 2$. They are **recursive descriptions** because the lefthand side of the definition in each case refers to itself on the righthand side, once for the factorial, twice for the Fibonacci. This is sometimes given a special name, for example **named recursion**, **recursion by name**, or **re-entrancy**. The critical thing for us is that they are descriptions, the telltale sign of a variant, not the values themselves that we are studying for admissibility. These are the **recursive values**, that is, values which are *more instances* of the *same kind*.

The descriptive compression of recursion can be quite seductive for claims of irreplaceability; for example, we may fail to notice nonrecursive formulations of the same functions:²⁶

(46) For $n \in \mathbb{N}$

a. $n! = 1 \times 2 \times \dots \times n$ Factorial

b. $\text{fib}(n) = \frac{\varphi^n - \psi^n}{\sqrt{5}}$ where $\varphi = \frac{1+\sqrt{5}}{2}$, $\psi = \frac{1-\sqrt{5}}{2}$ Fibonacci number

Everett's arguments appears to amount mathematically to the suggestion that although English is like (45a) in its recursion, Pirahã realizes it in the pattern of (46a), turning self-embedding clauses to nonembedding iterations, therefore not realizing recursion at all.

The truth of the matter is that none of this is constitutive; they are merely analogical arguments, for and against, or derivative concepts at best. The recursion-is-necessary argument is based on a discursive interpretation of indefinitely embedding structured stringsets. Ways of describing objects like them should not be confused with the object itself. The object itself, for example (44), is not structured something like (47a) but something like (47b).

²⁶There is a mathematical result going back more than a century, to Schönfinkel (1920/1924), that recursion is eliminable from any formal description as a by-product of solving a more fundamental problem about values, their equivalence. It is the paradoxical combinator Y which we shall study briefly.

- a. Name-recursive factorial
 $fact(n) = \text{if } n \text{ eq } 0 \text{ then } 1 \text{ else } n \times fact(n - 1)$
- b. Value-recursive factorial
 Let $h = \lambda f. \lambda n. \text{if } n \text{ eq } 0 \text{ then } 1 \text{ else } n \times f(n - 1)$
 Then $h(fact) = (\lambda f. \lambda n. \text{if } n \text{ eq } 0 \text{ then } 1 \text{ else } n \times f(n - 1))(fact) =_{\beta}$
 $\lambda n. \text{if } n \text{ eq } 0 \text{ then } 1 \text{ else } n \times fact(n - 1).$
 Then $h(fact) = fact$ because $h(fact(n)) = fact(n)$.
- c. Therefore h is the higher-order function (witness the λf abstraction), which fixes the factorial function, without names. Notice that there are no names in h , just placeholders.

One concern here may be that h is the fixpoint finder of only the factorial function. Notice that even as such it eliminates named recursion while maintaining recursive dependencies. The marvel of the fixpoint combinator Y is that it is the fixpoint finder of any **functional**, that is, it does to any $\mathcal{H}$ what the h above does to $fact$ (i.e. $Y\mathcal{H} = \mathcal{H}(Y\mathcal{H})$), and in doing so it eliminates all named recursion, not for the purpose of elimination but for the purpose of transparently finding its fixpoint, that is, equivalence of values whether they are recursive or not, without names.²⁸

Named recursion, therefore, is not an invariant; it is a very creative way to compress descriptions, that is, to form a variant. It is not necessary for capturing arbitrarily large, complex and recursive dependencies. And, given the mathematical understanding from combinator Y , there is only one kind of recursion, the one eliminated by the combinator (and similar combinators) in the process of finding fixpoints of functions. In other words, recursion is a semantic problem about equivalence of values (recursive values and other kinds of values), not descriptions of values, which is solved in the process of finding fixpoints of functions. It is not a syntactic problem. We skip these advanced topics.

1.6 Probability

This section is about some subtleties of the concepts of probability, about notation and terminology that can be easily misunderstood. Notation- and method-

²⁸In other words, the Y combinator is not a recursion remover. It is a fixpoint finder which eliminates recursion as a by-product because it does nameless computation of fixpoints. The idea of fixpoint is semantic in origin; it is name-free and logic-free; it is about equality of values. See [Lobina \(2014\)](#) for confusion caused by recursion in linguistics, and [Bozşahin \(2026\)](#) for the mathematical demonstration of the results just mentioned.

heavy first exposure to probability may twist the reception and appreciation of the true value of probabilistic methods in linguistics.

Classical probability is based on the idea that events may be truly independent. One can only wish that we could get away with it by calling it a mathematical idealization, but unfortunately we cannot, at least not in linguistics. First let us see what we get by that assumption.

Probability is concerned with predicting observable outcomes. The outcome is assumed to be nondeterministic, therefore we try to estimate or guesstimate it. Experimenting and probabilistic inference on a sample space are how it is commonly done. These activities must be clearly specified before probabilistic methods are sought.

We have **process of observation (experiment)**, **outcomes** and **sample space** in probability. These concepts can best be described by an example. Take for example coins. The coin flip can be the process of observation, the experiment; outcome would be for example coming head in a coin flip. Sample space is the set {head, tail}; let us assume $\{h, t\}$ from now on for brevity. For the sake of introduction let us assume a countable and identifiable sample space.

Even with this simple arrangement we can conceive arbitrarily large sample spaces (i.e. sets), for example the sample space of observing a tail for the first time is

$$(49) \quad T = \{t, ht, hht, hhht, hhhht \dots\}, \text{ where } h: \text{heads}, t: \text{tails}.$$

Let us assume that for example hth would not be in the sample space, in one common interpretation of ‘first time’. Sample space is therefore an abstraction, not something we take in as “raw sample.”

An **event** in probability theory is a subset of the sample space, sort of like a collective name for it. The process of thinking probabilistically about a problem starts with identifying a subset to be a problem to be studied. For example, consider studying winning a lottery if coin flips turn tail in at most four trials. This is an event in probability theory. It is a finite subset of T just mentioned.

It is when we are interested in more than one event that we need some assumptions about their independence to put probability theory to work on complex problems. Consider again T to be sample space mentioned with two events about it, say A and B , A being winning the lottery mentioned in four trials and B being winning the lottery in even number of trials, for example $\{ht, hhht, hhhhht, \dots\}$. Depending on the nature of the problem, the probabilistic interpretation of the expression $A \cup B$ is all elements of T that are in event A or event B ; likewise we can think of $A \cap B$ and $A \subset B$ kind of problems.

With these assumptions, probability theory assumes that the probability of any event is nonnegative, the sample space covers the whole problem, and its events are independent:

- (50) a. $P(A)$, the probability of event A in T , is $P(A) \geq 0$.
 b. $P(T) = 1$.
 c. If we have $A_i, i = 1, \dots, n$ for some $n \in \mathbb{N}$, then
 $P(A_1 \cup A_2 \cup \dots \cup A_n) = P(A_1) + P(A_2) + \dots + P(A_n)$. That is, A_i are mutually exclusive.

In the **frequentist** interpretation of probabilities, if there are N equally likely outcomes and n of these are what we call event A , then $P(A) = \frac{n}{N}$. In **Bayesian** interpretation of probability, it is a degree of belief in hypotheses about A rather than events of A , which are updated as more events are observed. That is the connection to experiment.

If events overlap, probabilities would have to take that into account. For example, if $A \cap B \neq \emptyset$, then $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

These are the rules of probability. From here we reach **event distributions** with independence assumptions similar to independence of events. Three in particular, **joint**, **marginal** and **conditional** distributions, mean different things in studying linguistic structures.

Let us take two events A and B . A joint distribution over A and B assumes that we can range **random variables** over the events A and B to see how they distribute over a **shared outcome space**:

- (51) a. Assume that the random variable X ranges over A and Y over B .
 b. A joint distribution of A and B , denoted either $P(A \cap B)$ or $P(A, B)$, is where X is some value and Y is some value, denoted commonly $P(A, B) = P(X = x, Y = y)$.
 c. $P(X = x, Y = y)$ is the **joint probability** of X and Y .
 d. A **marginal** distribution of X given $P(X, Y)$ is $P(X) = \sum_Y P(X, Y)$.
 Likewise we can define marginal distribution of Y , given $P(X, Y)$: $P(Y) = \sum_X P(X, Y)$. Notice the presupposition of having a joint distribution.

What could be these joint probabilities and distributions in linguistics? The keyword here is **joint**, which means having a good understanding of what two events share *in every observation* (notice the small x and y in (51b)), and we want to know how they distribute over the events themselves.

Having a joint distribution is equivalent to having a theory about their shared behavior. For example, take syntax and semantics of expressions. If we have their

joint distribution, say A is syntax and B is semantics, and we know $P(A, B)$, then we have the knowledge of how syntax and semantics connected in every observation, and we would want to know how they distribute. For example, if we are interested in clausal semantics, then we can **marginalize** over clausal syntax, that is, fix the A event to a subclass called clauses, and look at the distribution, which we can do by say $P(B_{\text{clause}}) = \sum_{A \in \text{clause}} P(A, B)$. That would be of course a dream come true for a linguist. In this endeavour it is important to read between the lines in the notation, because $P(B_{\text{clause}})$ out of context may indicate an isolated distribution but here it is marginal, meaning we already know there is a shared outcome space.

In many cases what we get at best is a **conditional distribution**. For the running example in particular, a conditional would be something like, *given* the clauses, how can we estimate their semantics? That is, we would not really know how they share an outcome space but try to guess it by looking at conditionals. That too may be confusing because the notation in the literature is sometimes similar to marginalization, using for example the joint distribution notation when there isn't any.

- (52) a. Let A and B be two events over a sample space.
 b. The conditional probability of B given A , denoted $P(B | A)$, is defined as $P(B | A) = \frac{P(A \cap B)}{P(A)}$, $P(A) > 0$.

If there is no shared outcome space, that is, if $P(A \cap B) = P(A) \times P(B)$, then $P(B | A) = P(B)$. In other words, A would have no influence on B . The difference between a conditional probability and marginal probability is the difference between not having and having a joint distribution, that is, having a knowledge of the shared processes but not so much about their distribution.

In today's numbers-are-everything heydays, we must clarify the ideas behind the notation so that we know what we are looking at. Joint probabilities over for example syntax and semantics assume a space of admissible syntax-semantics correspondences. Conditional probabilities do not. They would 'attach' a semantics to an expression from conditional distributions, and that is what most statistical approaches to language do.

1.7 Logic

Perhaps the best gateway to a massive topic like logic is remembering how Frege got into it from mathematics and geometry. He was interested in study of thought,

back then considered to be in the domain of **psychology**. He was not only interested in thought but correct thinking. Realizing that that would be too *normative* for psychology, normative not being a pejorative word for him, he turned to logic. Logic for him, and to many others, is about making correct inferences, not just inferences. Although every science seeks truth, logic's relationship with truth seems to be what drew Frege into it.

The following is the mapping from truth and falsity to models (i.e. values) of them:

- (53) a. True: 1
- b. False: 0

One logical operator that newcomers find perplexing is **implication**. It is defined as:

- (54) $A \rightarrow B$ is false when A is true and B is false, true otherwise.

The 'otherwise' is probably the source of confusion. How can false $\rightarrow$ true be true, even false $\rightarrow$ false? It is not a rule of inference; it is the kind of operator that supports correct inference, for example **modus ponens** (55a), and its logically equivalent reading in reverse, **modus tollens** (55b).

- (55) a. From A and $A \rightarrow B$, conclude B . In standard notation $A, A \rightarrow B \vdash B$
- b. $\neg B, \neg B \rightarrow \neg A \vdash \neg A$

They are logically equivalent, but people seem to have problems with modus tollens unless more content is given (Johnson-Laird 1983).

The inference notation was invented by Frege (1879). His **sense** must be structured *so that* **reference** can be evaluated. His $\vdash$ is a composite, '–' representing judgeable content (his 'thought'), and '|' judgment over it. '⊢' is then structured sense, so that we can see the difference between the following two expressions regarding content, reference and judgment in them:

- (56) a. Morning star is evening star.
- b. Morning star is morning star.

Although in principle all logical operators supporting correct inference can be reduced to one, called the Sheffer stroke, it is much more intuitive to use other standard logical operators as well as implication:

- (57) a. And: $A \wedge B$ is true of both A and B are true; false otherwise.
- b. Or: $A \vee B$ is true if at least one of them is true, false otherwise.

- c. Negation: $\neg A$ is true if A is false, false otherwise.

We can combine them in many ways to reach what is called **well-formed formulas** of **propositional logic**, so called because it can only entertain truth values of propositions. We leave that to logic books.

Regarding language, what if we need to talk about truth values of predicates? For example the following example is not expressible with propositions only:

- (58) Every child slept.

We need predicates because of properties such as being a child, and quantification because of things like *every*. **First-order logic** is an extended notation to do that; it is first-order because quantification is over values not predicates:

- (59) a. If p is a proposition and x is a placeholder for values, then $p(x)$ is a predicate, for example $child(x)$. There are unary, binary, ternary etc., predicates. For example, we have the set of unary predicates $\{p(x) \mid x \in U\}$, where U may be an arbitrarily large set.
- b. Universal quantification: $\forall x p(x)$ means for all values in U , $p(x)$ holds, that is, it is true.
- c. Existential quantification: $\exists x p(x)$ means $p(x)$ holds for at least one value x in U .
- d. $p(x)$ can be any well-formed formula of first-order logic. We leave their complete description to logic books.

Now (58) can be expressed in First-order logic: $\forall x child(x) \rightarrow slept(x)$. Notice that if we were to express it in propositional logic, we would need as many formula as the size of the set U in (59), which can be arbitrarily large. This is the benefit of first-order logic.

What if we want to go further in expressivity? What propositional logic faces as a representation problem for (58) is first-order logic's problem for the following expression, as can be seen from its quantification over predicates in (60b), which is not first-order.

- (60) a. I do what you do.
- b. $\forall P P(you) \rightarrow P(i)$

The problem, as it turns out, is that although we can characterize admissibility (invariants) of any level of logic, computing their variants over and above first-

order has been problematic.²⁹ We leave such advanced topics to logic textbooks, for example [L.T.F. Gamut \(1990, 1991\)](#).

There is however more linguistic points to ponder concerning these results. It will be noticed that quantification only affects the number of readings, not grammaticality:³⁰

- (61) a. Every professor read a book.
 $\forall x \exists y \text{prof}(x) \rightarrow \text{book}(y) \wedge \text{read}(x, y)$, and
 $\exists y \forall x \text{book}(y) \wedge (\text{prof}(x) \rightarrow \text{read}(x, y))$
 b. Every professor read Ulysses.
 $\forall x \text{prof}(x) \rightarrow \text{book}(\text{Ulysses}) \wedge \text{read}(x, \text{Ulysses})$
 (If there are no professors, we would expect the statement to be true whether Ulysses exists or not as a book.)

We must therefore be clear about what it is for which we seek invariants so that we can assess the methods for variants.

1.8 Mathematics and computation

It is often said that mathematics and computation must be one and the same thing. Notwithstanding the tug of war in which mathematicians are unable to convince computer scientists that they are mathematicians with gadgets, and computer scientists counterarguing equally unsuccessfully that mathematicians are computer scientists with pencils, there is a real distinction that is very relevant to linguistics: Mathematics does not require an **encoding-decoding relation** in its abstractions, whereas computer science does in all its abstractions.

So what exactly is **computer science**? One of its founders, Donald Knuth (1996), defined it as the study of whatever ideas we can put in something we can call a computer as an abstraction to the extent that it would require no external interpreter to try them out. **No homunculi** is the computer science code of conduct

²⁹Recall the caveat in the very beginning of this chapter. First-order logic can be made computable algorithmically under certain restrictions, which are embodied in what is called Monadic first-order logic, also called Monadic predicate calculus. We need more severe restrictions to make second-order logic computable algorithmically.

³⁰Although [Barker \(2022\)](#) is advanced reading, the examples in it are mind openers for everyone. I can only vouch for my own native language, Turkish. Native speakers of Turkish never fail to amaze me on coming up with more or less readings than linguists or logicians can possibly come up with. Less readings would be the least of problems for the reifying theorist.

for the human practice with the computer.³¹ This is one of my favorite brief and informally specified invariants. Its subtle smuggling of admissibility is art work.

So why must we have an encoding-decoding relation in computer science when we do not need it in mathematics? It is because in order to have the computer do the work for *us*, about a human idea, we have to encode that idea to be run by the computer since we are not doing the execution ourselves. As mathematicians we do not need that. In order for the computer to make its output recognizable by us, we need the decoding relation. (And yes, most of us have to decode the mathematician to do that among humans.) Let us not forget for one moment that both encoding and decoding are human ideas passed on to a computer. The computer itself is a human idea. It does not grow on trees, spring out of the seas or pop up in the sky.

More recent work, for example [Horsman et al. \(2013\)](#), defines **computer science** as studying abstract evolution of a physical system. Evolution of the system is abstract because a computer is assumed to be programmable, interpreting an encoded abstraction. The system is physical because of the necessity of encoding and decoding. This takes us back to the fundamental distinction between mathematics and computer science.

Encoding means mapping an abstract property, say the letter A, to a physical medium, first say to bunch of bits then to voltages for an electronic digital computer. **Decoding** means mapping a physical property, for example voltages turned to bits, to the programmer's world, that is, to abstraction, say to letter A. It is evolution because, via encoding-decoding, a program is run in a physical substrate, and the physical states are changed *because* of the program, not randomly.³² That is what computation does. Then, following Knuth kind of thinking, we assess whether it does what we intended. In short, computer science is human practice of ideas that can be made explicit enough so that we can try them out without us doing the execution.

These are important distinctions to keep in mind when we talk about the role of computation in study of linguistic structures later in the chapters. We need no more than the distinction made here to have a renewed look at for example automata and formal languages to understand the variants, two well-known fields of computer science.

³¹[Weizenbaum \(1976\)](#) was an early wake up call that computer science is indeed human practice, not practice of the extended human.

³²Sometimes it is suggested that analog computing does not require the encoding-decoding cycle. That is not correct. Analog computers are programmable computers too. They are designed machines, not randomly chosen conducting material. Their difference is that they accumulate error at every step rather than correct it at every step by digitization, called error correction in the digital world. [Mills \(2008\)](#) explains their encoding-decoding mechanism.

If at this point you are getting the impression that mathematics addresses the invariant and computation addresses the variant for pretty much the same problem, **connectedness**, manifested in mathematical **category theory** as conceptual counterpart of **function composition** (Simmons 2011), then you would be moving in the right direction. It is not a simple schism of ideas for the invariant and methods for the variant; it is more about having or not having to worry about the encoding-decoding relation in the method for the invariant and method for the variant.

1.9 Linguistic structures

Nobel laureate Herbert Simon once said that we study nature at levels because it comes to us at levels. Linguists can relate to that, themselves having difficulty explaining to others why they have phonology, morphology, syntax, semantics and what not for something that seems to be patently obviously one thing: a string of words or signs in an expression.³³

We have reasons to believe that linguistic structures are not reducible to one another. This is important because it tells us what kind **admissibility** conditions we must consider to constitute a linguistic theory. Consequent differences in **variants** are made more meaningful that way, even when we choose to use the same method for different levels. Pairwise comparison of the well-known linguistic structures reveals the need.

Consider the pair of **phonology** and **morphology**. The following example shows the morphemes (62a) and syllables of the expression (62b). Morphemes are separated by ‘-’, syllables by ‘.’.

- (62) a. ev-ler-e
house-PLU-DAT
‘to the houses’ Turkish
b. ev.le.re

It is clear that morphemes and syllables do not have to coincide; for example *re* of (62b) is not a morpheme in Turkish. The distinction will be important when

³³There is an argument in sign linguistics that sign languages realize a parallel sign space. However, signs have to be realized in time and space, and there are theories of variants in their computation, for example Bird & Ellison (1994), that they can be given string semantics in the invariant, from structured order. Once again this is a distinction between invariants and variants. The invariant does not seem to need separate admissibility for spoken and signed languages.

we come to mathematical method selection for the variants. For example, phonological automata and morphological automata would be different species. And if we want to connect morphology and phonology with automata it would have to be a relational automata because it must now be clear that there is no functional dependence from morphology to phonology (e.g. one morpheme to many phonemes), or from phonology to morphology (e.g. same phoneme in many morphemes).³⁴

Now consider the pair of **morphology** and the **lexicon**:

- (63) a. ev-ler
house-PLU
'houses' Turkish
- b. kiler
cellar
'cellar'

Apparently every Turkish child learns through time that not every familiar fragment or ending of a word is a morpheme; for example, *ler* in (63b) is not whereas *ler* in (63a) is. We can understand that they know the difference when they pluralize (63b) as *kiler-ler* 'cellars'. We then have to worry about on what basis children can do this. Proposing learnably distinct structures for morphology and the lexicon (perhaps, vocabulary) is the consensus.

Consider the pair of **morphology** and **syntax**. There is a well-known problem in morphology, called **portmanteau**. It is the phenomenon where one form serves two distinct semantic functions but it is impossible to tell which part is doing what, for example tense and person:

- (64) I/She/*They was working. English
- TENSE
- PERSON

These meanings are not separately identifiable. We do not see that in syntax; for example the following contractions are closest to multiple function but they have separately identifiable parts:

- (65) a. I wouldn't try. English
- b. I wanna try.

³⁴ Mechanisms such as automata and formal languages are nice, probably not by themselves. We will see that once we let meaning chose mechanism, all these dull but venerable mechanical methods come to life in a different way.

There are many theories and claims in linguistics that morphology and syntax compete as formal devices of expressing meaning (for bracketing, scope, specificity etc.), therefore it is curious why they do not compete here. It appears that portmanteau is an exclusively morphological phenomenon on forms.

Consider the pair of **syntax** and **semantics**. The following expressions (66a) have more or less the same syntax irrespective of the verb; for example they passivize, coordinate and relativize the same way (66b).³⁵

- (66) a. I persuaded/promised/expected/wanted her to study. English
b. The student that I persuaded/promised/expected/wanted to study

Depending on the first two verbs, the expression means, respectively, ‘she studies’, ‘I study’. For the third and the fourth verb, ‘she’ is not even an argument of the matrix verb. Therefore all four meanings are different although they have more or less the same syntax. It seems then that syntax and semantics are not reducible to one another.

Consider the pair of **semantics** and **information structure**. For the purpose of introductory remarks let us focus on clausal information structure, for example focus, topic and background. The following examples have the same truth conditions and same referents; yet they have different information packaging:

- (67) a. I like this book. English
b. This book I like.

Finally, consider a three-way comparison of **phonology**, **syntax** and **morphology**, that is, all the dimensions of form that linguists insist on treating distinctly. We can see the same phonological shape, /z/, underlined in the following examples, with different syntax (noun phrase vs. verb phrase) and different morphology (nominal vs. verbal inflection):

- (68) a. The boys like the video. English
b. She toys with the idea.

In summary, the linguistic level distinctions are not arbitrary. Maybe the names are not up to everyone’s expectation, but the levels appear to be distinct enough to seek their own admissibility conditions. The choice of the mathematical method for studying their variants is bound to be affected by the distinctions.

³⁵ *Promise* seems to be a bit of an outlier: ?*John was promised to study*. Promise-class is notoriously difficult for the child cross-linguistically. However, they are acquired early (Sherman & Lust 1993), but later than other verbs, around the age of four to five years and a half (Hyams & Orfitelli 2018).

1.10 Linguistics and philosophy

Linguistics asks **fundamental questions** that are not reducible to questions of general science. A nondiscursive language of mathematics may help us reveal some properties so that the differences of linguistics from other sciences begin to make sense to the parties involved, rather than sound like hardline position statements. In doing so some forgotten (or neglected) links to venerable ideas in philosophy of science are worth remembering from the beginning.

One such concept is **cause**. We have known since Hume that causation may be out there and in the mind, conceiving causal links when there aren't any, because the mind "is set up that way." **Counterfactual reasoning**, considering what the states of affairs could have been had certain events not occurred, is very much part of causal thinking, and some of us, for example [Pearl & Mackenzie \(2018\)](#), are optimistic that one day computers can do that too. For the time being, we are concerned with how people attribute causal relations to the extent of persuading others that they may be on to something toward explanation. Involving others in the knowledge construction campaign (not self-persuasion campaign) is quite a Platonic dialogical argument, and it is time we remember one such crucial contribution to causal attribution from philosophy.

[Mackie \(1965\)](#) suggested that **causal knowledge** is the process of convincing the parties involved in the explanation of why things occur where they have vested interests to think differently from each other.³⁶ He showed that something more complex than necessity and sufficiency is involved in such processes of explanation. Identifying **cause** according to Mackie is the process of looking for conditions that are unnecessary but sufficient for something to arise which is necessary but not sufficient for the process of causal attribution (INUS). His everyday example is as follows:

- (69) Suppose that a fire has broken out in a certain house, but has been extinguished before the house has been completely destroyed. Experts investigate the cause of the fire, and they conclude that it was caused by an electrical short-circuit at a certain place. What is the exact force of their statement that this short-circuit caused this fire? Clearly the experts are not saying that the short-circuit was a necessary condition for this house's catching fire at this time; they know perfectly well that a short circuit somewhere else, or the overturning of a lighted oil stove, or any one of a number of other things might, if

³⁶ Compare this for the time being with ideas about science consisting mainly of causal knowledge in the physical sense, as John Stuart Mill had proposed.

it had occurred, have set the house on fire. Equally, they are not saying that the short-circuit was a sufficient condition for this house's catching fire; for if the short-circuit had occurred, but there had been no inflammable material nearby, the fire would not have broken out, and even given both the short-circuit and the inflammable material, the fire would not have occurred if, say, there had been an efficient automatic sprinkler at just the right spot. Far from being a condition both necessary and sufficient for the fire, the short-circuit was, and is known to the experts to have been, neither necessary nor sufficient for it. In what sense, then, is it said to have caused the fire? Mackie 1965:245–6

He suggests that experts are aware of INUS conditions whether it is scientific knowledge or everyday knowledge—recall Ryle, from (3) of Chapter 2, that we are the experts when it comes to everyday knowledge. They are saying that no single factor is enough; but, we remove one factor and the outcome collapses. We all feel that, in scientific practice and daily practice. We the native speakers and signers are the experts when it comes to language. Yet there are other ways the outcome could have happened. Therefore multiple conditions must align, none alone determining the result. Removing one part breaks the agreement of terms of discussion, yet there can be different ways to reach similar agreements.³⁷ In short:

- (70) A cause is an **insufficient (I) but necessary (N) part of an unnecessary (U) but sufficient (S) condition** for something to happen; to explain the process to the parties involved.

In the book we shall see that constraints on grammaticality are much like INUS conditions for it. We can in fact postulate the following **realist position** for grammar:

- (71) The **grammar** as the locus of grammaticality conditions is the **INUS cause** of **meaningfulness**.

The notion of grammar is necessary to understand grammaticality, but not sufficient to understand meaningfulness which is a consequence of grammaticality. Meaningfulness itself is unnecessary because it is a consequence, but it is sufficient.

³⁷I am sure the reader has noticed that this way of thinking does not necessarily coincide with so-called pluralism.

This is a realist position which can be considered antinominalist because objects of grammar and the grammar itself are not just names but *conditions*; they are not arbitrary or verbal. If we think of the **nominalist position** in linguistics such as Bloomfield's as a revolt against **mental realism** of the **conceptualist position**, positing that all linguistic signs spoken or signed are just acoustics or imagery, then one realist position is promoting **mathematical reality** in addition to physical and mental reality, that **admissibility of grammars** belong to mathematical reality as study of **invariants**, with natural consequences for mental and physical reality in the **variants**.³⁸

As such **grammar** is an **abstract object** in the Platonic sense, abstract because it resists reification to reveal its constitutive role, object because it is indeed a cause. We do not need mentalistic, nominal or materialistic vocabulary to make these connections. The running theme in this book is that such reified vocabulary might have gotten in the way of understanding the connection of linguistic ideas to linguistic methods.

In closing this section, with some risk of regimentation and oversimplification for the sake of introduction, we can put INUS in a historical and philosophical context concerning the relation between **dialogue** and **knowledge**:

- (72) a. Socrates examines and convinces,
 b. Aristotle organizes, informs, and asserts,
 c. Plato invites and builds,
 d. Bharṭṛari cooperates and shares.

Rather than merely thinking of INUS as an ontology of causal explanation, we can think of it as an **epistemology** of causal persuasion concerning (72). It identifies factors that are *worth mentioning* in the dialogue of knowledge construction and interpretation. As a complex amalgam of ideas and factors listed above, INUS is more than a method; it is an idea to jointly consider them in any kind of explanation for phenomena that are complex enough.

³⁸In fairness we must not of course pigeonhole everyone who is sympathetic to a position. William of Ockham as a nominalist, for example, is not a nominalist who is a staunch antirealist. He advised us to avoid excessive ontologies including those in the so-called universals and theories, the latter of which has come to be known as **Occam's razor**. Similarly, Poincaré is considered a **conventionalist** but that is hardly a position against realism. It would be sufficient for **conventionalism** not to be totally dependent on nature yet not fully independent of it either. In brief, it says that scientific knowledge does not totally depend on external results and experiments. The idea of a **model** as a forum for public discussion of explanations goes back to this perspective. Old school or hardline **positivists** would be at a loss about this prospect.

1.11 Typology and mathematics

An introductory book with a mathematical focus on general linguistics is not the best place to instil subtle concepts of **linguistic diversity** and **typological variation**. What I will do instead is harness the basic mathematical thinking in this chapter to see if it can bring another angle to typological abstraction.

I will concentrate on the relative clause, more specifically, on the restrictive relative clause. Its definition is elusive, and its identity in the typological space of language variation has been a major topic in the classics of the field, for example [Mallinson & Blake \(1981\)](#), [Comrie \(1989\)](#). Here is an informal description:

- (73) A restrictive relative clause relates two clauses by sharing an element—usually a common noun or nounphrase—in one clause to restrict or further qualify an element in another clause somehow.

Although there is cross-linguistic variation on “somehow,” there seems to be no exception to the generalization that only one element of a relativized clause is shared with one restricted element of another clause. Multiple extraction and multiple sharing have not been attested. It is a restriction which does not seem to be logically necessary, and no effort to treat it as procedural efficiency pressure for example by [Hawkins \(2003\)](#)—or some kind of functionalist constraint on the variant—is without problems. We therefore look at it in its invariance by noting some properties of the “somehow” part first.

In languages of the world we observe the extracting variety (e.g. English and Turkish); nonextracting and indexing variety is where arguments in question are verb-indexed in both clauses rather than extracted (e.g. Navajo—[Willie 1989](#)); in some languages resumption of the shared argument is preferred rather than extraction (e.g. Arabic, Irish); and, both extracting and resumptive strategies are possible (e.g. Mandarin Chinese—[Li & Thompson 1981](#)).

Examples of the extracting variety are common in the linguistic literature, for example the following where relative clause boundaries are marked for convenience:

- | | | | |
|------|----|---|---------|
| (74) | a. | the book [that I read yesterday] | English |
| | b. | [ben-im dün oku-duğ-um] kitap | |
| | | I-1s yesterday read-REL-1s book | |
| | | ‘the book that I read/was reading yesterday.’ | Turkish |

Others need exposure for an appreciation of what an invariant would be expected to admit. The following are examples of Navajo relative clauses:

- (75) a. [ʼatʼééd danahachaʼéɛ] yaltíʼ Navajo
girl 3ss:was jumping:REL 3ss:speak
‘The girl who was jumping around is speaking.’ Willie 1989:30
(the girl—the one who was jumping around—she is speaking.)
- b. [Ashkii hastiin yiztał-éɛ] atʼééd yiztsʼqs
boy man 3:perf:3:kick-REL girl 3:perf:3:kiss
‘The boy who kicked the man kissed the girl.’ Platero 1974:136
‘The man whom the boy kicked kissed the girl.’
* for ‘the boy kissed the girl who kicked the man’

Notice the verbal indexing of all arguments for the relative clause and the matrix clause. Willie (1989) explains that a postposed noun phrase out of the embedded clause is not a relative construction, if grammatical at all. Therefore Navajo harnesses verbal indexing for relativization of the elements in question, which is itself independent of the relative construction because all finite verbs are indexed on all arguments in Navajo, hence it is the head of the relative clause (glossed as REL) that does index sharing.

Irish achieves the same effect of index sharing by resumptive pronouns under the control of the relative marker:

- (76) an fear [a labhair mé leis]
DEF man REL spoke I with-him
‘the man that I spoke with’ Irish

So what do all of these strategies have in common—after all we can informally describe what a relative clause does in one sentence as we did in (73), and can we learn something from a common mathematical abstraction rather than being content with making a posthoc generalization no matter how accurate?

Semantics of the restrictive relative clause is the following function:

- (77) $rel(P, Q) = \lambda x. P(x) \wedge Q(x)$, where
 P is the restricting clause,
 Q is the modified (restricted) head, and
conjunction and x -sharing represent the restriction.

In other words, P and Q concern the elements of the relative clause in the expression, syntactically and morphologically, and x and conjunction arise from the meaning of the head of the construction, not from P or Q . The relative marker can be omitted or it can be overt, for example (a) and (b):

- (78) a. the man I know English

- b. the man that/who/whom I know

(77) assumes that Q can be predicated over to make $Q(x)$ meaningful, which is standard assumption for restricting common nouns, for example ‘man’ in the preceding example is a property such as $\lambda y.man'(y)$, so that $Q(x)$ restriction unfolds as $man'(x)$ when x is applied to it. Headless relatives such as the following show that Q ’s reference narrowing (with semantics say *one*’) can be conditioned on the descriptive content provided by P ; notice the consequent shift from *that* to referentially dependent elements such as *what* for that (also *who*, *which*):

- (79) What I know can change the outcome.

We are now in a position to determine how (77) is realized morphosyntactically, that is, determine how its construction and transmission of configurational and referential properties of the relative clause is realized in an expression. If the relative marker is overt in determining the meaning of the relative clause, then (77) is not explicit enough as a function because it is not clear in what order the relative marker takes its arguments P and Q , and what their referential properties are. If relativization is headless as in the preceding example, then (77) is a function of a referentially distinct element (*what*, *which*, *who*), and Q is a referentially dependent function on such elements. If (77) is realized without any relative marker at all, as in (78a), then (77) must be a binary relation between two functions rather than a function because it must relate one category, most likely the predicative nominal ‘man’ in the example, which is nominal, to another category, the relative clause, which is clausal. All of this sounds as complicated as leaving it to the variant to be done with it.

To arrive at a linguistic characterization of the relative clause covering its syntax, morphology and semantics, we can start by shifting referentially-free syntactic categories such as NP to referentially-dependent functions; for example, NP^{NP} is a function from NPs to NPs in the function notation of (23a). NP is the syntactic function of Q in (77). P is a clause, with one referent singled out for sharing, which we can represent as the syntactic function S^{NP} . Altogether we can write these as one-to-one correspondences (denoted by ‘:’) which make them linguistic functions by making (77) more specific linguistically:

- (80) $rel(S^{NP} : P, NP : Q) = NP^{NP} : \lambda x.P(x) \wedge Q(x)$

rel is realized in English as a relative marker configurationally requiring its first argument to the right and its second argument to the left as local sisters parented by *rel*, where S^{NP} is extracting, that is, “missing” that NP in S . In Irish and Navajo,

this function selects an index for NP from the full clause S . In Turkish rel is morphologically realized as a suffix configurationally requiring the inverse surface order compared to English although parenthood and sisterhood are the same:

- (81) ben-im konuş-tuğ-um adam
 I-1s speak-REL-1s man
 ‘the man that I spoke/am speaking/speak/ with’ Turkish

If rel itself is not phonologically realized as in (78a), then (80) is a relation relating a function such as $\lambda y.man'(y)$ for Q to the function $\lambda z.know'(i, z)$ for P , same as the result of (80), which is a *function*, of x and conjunction. If rel is referentially dependent as in (79), rather than share two independent properties, then $NP : Q$ is such a function, say $\lambda y.one'y$, to co-refer with $S^{NP} : P$ via x in the result of (80). The syntactic result of (80), NP^{NP} , is always referentially dependent, arising from argument sharing.

What can these mathematical abstractions tell us beyond rigor and empirical clean up? Realized with or without a relative marker, with or without a head to be restricted, such clauses must be single-argument sharing because it is the only way to realize (77) as a linguistic function such as (80) as befits a **headed construction** which completely determines its syntax, morphology and semantics whether the head is overt or not. We would not need ontological commitments, processing efficiency and other mechanical machinery, which are problematic to begin with cross-linguistically.

A headed construction without the head functionally *determining* what it does syntactically, morphologically and semantically would be an unfathomable construction, and we have yet to find one in any language no matter how complex, simple or understudied its syntax and morphology are.

This is a concern for admissibility conditions for heads of any construction, that is, it concerns the invariant regarding possible correspondences for constructions, which is common to all languages. It is not about the variant conditions of their mechanism or computation. Once we understand the invariant in its abstraction, in its mathematics, the study of consequent variants crosslinguistically would be a very rewarding typology indeed.

1.12 Theories and models: a brief foray into modeling

One common conception of **scientific model** is that it is simplified representation of reality. This description is quite loaded with philosophical assumptions. For example, if one believes that everything is made up, then there wouldn't be

a reality to represent, directly or indirectly. **Nominalists** probably come close to that. So-called Platonic idealism, whatever that is, does not. For Plato abstract objects and ideas are as real as, in fact more real than, things concrete because, among other things—their independent existence, they allow us to conceive **admissibility** as a precondition for modeling spaces over and above names.

It is good to know that he was not left alone in the beginning of 20th century when science really exploded with discoveries and remembered that dictum. **Heisenberg (1958)** famously argued that science does not study reality directly; it studies what nature does to our equations.³⁹ Therefore somebody has to come up with these equations although the structures that is studied through them may exist independent of our experience of them. Also recall **Poincaré (1905)** from the beginning of the chapter, that science does not study facts but what remains invariant across facts.⁴⁰

The **indirectness** that Heisenberg and Poincaré point out is the stuff of **modeling**, through which ideas make contact with data.

The truth of the matter is that in linguistics, in any science in fact, not every theorist has to be a modeler; but, everyone, theorists and practitioners alike, must insist on **modelability** otherwise all linguistic conjectures would only be evaluable by the believers and practitioners; critical assessment would be unreachable to others. Critical assessment cannot be reduced to verification v. falsification, induction v. deduction, etc.; we must as first attempt disentangle **theories** from **narratives**. Neither is self-virtuous or inherently inferior. Pigeonholing is very common in this debate, for example narratives for social science, theories for others. It is the process of debate that counts. Narratives tend to be coherent,

³⁹In his own words: “[...] we have to remember that what we observe is not nature in itself but nature exposed to our method of questioning.” **Heisenberg 1958:25**.

⁴⁰Altogether it means that science is not the only form of discovery; but, it is the one on which people can be trained to replicate work and criticize and negotiate ideas. What we do in this practice cannot be reduced to falsification vs. verification or deduction vs. induction, as for example **Popper (1945)** did but his contemporary **Neurath (1983)** avoided although he was just as polemical as Popper was. Clarification is a different practice than pigeonholing. A case in point: **Hume (1739)** was first to note that induction presumes that conditions would remain uniform in the future as when induction was made, which he attributed to human nature, which Popper turned to a form of deduction to criticize so-called inductivists for their purported circularity, which was not intended by Hume—i.e. this is a **straw man argument**; see **Burns (2009)** for details of the argument. Straw man arguments are more difficult to dissect in philosophy; it is one more reason for new linguists to read more philosophy and do it critically before importing ideas. If we have to personalize a philosophical discussion, which apparently even the best philosopher sometimes succumbs to, we better try **steel man arguments**, where you face your opponent in his/her strongest point. I apologize for the historical machoism in this terminology.

chronological, motivated and use explanatory vocabulary. Given these they can be very convincing in public and professional debate. Theories add admissibility, constraints, consequences, suggesting **thought experiments** or real experiments to falsify (but see footnote 40) or compare alternatives. Narratives suggest **hypotheses**, theories **constrain** them.⁴¹

This process is worth looking into in some detail. In this section we have a brief close look at stages of **scientific modeling**, which starts with the theory, leads to models, then to model selection, then back to reassessment of theoretical claims. We compare scientific models with **engineering models** and **large language models** (LLMs) in the process.

Figure 1.1 is the somewhat conventionalized setup. We start the modeling process with an explicit theory, or an idea to put to thought experiments. Thought experiments in science are not supertangential ideas where anything goes; the reason why they are conceived as thought experiments is that science and technology may not be yet ready to really try them out or do model selection.

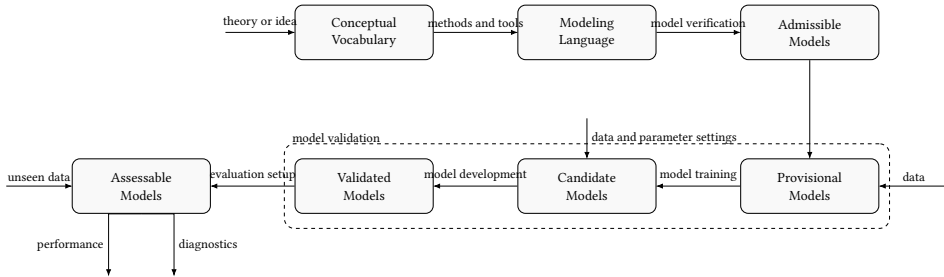


Figure 1.1: Modeling workflow from conceptual vocabulary and admissibility to validation and empirical assessment. The dashed box is in practice a feedback loop from candidates to model selection.

If the theory or idea is sufficiently well-developed to produce a conceptual vocabulary, we can choose or build methods and tools to develop a **modeling language** in which the vocabulary is disambiguated enough to be put to practice.

At this stage modelers, believers, disbelievers and agnostics alike, are at the same level of understanding, or at the very least, at the same footing. They use the modeling language toolbox, accepting the assumptions that led to it, in trying to confirm or refute what they or others have in mind.

Model verification is the process of checking the well-formedness of a model with respect to the theory or idea. In engineering models this may be supplanted

⁴¹This is something for **nominalists** to think about: Admissibility is about conditions, not names. If reification and ontology are temptations to avoid, we can think of them as situated in mathematical reality of abstract objects, but objects nevertheless because they are conditions.

by a standards check and feasibility assessment. In the end what we get is what is admissible by the theory or the idea.

Model validation is the process of checking the theory with respect to the world. If the previous stages mentioned can be likened to intellectual work, this stage which involves countless and excruciating feedback loops is the blue collar work.⁴² There is no white collar work in this field, unless you develop a **large language model (LLM)** for business.

The term ‘language’ in such models puts the terminology of §1.8 in perspective. You may have heard that LLMs have achieved success after a round of encoder-decoder models, encoder-only models, culminating (as of today) with decoder-only models as the most successful architectures.

The term ‘decoder-only’ does not mean that they do not develop an internal representation of the input (called **encoding** in §1.8). There is actually no way a computational system can work without an internal representation that is minimally encoding-decoding. It goes back to Turing’s idea that there is no computation without representation. This is not a mental claim. If there are other kinds of computations out there, they are not the ones we employ here because Turing computability is all we know at the moment. That requires encoding and decoding.

The **contextual representation** of an LLM is based on an input encoding, in their **modeling language** in the terminology of Figure 1.1. If we take the original transformer design of Vaswani et al. (2017) as the point of departure, in the encoder-decoder architecture, or in the terminology of Figure 1.1, in the **admissible models**, there are two “stacks”—builders of provisional models being validated in the terminology of Figure 1.1, one for encoding one for decoding, for building a response and developing it. So the input-output encoding-decoding is built-in, but not the modeling of the developing response.

The critical question for them is not how to predict the next output best but to solve a structural problem, developing what computations are available at layers of the architecture under what attention constraints—relations). Encoder-only systems do that with a bidirectional view of the input for the incremental structural state update,⁴³ interpreting the input encodings both ways, from beginning to end and from the end to beginning, whereas decoder-only models view the input the causal way for the incremental structural state update, that is, from the beginning to end, one-way, as they say in the business, “just like language.” That

⁴² As all analogies are, this is probably not apt in unintended ways but here we are.

⁴³ This **state space** is anticipated and architected by us, the designers. We are not talking about internal states of a conscious being. Note also that us humans do the design in encoding and decoding.

is why it is more appropriate to call systems of continuation probabilities *language* models and others *contextual* models.⁴⁴

This may sound like a performance improvement, and indeed it has made a massive difference in performance, but there is an **admissibility** concern which is also being addressed, although in a very engineering way. The concern is the following: Is **interpretation** of a sequence a problem which has to be solved independently of **generation** of a sequence? Seventy plus years of formal linguistics and fifty plus years of computational linguistics had thought so, solving these problems relatedly but independently, for example by using a grammar in different ways and in different forms and with different sets of information. That is why **parsing/analysis** (read: interpretation) and **generation** (read: production) have been considered subfields in their own right. The problems they solve are obviously different, but does that have to arise from different admissibility conditions?

A decoder-only system asks, looking one way, what admissible continuation exists (the equivalent of generation), and solving the problem requires knowing what is admissible structure with the same assumptions about continuations (the equivalent of parsing/analysis).⁴⁵ In other words, a decoder-only system shows us *through its mathematical architecture* that these two problems do not have to be independently studied or solved; they **generate** a response as they reanalyze their prefix, that is, by revising the *same model* on earlier material that is being **interpreted**. It is impossible to say they are the last word in performance for linguistic exchange with the computer, whether such models are here to stay, but it seems certain that we have learned through them something over and above performance.

These modeling practices are becoming increasingly more relevant to linguistics. Linguists of next generation must build skills to understand the methodological subtleties of these models. They will reach a saturation point, with diminish-

⁴⁴One quick way to find out what kind of a transformer model you are dealing with: If you see two stacks in the model, it has got architectural encoding. If you see only one, it is decoder-only architecturally. All have input-output encoding decoding anyway, so it is the architecture that makes the difference.

⁴⁵The term **analysis** is yet another source of confusion in science and mathematics. Literally it means ‘untie’, but, in contrast with **synthesis** with which it is commonly compared, which means ‘already put together’, it also means putting things together. We can go the category theory way and treat the term as structuring of comparison, leaving in what direction we do that (‘untying’ or ‘through combination’) to variants, for example to parsing and generation. This is another way of saying that we should not equate analysis with parsing. Hippocrates of Chios (death 410BC), who basically invented this abstract conception of analysis, would be happy.

ing returns after that. More data will not suffice. The **architecture**⁴⁶ of encoding and decoding, however, is more than data. Enter linguistic ideas.

An example would be quite telling. In studies of **syntax** in connection to **semantics**, we have discovered that in human languages there are **bounded dependencies**, for example the dependencies manifested by **control verbs** where a **head**, namely the control verb, determines the dependency and keeps it in its local domain. For example, (82) means that the student studies in all examples, although some intervening material gets in the way in the second and third examples, and the way we get to that dependency in (82c) is altogether different.

- (82) a. Mary persuaded the student to study. English
- b. The student which you think I believe Mary persuaded to study arrived.
- c. Mary persuaded the student to begin to try to study.

That is because the dependency of ‘student’ and ‘study’ is mediated by *persuade* in the first two examples, by ‘student’ and ‘begin’ in the third example; only through that dependency and with ‘try’ we get that the student studies in the third example.

An **unbounded dependency** can cross clause boundaries without having to be an argument of the any structurally intervening clause, and it is not always headed like the unbounded dependency in the relative clause of (82b), the other dependency in that example:⁴⁷

- (83) This student I know you feel Mary persuaded to study arrived.

Such dependencies had been notoriously difficult for early LLMs. The way they rose above this difficulty is not a custom solution to the problem, and that is where modeling ideas and linguistic ideas begin to interact.

⁴⁶This term is also loaded with many interpretations, from physical properties to ontological ones and beyond. My favourite interpretation is perhaps best exemplified by Amdahl et al. (1964), who defined *computer architecture*, for the first time, to be the programmer’s view of the computer at a functional level, in their own terms: “the conceptual structure and functional behavior.” That is what good architectures are: abstractions for the sake of abstractors. It gives a functional view of something, whether that thing is a substance or idea. In this sense it is quite mathematical.

⁴⁷In one analysis it is headed by **topicalization**; unlike Japanese, English has no word or particle for that.

It may appear that the newer generation of LLMs solve the problem by making the moving window of **attention**⁴⁸ large enough in terms of token count, but that is only half the story (they did increase the window/vector size), for everyone knows that no moving window of fixed size could explain the behavior. They have learned to retain good continuations and forget not-so-good ones, and these good continuations are measured by what completions remain admissible, in other words, how many dependencies remain unresolved. I am not suggesting that they approximate what humans do. They tell us something about the relation between the number of dependencies that can be kept unresolved—by repeatedly recomputing an internal state in the architecture—and continuations that maintain constraints that are relevant to future continuation.

Whether that is the direction of further developments remains to be seen. I asked this *quo vadis* question to one of the LLMs, and its response, in its words, was that the next generation may do less of sequence prediction and more of maintaining continuation-relevant invariants over a growing prefix.

You be the judge for further linguistic involvement in the process. I would start with the following question: Why is (82c) grammatical, trickling down to the dependency of ‘student’ and ‘study’ through ‘begin’ and ‘try’, whereas the following is hopelessly ungrammatical although it seems to involve same kind of embedding with same kind of infinitival clauses?

(84) *Mary persuaded the student to study to begin to try.

Appreciating good abstractions is a bit like appreciating grandma’s recipe for cooking: not too much, not too little. In this chapter we have established some time-honored **abstractions** on which later notions of **admissibility** are built.

⁴⁸Here is another magical word. Attention is an architectural general measure to see which earlier pieces of the sequence would influence the current assessment of admissibility *now* in the process of interpretation and production; it assigns weights to earlier tokens for every token based on that.

2 Mathematics and philosophy of linguistics

Linguistics asks **fundamental questions**. These questions are not addressed by general questions in science. It is in this sense that **philosophy of linguistics** has a natural niche in philosophy of science, not only because philosophers have had longstanding interest in language.

Linguistics appears to be one of what philosophers of science call **special sciences**. These are the sciences which cannot be explained away solely by inference from general mechanisms, from general principles.¹

An early example from linguistics is **language acquisition**. Every child that can be thought to be exposed to linguistic and nonlinguistic world early on acquires a natural language starting around the age of one and beginning to come to shape in productivity around the age of two. There are variations of course, but this observation seems to hold independent of language type, richness of vocabulary in a language, and educational or financial well-being of the child carers that provide child-directed speech or sign.

There are also cases which are quite tragic yet revealing, for example what happens when a child is not exposed to such a world from early on, for an extended period. Many children are exposed to music, painting and complex problem-solving (e.g. chess) very early, yet not all of them develop into a Mozart, Picasso or Capablanca. In language they all do. These differences are difficult to explain without some special principles, therefore a special science.

Mathematical thinking in linguistics suggests that this constrained behavior can be understood as an invariant yielding admissibility conditions for variants to the extent that these differences begin to make sense in **method selection**, in effectiveness, efficiency, feasibility, etc. Their correlates can then be sought in empirical domains like **biology**, **psychology** and **sociology**.

For example, if we are to seek an **evaluation metric** to choose one grammar from many **descriptively adequate** grammars, as **Chomsky (1965)** called them, to reach **explanatory adequacy** in his terms, we would know that admissibility

¹**Salmon et al. (1992)** is a comprehensive introduction to philosophy of science. A more historical and critical overview, including critique of **Eurocentrism** in science, is **Koestler (1959)**.

is restrictive enough and we are in the domain of variants to understand how a child converges to some candidates and not others *given* the exposure to some language in a limited timeframe. Even the terminology makes it clear that we are talking about the distinction between making the invariant restrictive enough and making the variants restrictive enough. For the invariant, computing power is not the concern (but connectedness is, including the connection to the world of choice); for the variant, admissibility is not the main concern but computing power is, including its dependency on the linguistic *and* nonlinguistic world, that is, its dependency on an **encoding-decoding relation** .

Incidentally, Chomsky called these candidate grammars **strong generative capacity** manifesting understanding of **structural descriptions**, compared to **weak generative capacity** capturing the stringset of a language independent of description. The distinction will be useful when we come to method selection in later chapters.

We may need invariants in linguistics if we want to see the consequent variants of the invariant so that we can avoid *unnecessary* commitments to some ontology. Such avoidance would preempt proliferation of further commitments to that ontology which would get more and more difficult to retract from. An empirical claim, that is, a claim of sufficiency, may then connect them to some kind of ontology, say to **psychology, biology, social semiotic, logic or geometry**. The book is an opportunity to live through the experience yourself to prepare for hard empirical problems.

2.1 Philosophy of linguistics as philosophy of science

The philosophical issues discussed below are not digressions from mathematical thinking in linguistics; they explain why mathematical abstraction occupies such a central role in the rest of the book.

Katz (1985b) effectively introduces the **philosophy of linguistics** as a field of inquiry. This classic weighs whether linguistics is a branch of **psychology, acoustics**² or mathematics, roughly corresponding respectively to **conceptualism, nominalism** and **Platonic realism**. These schools are historical, and I resist the temptation to freeze them at some stage of their development and provide definitions for moving targets. The reader is encouraged to check their history of development from philosophical sources such as the *Stanford Encyclopedia of Philosophy*.

²equivalently, ‘gesture’ spaces rather than sign spaces.

Katz himself considered linguistics to be a branch of mathematics also spanning logic. The reader may be familiar with later claims, for example linguistics being a branch of biology or even claims of exogenesis for language in perfection. I have hinted at **conventionalism** with references to Poincaré in the beginning of the chapter, and from there **structural realism** seems to be within reach for identifying where linguistics belong. These ideas will be touched upon when discussions come up. Old-fashioned **realism** is much easier to connect with given what [Heisenberg \(1958\)](#), [Penrose \(2006\)](#) have said (see §1.12), and we do not have to be physicists to make the connection for linguistics, Plato would be enough: We do not really know what **physical reality** by itself would look like, we do not really know what **mental reality** by itself would like,³ but we can feel them by seeing how they may connect with our linguistic equations, how we can revise our theory and models if we have to when we see something not connecting well, in the process deliberating alternative views and doing **model selection** rather than adopting the winner-takes-all (or rather, loser-loses-all) attitude of **positivism**. For that we need to know **admissibility** conditions of an **invariant** so that alternative views can be built as **variants**. This would give a meta perspective on **method selection**, the main theme of the book. It is inevitably tied up to variants but meaningful only when we understand the invariant. That is why **mathematical reality** is the one that connects the dots in knowledge construction.

This is not yet a question about methods; it is a question about the conditions under which methods become meaningful.

Mathematical thinking which is promoted in the book does not presuppose that linguistics is a branch of anything, not any more than thinking that physics is a branch of mathematics because theoretical physics is all done in the language of mathematics.⁴ The language of mathematics itself is full of presuppositions according to [Ganesalingam \(2013\)](#) (about sets, rules, countability etc.). We have been cautioned about this vigilance all along, by Gödel, Einstein and company of [Poincaré \(1905\)](#), [Hadamard \(1945\)](#), [Wang \(1974\)](#), among others.

The book offers one package of choices for mathematical methods which are put together with this way of thinking. In the choice, **grammars as abstract objects** are considered necessary to understand intralinguistic structures but not

³For linguistics in particular, see for example [Sandra \(1998\)](#) for potential fallacies on two sides of the spectrum of psychological commitments concerning grammar, **generative linguistics** (syntactic generalism) and **cognitive linguistics** (semantic specialism).

⁴Here is what Bertrand Russell said about the relationship between mathematics and physics: “Physics is mathematical not because we know so much about the physical world, but because we know so little; it is only its mathematical properties that we can discover.” [Russell \(1927\)](#).

sufficient to understand their crosslinguistic variation. And **connectedness** is considered sufficient to study language acquisition but not necessary to understand its crosslinguistic diversity. In Mackie's terminology, grammar would be the agreed upon cause (not necessarily physical cause) and linguistic variation and timeline of language acquisition would be the conditions to explain.

This kind of cause is not physical cause but an abstraction for causal attribution to make an argument, with parties involved. From here it is not too far to reach the Platonic dialogue as it was intended before neoplatonists covered it in a shroud of mystery.

Cross-linguistic variation and the acquisition timeline as outcomes in this sense are licensed by a configuration that satisfies admissibility conditions. Admissibility conditions are not about being necessary and/or sufficient for something to materialize. Structural admissibility shapes the variants, including choice of methods for them.

Altogether it aims to help clarify one position of linguists; it consists of two things that linguists have difficulty explaining to others: (i) Language acquisition appears to be what happens to the child whereas language learning is what the learner does; (ii) There seems to be a common ground in the structure of linguistic diversity; it is not a domain in which anything goes. This is why linguistics is a natural science.

2.2 Ontogeny of grammaticality

Ontogeny concerns the development of an individual organism, as opposed to **phylogeny**, which concerns evolutionary history.

Considering the commonly held assumption that grammaticality is one fundamental problem that grammars address, it would be tempting to think that, if the decision processes of grammaticality were invariantly mechanical, then not only the admissibility conditions of grammaticality would not change over time; also, whatever develops as knowledge of grammaticality would be just further judgments on variants—which can become more and more complex as the child develops, but the *method* of grammaticality in the variant would be unchanged as well.

We have reasons to believe that it is time to revise that assumption; in fact, my purpose of covering systems theory (§6.5) in an introductory book on linguistics is primarily motivated by these reasons. Concern for **connectedness** in the **invariant** demands a concern for it in all its dimensions related to connectivity including **encoding-decoding relations** (see §1.8 for encoding and decoding). We

would be concerned with these aspects if we assume a realist position one way or another, with or without abstraction.

Consider the following examples from child language production for a start on discussion of grammaticality (child age around 2;3):

- (1) a. It colds my bottom. Clark 1982:392
 b. Mummy trousers me. (talking about getting dressed) :406

Although we can envisage an isolated community in which verb novelties such as (1b) can survive into adulthood for the child to hear it, it seems less likely that the verb in (1a) would have been heard by the child from child carers. Children are not parrots. It is more likely that the child rises above what is heard from child carers by leveraging representations of sentence-world pairings rather than word-world pairings (Gleitman & Gleitman 1997), making words up in the process.⁵ Otherwise, they would have just used adult words, rightly or wrongly. Computational models of the process, that is, what its variants could be, have been proposed, for example Abend et al. (2017). That abstraction, if it is real, needs a closer look into the process of deciding grammaticality.

It is a small step to arrive at grammaticalities that are difficult to make sense of unless one also looks at **semiotic conditions** in which such utterances are freely made. Aitchison (1987) comments on the following data from teenagers in Reading, UK, frequently attaching “a nonstandard -s to verbs *which are typical of their subculture*” [my emphasis]:

⁵It seems that some Gestalt principles are at work here; for example, Wertheimer (1924) defined the program as starting with the whole which is presumed meaningful—because it is a whole—and reaching the parts that would make it so, rather than starting with parts to see if they compile into something meaningful. Linguist’s and psychologist’s position regarding Gestalt has been more or less dismissive if not disparaging, for example claiming that an ungrammaticality judgment from a native speaker could be interpreted in some cases as “rather vague, gestalt-like impression, .. the sentence may simply sound bad.” (Birdsong 1989:110). No gestaltist (e.g. Koffka 1936, Köhler 1938, Arnheim 1954) suggested wholes to be unaccountable, vague holistic impressions; their suggestion was to turn the table around, that we seek parts only because understanding a whole makes the process itself meaningful. At the risk of extending a footnote that started on something else, I must quote from its manifesto to put the historical injustice in perspective:

The fundamental “formula” of Gestalt theory might be expressed in this way: There are wholes, the behavior of which is not determined by that of their individual elements, but where *the part-processes are themselves* determined by the intrinsic nature of the whole. It is the hope of Gestalt theory to determine the nature of such wholes.

Wertheimer 1924

As such, Gestalt transcends psychology and linguistics. It is a philosophy of human behavior.

- (2) We fucking *chins* (hit on the chin) them with bottles. Aitchison 1987:111
I *legs* it (run away) up Blagdon Hill.
We *kills* (beat in a fight) them.

“They also attach this -s to other verbs, but not so often.” (*ibid.*) We are here facing a problem not of native speakers according to Ryle (1937); it is caused by the unavoidably freer nature of second-guessing of the performer by the analysts, same as we do when we look at child data. Some analysts would consider the examples ungrammatical, some others would seek another source and way of explanation for their grammaticality, as we have seen in the preceding example. Explicit flow of ideas in the form of a dialogue for resolution by the parties, that is, some **social semiotics**, would always help. In this sense **knowledge** is what we arrive at through dialogue, a reasoned explanation, other things being **information**.

Although the idea of social semiotics is very old (e.g. Bhartṛhari, Plato, Aristotle), its recent resurgence has been shadowed by **logicism** and **psychologism** in meaning. For Sapir (1924/1949), Ryle (1937), Geertz (1973), all meaning is not first created in one mind then projected onto society, a common network of meaning-producing subjects create communities (e.g. Sapir’s insistence of talking about psychological “spaces” rather than “states”), to the extent of affecting grammaticality. Poor Sapir was later pigeonholed into **linguistic relativism** but he did not claim that communities build and live in their own cocoon of meaning spaces, he suggested, much like Geertz later, that people of the one and same world create different **meaning networks** for **meaningfulness**. The modern linguistic impact on this idea is that that is consequent to **grammaticality**. This concept is not just a name; as an abstract object it is a condition for meaningfulness.

In studying human “behavior” the dialogue is all about how we analysts conceive the notion of category, as something rock solid and mechanical or something fluent yet persistent and pervasive. Here is what Ryle said about the process:

- (3) Is there such a thing as analyzing or clarifying the meaning of the expressions that people use, except in the sense of substituting philologically better expressions for philologically worse ones? (We might have put the problem in the more misleading terminology of “concepts” and asked: How can philosophizing so operate by analysis and clarification, upon the concepts used by the plain man, the scientist or the artist, that after this operation the concepts are illumined where before they were dark? The same difficulties arise. For there can be

no such thing as a confused concept, since either a man is conceiving, i.e., knowing the nature of his subject matter, or he is failing to do so. If he is succeeding, no clarification is required or possible; and if he is failing, he must find out more or think more about the subject matter, the apprehension of the nature of which we call his “concept.” But this will not be philosophizing about the concept, but exploring further the nature of the thing, and so will be economics, perhaps, or astronomy or history. Ryle 1931:141

In other words, in employing **categories** in our theories we are in the business of understanding how people put them to use, not what they ought to be, because the performer already knows what they are, knows what to do if she does not.

An **INUS explanation** (§1.10) through categories is the process of convincing the parties involved for a **causal explanation**. In our particular case, the **linguistic categories**, the process involves **grammar** as a necessary but insufficient mechanism for **meaningfulness**, itself being the source of **grammaticality**. The sufficient mechanism to explain is the **linguistic meaning**, which is unnecessary but sufficient to hold on to to build on. These can pave the way for a **Platonic dialogue** of construing **social semiotics**. Apart from **activities, events, states and processes, accomplishments and achievements** demand a **social understanding**. In principle, a grammar can be made as transparent as it can be through these categories, a personal understanding of the social conditions and the world around us, as a consequence of **connectedness** as the source of **admissibility**. Naturally, from the perspective developed here, we need a mathematical understanding of these concepts first.

Ryle is also known for a striking statement about the seemingly unfathomable relation between knowledge and categories:

(4) **Knowing that presupposes knowing how.**⁶ Ryle (1937)

That is what categories are about, including **linguistic categories**: They are about *how* we reach or express **meaningfulness** as a consequence of grammaticality

⁶Heidegger (1927) is another philosopher who prioritized “how”. In fact Ryle was among the first in the non German-speaking world to pay close attention to Heidegger. However, while Ryle was trying hard to make ideas more accessible and transparent, it is hard to say the same thing about Heidegger, that being one of the points of disagreement between them. The idea of **category mistake** is one of Ryle’s points in efforts of clarification, in addition to the quote we have just seen. Roughly, making a category mistake is mistaking one level of category with another level. His now classic example is a visitor making a mistake by asking *but where is the university?* after having shown the buildings and the facilities of the university.

and structured behavior. They are indeed abstractions, not distributional labels or posthoc labels, not even **judgments** (more on that at the end of the section).

If second-guessing is unavoidable as long as we are prepared to call on proxy devices and shift of categories when it is necessary to understand knowledge of language, including its role in the process of arriving at grammaticality, then we are facing the same problem in the Chomskyan classic (5a) and (6), although the latter appears to be dealing with a different kind of problem.

- (5) a. Colorless green ideas sleep furiously. English
 b. *Green furiously ideas sleep colorless.
- (6) a. *She won the competition for an hour.
 b. She won the competition for a year.
 c. She played the piano in a year.
 d. *She played the piano in an hour.

Recall that (5a) has been used to show that grammaticality does not have to make sense; it just has to be meaningful. Same words are involved in (5b), yet we do not attempt making sense of it precisely because it is not grammatical.⁷

In the other example, it is not the change of duration or change of preposition that seems to be making the difference, it is the shift of reference from activity (6a) to accomplishment (6b). The shift is **categorical** in Rylean sense, showing how the difference is made.

The second-guessing game still plays out. Imagine (6d) to be grammatical. One way to make it possible is to think that she must be gifted, that is, we would insist on reference to accomplishment, gifted because it is such a short period *for an accomplishment*. We can play the same game of making things meaningful by reference shift in (5a), for example having environmental (i.e. green) ideas that proved useless (colorless), us having uneasy sleep over that state of affairs. The critical question concerning categories is the following: Why don't we play the same game of making sense of (5a) when the expression is hopelessly unrecoverable grammatically, as in (5b)?

The hopelessness of recovery in such examples, that is, lack of finding another reference anchor, may be attributed to English being a rigid word-order language,

⁷The difference of grammaticality in such examples had been used in early generative studies to point out that it is not a question of probability of words since the same words are involved in (5a) and (5b); but, that is unhygienic (if not dismissive) use of probability theory, because without knowing whether we are assuming a joint distribution of forms and meanings, the discussion would be ungrounded; see §1.6 about the linguistic significance of different kinds of **probability distributions**.

where word-order effects would be felt more severely. Turkish is very flexible in word order, yet we still have the same hopelessness in the following (7a) versus (7b).

- (7) a. Mehmet Ahmet'in hiç okumadığı kitapları çok
 Mehmet.3s Ahmet-3s none read-NEG-REL-3s books-ACC very
 seviyor.
 like-PROG.3s
 'Mehmet /very much likes/is very much liking/ the books that Ahmet
 never read.' Turkish
- b. *Hiç okumadığı Mehmet kitapları çok Ahmet'in seviyor

Ungrammaticality is a structural problem, structure being **configurational** and **referential**—recall from footnote 5 Gestalt's *structuring* of the parts by reference. Their competence develops over time in the variant, but structure-dependence of configuration and referential-dependence of category choice probably do not. They may be the part of the invariant because, had they develop over time, we would have had to have seen much greater variation in timeline of development of their competence across languages.

At the end of this section we can go back to times when grammaticality began to take center stage in linguistics, beginning with Chomsky (1955/1975). Schütze (1996) is a comprehensive summary of history of ideas around grammaticality and acceptability. The asterisk (‘*’) notation for ungrammaticality that we take for granted today was meant originally to mark forms that were quite obviously ungrammatical by anyone's standards (Householder 1973). In hindsight it seems that it was optimistic to assume that psychological **judgments** can satisfy everybody's standards.

The field never felt easy about the concept of grammaticality, including its modern developer, Chomsky. It can easily be conflated with acceptability, not just by the native speaker but the analyst too. If Ryle is right, arguing decades before the linguistic debate, then the conflation seems inevitable. If we ask for judgment, how can we be sure that we are not getting acceptability reports, no matter how explicit our instructions are? Husserl's (1900) proposal for a better alternative even earlier—because categoriality of human thought had been discussed since Frege (1879)—is to posit **categorial jumps** that rise above sensuous material.

- (8) We could not speak of something super-sensuously or categorially perceived, if we could not imagine this thing 'in the same manner' (i.e. not merely sensuously). We must therefore draw a quite general

distinction between sensuous and categorial intuition (or show the possibility of such a distinction). Husserl 1900, Vol.II:281

To see the difference between grammaticality and acceptability, we often appeal to the principle that “it must be shown that the conditions that make it [grammaticality] less than acceptable are extragrammatical, i.e. that they exist independent of language structure” (Schütze 1996:23).

Are categorial jumps to the extent of changing event reference and event type (e.g. activity and accomplishment) to make an expression meaningful (not senseful), *because* of newfound grammaticality, extragrammatical? Knowing that **first-person psychology** of the Husserl kind is even more difficult to practice than common isherian **third-person psychology**, an appeal to mathematical abstraction without escapism is one way to stick to Russell’s dictum (footnote 4), that we turn to mathematics when we know so little, not when we know so much. The more we know about grammaticality as an abstraction, the more we ask better questions about it in the variants.

One mathematical concept in systems theory covered in §6.5, the idea of **open-loop**, will prove relevant to situating categorial jumps in the discussion of grammaticality, therefore grammar. For our current purposes, we can reiterate it as follows.

A **closed-loop** in the systems view of self-organizing bodies is a system which situates itself in the world (i.e. responds to it) always depending on **feedback** and input from the environment. In contrast, an open-loop system **chooses** a niche, an output without dependence on input; in other terminology, it chooses desirable input rather than “take” the input in, in response to some but not all of the feedback from the environment.

The operative words here are “choose” and “desirable,” altogether one reading being “meaningful.” It is one mathematical way to reveal that choosing a category and deciding on its consequences (such as grammaticality) are different *acts*, in systems terminology, different controls. With this in mind we can reassess the reports of (un)grammaticality in this section as referentially recoverable (i.e. categorially recoverable) controls versus hopelessly unrecoverable (i.e. categorially unrecoverable) controls, which I leave to readers for self deliberation before reading §6.5.

Grammar is neither identical with **biological mechanisms** nor **computational mechanisms**. They are candidate realizations. **Grammaticality** itself remains an empirical phenomenon requiring explanation.

2.3 Some questions that make linguistics a special science

In the beginning of this chapter we have seen that language acquisition is one domain which seems difficult to explain by recasting linguistic questions as questions of other sciences. In the case of linguistics these sciences are **biology**, given that only human children develop language, and **psychology**, given its central role in studying thought, assuming language is expression of thought. We have seen in the previous section that that assumption may be too narrow, that language may be more than **expression of thought**, given the distinct act of **category choice**. Nevertheless, uniqueness of language acquisition has been the main impetus for drawing linguistics closer to natural sciences, if we take it to be a fundamental problem to study. We owe this to [Lenneberg \(1967\)](#).

What makes **language acquisition** special is that children are exposed to many other things involving thought and organic development from day one to say the age of two years, for example music, painting and problem solving; yet, it is language that comes to full production *in all children* in that period, not only in specially gifted children, give or take months depending on amount and kind of exposure.

Linguistic asymmetries are quite unique in other dimensions as well. For example, in languages where **relativization** is achieved by extracting an argument from a subordinate clause, for example (9), extraction shown informally with ‘_’ where the object would be if it weren’t extracted, we have seen no cases where we can extract the object but not the subject. This is part of the **noun phrase accessibility hierarchy** which extends this asymmetry to indirect objects versus subjects and others ([Keenan & Comrie 1977](#)).⁸

- (9) The cat that [the dog chased _] ran away. English

Extraction phenomena is also unique in another sense. In languages with extraction, we have seen no cases where a verb can be extracted, whereas all kinds of nonverbs can extract. Logically speaking, there seems to be no reason for this gap in the data.

Agreement hierarchies are similarly asymmetric ([Moravcsik 1974, 1978](#)). In languages with agreement, there seems to be no case where object or indirect object agrees with the verb but subject does not.⁹

⁸There are case-specific extractions where the relative pronoun and extracted argument bear the same case, as in Russian. This is a different matter than asymmetry, an idea which applies to a range of arguments.

⁹There are lacunas in such phenomena where for example agreement is blocked when a certain noun is definite or indefinite.

These asymmetries cover ranges of arguments; but, there are also **asymmetries** with one argument type only. For example, there seems to be no language in which we have genuine subject reflexives, whereas object reflexives that are bound to subjects abound, for example *John cut himself*; cf. **heself cut John*. Logically speaking, $cut(x, x) \wedge x = \text{John}$ would be fine but this admits too much nondata.

These all sound very syntactic and semantic, but we have also seen no language with **morphological operators** where a verb in root form is a full proposition, although nonroot forms as full propositions are common for example in prodrop languages:

- (10) uyu-du-m
 sleep-PAST-1SG
 ‘I slept.’ Turkish

Last but not least, consider the **lacuna** in number of arguments for any verb depending on **argument type**. Although logically speaking we can envisage any number arguments for a verb, for example four for *bet* (11a), there seems to be no verb in any language where the number of full clausal arguments is more than one,¹⁰ although we can envisage one full clause and other dependent clauses as arguments, for example (11b), (11c); cf. (11d), which is ungrammatical as a single clause, and (11e).

- (11) a. I bet him five euros that the World Cup final goes overtime. English
 b. I convinced him that the World Cup is this year if not next year.
 c. She told me that if World Cup is this year, we should watch it.
 d. *She said that we may win that we are strong.
 e. I think/believe that we may watch the final.

Again, there is no logical reason for this lacuna, but it is there to be understood. The current consensus is that these kinds of unusual constraints call for language-specific **admissibility principles**. A complete list of phenomena suggesting special admissibility principles is not my purpose. It suffices to know that it is difficult to reduce study of fine structure in language to principles and admissibility of another field in science.

¹⁰Mark Steedman, personal communication.

2.4 Mathematics in linguistics versus mathematicizing linguistics

A principled division of labor in the complex problem of explaining natural languages is the following:

- (12) Mathematicizing linguistics begins with **methods**. Mathematical thinking begins with **admissibility**.

Mathematics is not privileged because it computes. It is privileged because it distinguishes necessary structure from contingent realization. Let us look into some historical developments in this regard.

Some philosophers such as [Katz \(1985b\)](#) consider linguistics to be a branch of **mathematics**, not a field in its own right. [Katz \(1985a\)](#) proposed that grammars are **abstract objects**, that is, they are abstractions, therefore they belong to a Platonic realm of **mathematical reality** (not to neoplatonic realm of deliberate mystique).¹¹

We have seen some questions in linguistics (§2.3) that are difficult to study as unexpected gaps if mathematical abstraction was all there is to **linguistics**. Linguists' appeal to mathematics may not be too far from physicists' appeal to mathematics without mathematicizing physics,¹² or any other field for that matter. Mathematical methods in physics solve physics problems, they are about the variants; and, mathematical thinking in physics solves problems of knowledge in physics, they are about the invariants that provide admissibility for variants in the first place, to make them studiable.

The distinction and its relevance to linguistics were made clear by Chomsky very early on in the following quote, which is probably the most freely interpreted statement about the nature and role of linguistic method in linguistics:

- (13) [T]he critical factor in the development of a fully adequate theory is the limitation of the class of possible grammars. Clearly, this limitation must be such as to meet empirical conditions on strong (and, a fortiori, weak) generative capacity, and, furthermore, such as to permit the condition of explanatory adequacy to be met when an appropriate evaluation measure is developed.
⋮

¹¹I am following [Linnebo \(2017\)](#) in mentioning Plato's ideas with big 'P' and some of his later interpreters' ideas with small 'p'.

¹²Recall the Russell quote in footnote 4.

[M]athematical study of formal properties of grammars, is, very likely, an area of linguistics of great potential. It has already provided some insight into questions of empirical interest and will perhaps some day provide much better insights. But it is important to realize that the questions presently being studied [generative capacities] are primarily determined by feasibility of mathematical study, and it is important not to confuse this with question of empirical significance.

Chomsky 1965:61–2

Apparently some linguists took that to mean that empirical work including its impact on linguistic theory would not need as much mathematics as it did in its early development. We can see the effect of this way of thinking from historical development of **linguistics curricula**. Once formal methods were introduced to linguistics in the 1960s, there were undergraduate courses in for example formal languages and automata theory, also, type theory, in departments of linguistics. And then, these courses vanished from the curricula around mid to late 1980s. It has become a bit of irony that computer science undergraduates receive more exposure to Chomsky's early work on mathematics of generative capacity, for example the **Chomsky hierarchy of languages**, than linguistics undergraduates.

It is a pity because consequent developments in linguistics, such as **parameters** and **feature geometry** in **generative grammar**, **function space geometry** in **categorical grammars**, and comparison of **monostructural theories** with **multistructural theories**, would have benefited from joint discussion of their abstractions rather than create separate identities and sects because of seemingly necessary reifications and methodological commitments which only created schisms.

It is time we regroup to face more challenging problems of today, and in that effort I suggest that mathematics is once again our best hope in reassessment of what we have learned since the 1950s. This time it requires some knowledge of **scientific modeling** as well. That is because, although not everyone is a modeler in science, especially theoretically inclined scientists, **modelability** of ideas is necessary for everyone, from practitioner and agnostic to theorist, otherwise scientific ideas would be left to goodwill of believers and practitioners only.

Mathematics is a language for discovering problems, rather than merely solving them. It can be made transparent to the extent that it facilitates wider community effort.

3 Degrees of structure from string admissibility

The easiest way to conceive the abstraction in **structural embedding** is the problem of matching parentheses: Every opening parenthesis must close, and close only after nested open parentheses close. It is what makes parenthesization meaningful. Notice that this **meaningfulness** does not refer to content; it refers to structuring of content. It is an abstraction, an abstract suggestion about reality, reality because we can work with it and organize our lives around it. We change the world by using them. Matching parentheses is not interesting because parentheses exist in language; it is interesting because it isolates the abstract notion of **dependency**.

The current understanding of **dependency structure** is the domain of **string admissibility**. It is a theory about what kind of strings can be structured to be meaningful, and to what extent. It will be seen that admissibility can be conceived free of a **descriptive language** and independent of **computation**. In that sense, it seems that we may have reached an **invariant** for structure-bearing meaningful human things we call **strings** (of linguistic expression).

The “after” in the description of the matching-parentheses problem is therefore an abstraction, a structural relation. It is not a linear metric of number of left and right parentheses. We shall see in this chapter that all the formal methods we employ in linguistics are dependent on the **degree of structure** we expect to understand. Therefore degrees of structure must be made explicit.

We are about to cover topics that have been standard in computer science, and, up until the late 1970s, in linguistics, mathematics of which is explained in much more detail in well-known textbooks than here which have become standard reference, for example [Aho & Ullman \(1972\)](#), [Harrison \(1978\)](#), [Hopcroft & Ullman \(1979\)](#), [Lewis & Papadimitriou \(1998\)](#), [Sipser \(2006\)](#), [Sudkamp \(2006\)](#). I will refer to a specific resource for further reading on topics under discussion.

3.1 Cutting the Chomsky hierarchy at different seams

Figure 3.1, representing the Chomsky hierarchy, is one succinct summary of formal ideas we have accumulated in more than six decades, which we have come to associate with principled choice of mathematical methods for a particular kind of embedding. The hierarchy classifies **admissible structures** rather than languages themselves.

This identification is important because if we want to *do* formal analysis with a certain kind of embedding, that is, if we want to **model** the analysis or want to show its feasibility, the consequent choice of automata (i.e. the computational variant) would be the least powerful mechanism that would be sufficient to do such analyses. That is crucial because, modeling-wise, more powerful mechanisms would be more resource-demanding, making **model selection** more difficult than needed, and theory-wise, science-wise, they would be unnecessary as part of an explanation. (See Chapter 1.12 for modeling concepts.) This chapter is about understanding what these aspects mean.

The first thing to note about the hierarchy is its terminology. For a mathematician, a language is a set with certain properties. Structure comes into play with the qualifier: certain properties. Mathematician's language altogether is linguist's collection of structured descriptions.

A formal language in the figure is not a set of structural descriptions. Every language in a family of languages, say context-free, is a set of strings, therefore the family of context-free languages is the set of set of strings that satisfy the context-freeness property for all its strings, which we shall have a closer look soon. Admissibility is where we are concerned with structural descriptions as invariants (§3.2).

As sets and sets of sets, many of them **arbitrarily large**, these families manifest set properties. For example, the bottom of the hierarchy, the family of regular languages, is contained in every superset. Assuming the classical division of families (solid lines), all regular languages are also context-free, context-sensitive and recursively enumerable, all context-free languages are context-sensitive and recursively enumerable, and so on. One important take away from the figure is then the following:

- (1) More structure does not mean more strings.

The reason for the dotted lines in Figure 3.1 is that we have found that some natural languages are provably not context-free in their syntax-semantics, for example Swiss German (Shieber 1985):5:

3.1 Cutting the Chomsky hierarchy at different seams

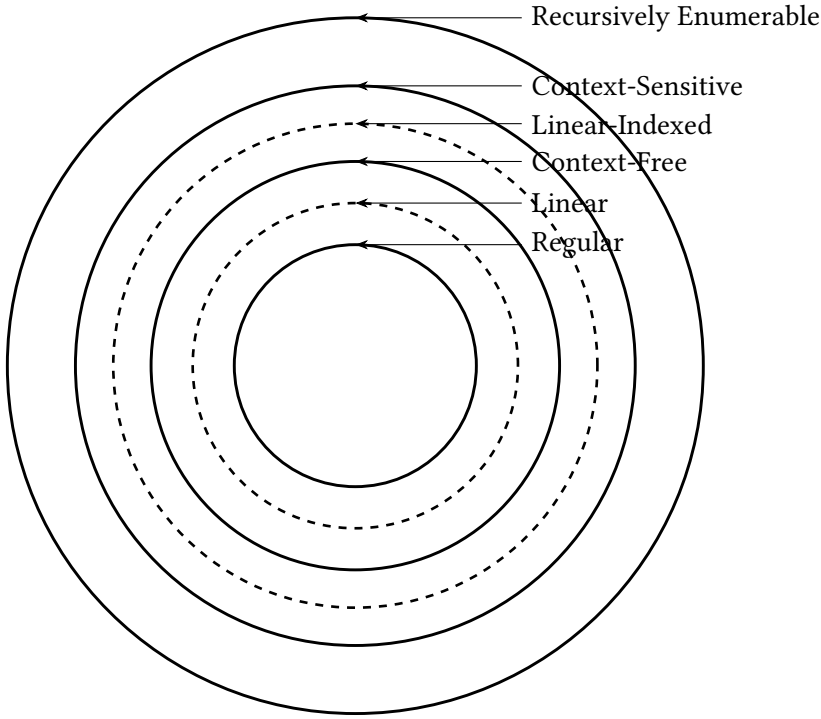


Figure 3.1: The Chomsky hierarchy of family of formal languages. Every formal language is a set as a data point in this hierarchy. Solid lines are the classical set inclusions for the families, from Chomsky (1959). Dashed lines are new families that are found to be better suited for finer characterizations of natural language syntax and morphology, respectively linear-indexed languages and linear languages. Classical results are described computationally by Sudkamp (2006). Linguistic significance of new results are described by Gazdar (1988), their computational significance by Joshi et al. (1991), Kallmeyer (2010). Their mathematics is described by Hopcroft & Ullman (1979).

- (2) Jan säit das mer d'chind em Hans es Huus lönd hälfe
 Jan says that we₁ children-ACC₂ Hans-DAT₃ house-ACC let₁ help₂
 aastriiche Swiss German
 paint₃
 'Jan says that we let the children help Hans paint the house.'

To follow the semantics given in the gloss, we mark who is doing what to whom (i.e. **dependencies**) in syntactic structure using indices, giving the dependency sequence $s_1 s_2 s_3 v_1 v_2 v_3$ where s is 'subject' and v is 'verb', and s_1 means it is the subject of v_1 etc. The important point of the proof is that Swiss German speakers

can do this indefinitely. We shall see in the next section that this structural pattern of dependency is inadmissible in any context-free language. If one natural (acquirable) language is then provably not context-free in its **string admissibility**, since all languages are capable of manifesting the same property, then human languages are not context-free. That is the main line of the argument, details of which are not our concern in an introductory book. The point is not merely that Swiss German exceeds context-freeness, but that **empirical evidence** constrains our **admissibility** assumptions.

The inner dashed line in Figure 3.1 is less obvious. Some motivation for rising above regular languages for morphology are nonconcatenative morphology, for example templatic Semitic morphologies (Kiraz 2001), **infixation** in Tagalog and **circumfixation** in German, and **reduplication**, which is a morphophonological phenomenon about repeating a segment, for example Latin *cu-curr-i* ‘I ran’, *mo-mord-i* ‘I have bitten’ (Matthews 1974:128), or Turkish *masmavi* ‘deep blue’ from *mavi* ‘blue’, and *apaçık* ‘obvious’ from *açık* ‘clear’. There is semantics in these processes; for example, Turkish reduplication is basically an intensifier. However, they do not extend to morphology-semantics dependencies indefinitely as Swiss German examples do, that is, they are morphophonological; for example, **map-masmavi* although there is nothing in the segment *mas-* to avoid iteration phonologically, for example *mapmaskara* ‘very silly’ from *maskara* ‘silly’. In Tagalog we cannot voice and already voiced verbal form (Kroeger 1993). In some cases reduplication can apply iteratively to the extent of considering it not even context-free (Culy 1985). Yet we cannot help feel that they do not embed like syntax or semantics. This is a powerful linguistic intuition, and it will continue to guide us in method selection.

These are advanced topics, but we must keep in mind that the lines have been redrawn in Figure 3.1 because our notion of syntactic and morphological admissibility are affected by **semantic dependencies** in phrases and words. This is something to keep in mind when we come to putting things together in form and meaning in Chapter 6. We shall see in these chapters that context-freeness, which is a formal set-theoretic property, is only half the story in syntactically mediated semantic dependencies. This is another way of saying that any level of string admissibility is only half the story, because it just lays the ground on which we can build methods to model empirical theories because of our good understanding of admissibility as well-formedness.

For example the set $\{a^n b^n \mid n \in \mathbb{N}\}$ as a formal language can be proven to be context-free and not regular in the language space of Figure 3.1; see e.g. Sudkamp (2006) for the proof. It happens to be in the family of linear languages, the inner dashed line in the figure. However this is based on the *interpretation* of its strings

as manifesting the dependencies $a_1a_2 \cdots a_kb_k \cdots b_2b_1$ for some k where indices are dependencies, for example a_1 and b_1 depends, a_2 and b_2 depends, etc. but not say a_1 and b_k . If the interpretation of its strings is $a_1a_2 \cdots a_kb_1b_2 \cdots b_k$ it is not context-free although it is the same string, because if it were, we could build a pushdown automaton for it just like any context-free language (see sections §4 and §5), and with these dependencies we cannot do this. This is what it means when we say “embed like syntax and semantics.” That is why the idea of putting things together in methods in Chapter 6 presume string admissibility but does not stop there.

To sum up, **string admissibility** is an INUS condition on grammars for **meaningfulness** to arise. These concepts are neither necessary nor sufficient. We have to develop an argument. These considerations will affect our choice of mathematical methods.

Looking in the direction of outside-in in Figure 3.1, for example not all context-free languages are regular, that is, the space separating their lines is not empty—there are some languages in that space; not all recursively enumerable languages are context-free, etc., therefore these delineations and different names are for a reason: They characterize properties of all the languages in a family, and different families characterize different degrees of structuring a string.

3.2 Structure: admissibility of strings

What are these properties? The preceding discussion suggests that whatever they are, they are about strings. In this chapter we shall see that they give us different **admissibility conditions** for any string to be considered a *possible member* of a language in a given family. We shall see that they are not about the content but structure of the strings. In doing so we are not classifying **grammars**. We are classifying admissible structural possibilities over **strings**.

Mathematically they are known as **pumping lemmas** for each family identified in Figure 3.1.¹ I will eschew logical terminology and formal proofs to highlight the common linguistic idea behind these results; see Hopcroft & Ullman (1979) for an advanced level and Sipser (2006), Sudkamp (2006) for introductory expositions.

3.2.1 Context-free admissibility

All strings in any context-free language satisfy the following properties. They define what it means to be structurally embedding to the extent of context-freeness.

¹A **lemma** in mathematics is a stage in the proof of a theorem which is worthy of remembering for reference. Some lemmas receive so much reference that we sometimes forget to mention that they are also theorems.

- (3) Let L be any **context-free language**. (Pumping lemma for context-freeness)
- Then, if there are sufficiently long strings z in L , say $|z| \geq n$, for some n depending only on L , then we can decompose it as $z = uvwxy \in L$ such that
 - $|vx| \geq 1$,
 - $|vwx| \leq n$, and
 - for all $i \geq 0$, uv^iwx^iy is also in L .
 - If there are no sufficiently long strings in L , then L is a finite set (language). To avoid confusion, we should really call it not an arbitrarily large set; see §1.1 for a caution against the use of infinity-talk. This family, which is not shown in Figure 3.1, is strictly within the family of regular languages. There is no pumping lemma for them.

This is the formal definition of the **pumping lemma for context-free languages**. Let us dissect the definition to see its essence and why it is called that.

Notice that it only provides admissibility conditions for any sufficiently long string to be in a context-free language; it does not depend on a grammar formalism or automata. That is because they are variants, *methods* to realize the invariant (string admissibility). It basically says that sufficient length depends on the language, and every context-free language has it. It is another way of saying that such strings force description of them to be **cyclic**, (3d), hence the word ‘pump’, and context-freeness means that if we are able to reach this point once (recall that $z \in L$; (3a)), we can reach it unconstrained and still remain in the language.

What is special about this cycle is that it has one **center** (w); it **controls** its left (v) and right (x). A better term for w would be *head*, which emphasizes more that it is not the string position that makes it the center, it is the property that it does not repeat. Notice that (3b) admits centers with null left contexts and null right contexts but not both because otherwise the context-free cycle in (3d) would be meaningless. Unlike regular languages that we shall study subsequently, they can be contexted by a nonempty string on *either* or *both* sides (3b). *Even then* a context-free string has no control over how many times a cycle can be repeated, that is, we have $i \in \mathbb{N}$ condition in (3d) and no control over the identical embedding of left and right contexts, respectively v and x . That is what “free” in “context-free” means. It means that the center/head is locally free to choose its left and right but not nonlocally able to do so.

In other words, a context-free hierarchy stands on its own no matter how deeply embedded a context-free string is and how it is surrounded; it can only

be surrounded by whatever the center/head that forced the pump (w) can contextualize, that is, with whatever it can provide as context *locally*.

In a regular string we lose the control in identical embedding as well. We cannot *contextualize* a regular string that way. That is the *only* difference between context-free language strings and regular language strings.

Going in the other direction, that is, adding more surround-control abilities to the center/head, gives more strongly structured hierarchies, for example the **linear-indexed languages** of Figure 3.1, for example Swiss German (2). The paragraph after that example describes the strength of control that we need. The current conjecture is that that is all the structural control that we need to understand the hierarchies and their meanings arising from strings of any human language (Stanojević & Steedman 2021).

If this control sounds like semantics going along with syntax, that is, if the v^iwx^i restriction feels like a **dependency** mediated by the center/head, then we are moving in the right direction in this jungle of formal symbols for understanding the conceptual basis of string admissibility in general and what it means for semantics.

3.2.2 Regular admissibility

All strings in any regular language satisfy the following properties. They define what it means to be **structurally iterating** to the extent of being regular. I prefer to call them iterating rather than embedding for reasons that we shall come to. The difference is significant for morphology and phonology.

- (4) Let L be any **regular language**. (Pumping lemma for regularity)
 - a. Then, if there are sufficiently long strings z in L , say $|z| \geq n$, for some n depending only on L , then we can decompose it as $z = uvw \in L$ such that
 - b. $|v| \geq 1$,
 - c. $|uv| \leq n$, and
 - d. for all $i \geq 0$, $uv^i w$ is also in L .
 - e. If there are no sufficiently long strings in L , then L is a finite set (language), same as (3). This family, which is not shown in Figure 3.1, is strictly within the family of regular languages. There is no pumping lemma for them.

First, notice that **cyclicity** of regularity (4d) requires a shorter context to understand it compared to context-freeness (3d). Also note the i restriction in (4d); it is

3 Degrees of structure from string admissibility

not quite a dependency like (3d) because v of (4d) does not depend on anything in cycling, whereas things that cycle in (3d), v and x , both depend on i .

This mathematical difference arises from the conceptual difference of distinguishing such strings from embedding strings: iteration is not embedding because it cannot create surrounding dependencies around a center/head. For this reason, (4d) is defined in many sources as uv^*w indicating no control.

The notation of uvw may indicate a bias toward center/head-final strings because of the position of w , but we could have wuv^* and $uwnv^*$ just as well. It is not the string position of the center/head that counts; it is the fact that it has no control over iterating on v . (The u is there for making the string sufficiently long mathematically, that is, to force the string to iterate; it is not part of the iteration itself.)

Any string of any regular language can afford this much control, which is none. This is something to think about when we talk about semantically motivated morphological structures in §6.4, a precursor of which we will discuss at the end of the next section. We need that section to see what is at stake in admitting more or less structured strings.

3.2.3 Linear admissibility

It is time to reflect on the admissibility of context-free strings and admissibility of regular strings to understand the relation between the amount of form control in string admissibility and the amount of structure in a string.

By now it must be clear that these formal properties are not formal because they are solely about form; they are formal because they rigorously focus on amount of structured meaning we can afford in admitted strings.

The following is the **pumping lemma for linear languages**, from Hopcroft & Ullman (1979).

- (5) Let L be any **linear language**. (Pumping lemma for linearity)
- a. Then, if there are sufficiently long strings z in L , say $|z| \geq n$, for some n depending only on L , then we can decompose it as $z = uvwxy \in L$ such that
 - b. $|vx| \geq 1$,
 - c. $|uvxy| \leq n$, and
 - d. for all $i \geq 0$, uv^iwx^iy is also in L .
 - e. If there are no sufficiently long strings in L , then L is a finite set (language), same as (3) and (4). This family, which is not shown in Fig-

ure 3.1, is strictly within the family of regular languages. There is no pumping lemma for them.

3.3 String admissibility: nonreified mathematics with implications for order and structure

Notice the subtle but very important difference of admissibility in (5c) compared to (3c). All other conditions are the same in (5) and (3). Notice also the context-free like dependency of the cycle in (5d), unlike regular and iterating dependency in (4d). In a closer look, we can see that (3d) and (5d) are **dependencies** precisely because two sides of the center w are conditioned on something (by i), whereas (4d) is not really a dependency but **iteration**. That seems to be the main difference between **embedding** and **nonembedding**. The amount of **meaningfulness** (not **sensefulness**) we can muster with these properties is *consequently* different. We do not need any reification to see that.

These conditions alone can give an appreciation of why the class of linear languages is a superset of regular languages and subset of context-free languages in Figure 3.1, without needing lengthy proofs. Regular languages admit less structure (not less strings) than linear languages because their iteration (4d) is more liberal than the embedding of (5d). Looking in the other direction for more (and less) structure, we can see that context-free languages admit more structure (not more strings) than linear and regular languages because their dependency (3d) is less liberal than that of (4d) and (5d). There are theorems that formally prove these results; see for example Hopcroft & Ullman (1979). These differences may appear to be merely formal mathematical results, but they are not distinct abstractions for the sake of abstractions; they capture the complex relation between **embedding**, **iteration**, **dependency** and **locality** very succinctly.

To give an example, the set (formal language) A defined as $A = \{a^n b^n \mid n \in \mathbb{N}\}$ is linear and not regular because it needs left and right contexts for its strings of a 's and b 's to ensure the same number of a 's and b 's in the right order. It is also context-free for this reason. But, not all context-free dependencies are linear because linearity admits less structure (not less strings) because of its more liberal cycle. For example, although it appears to manifest more of the same dependency as the linear and context-free A , the following language B is context-free but not linear: $B = \{a^n b^n c^m d^m \mid n, m \geq 1\}$.

Interestingly enough, something that looks similar to nonlinear B just introduced, for example $C = \{a^n b^m c^m d^n \mid n, m \geq 1\}$, is linear, which is something to think about in deliberating over embedding and iteration.

3 Degrees of structure from string admissibility

It is about how many dependencies we can control as well as what kind of dependencies we can control that we must think about in understanding the structure of a set of strings. These are properties that are independent of description and computation: *Any* variant and its mathematical method ought to accommodate these properties so that we do not have to worry about them further, about where they come from.

More philosophically, it is about **control** by order and structure in strings, not order *or* structure, not order from structure, not structure from order, but order *and* structure. There are no separate admissibility conditions for them. We have seen this throughout this chapter, in (5), in (4), and in (3).

What we control by making up such strings appears to be meaningfulness through grammaticality when we harness the order and structure in a string. Why do we need this control, not able to live without it? What do degrees of control bring to us? When did we get it? Where is the power of human language strings coming from? These questions put linguistic theories in touch with overarching properties of complex human behavior. We must keep in mind that they build on one admissibility criterion.

For example, is the mathematical difference between embedding and iteration significant for linguistics? We look at this problem with some real data in §6.4. For the time being, keep in mind that bound morphemes do not have morphemes in them (iteratively) unless they become inseparable like **portman-teau**, and phonemes, phones or syllables do not embed (but decompose) although phonological, intonational and prosodic phrases can. These are empirical consequences of the mathematical intuition that they do not embed *like* syntax or semantics.

The next natural question to ask is whether **words** can embed like syntax. This is an open problem, which is probably not surprising because there is more to being a word than what is available in its manifestation—it is itself an abstraction; see for example Aronoff & Fudeman (2011) for a start on thinking about this problem. Haspelmath (2011) points out that there is no crosslinguistically applicable empirical criteria for wordhood. It is best if we know them all.

Keep the mathematical distinction we have made between recursion (embedding) and iteration in mind while thinking about that abstraction we call **word**. Keep also in mind the idea of reference in semantics (§2.2), by which syntax is transparent transmission of reference as we build phrases whereas morphology would not have to be that transparent, requiring, in some languages, **correlational reference** like pluralizing things that can be pluralized, marking possession on things that can be owned, marking point-of-view on things that can afford event viewing from different perspectives, etc. Futrell & Hahn (2025) report

striking asymmetries in referential structure of various languages from a probabilistic perspective (transparent versus correlational reference, syntax versus morphology, morphology versus lexicon).

In the next two chapters we shall see how **admissible** structures have been **described** and **analyzed** most commonly in the field. Are these the best tools? We do not know that but this is the current state of the art, building a *lingua franca* for talking about admissibility in linguistics. The good news is that, whatever descriptive and analytical (currently, computational) vocabulary we develop will presumably appeal to same admissibility: Observe that the conditions for **structured strings** in this chapter are not based on descriptive or computational vocabulary. **Phrase-structure grammars** are the most common **variant** vocabulary now, grammars of **feature geometry** emanating from SPE wanting an understanding of invariance as one point of departure from them, **categorical grammars** and **dependency grammars** becoming also quite common.

We end this chapter with a summary of progress of ideas about strings, grammars, and structure, in Figure 3.2.

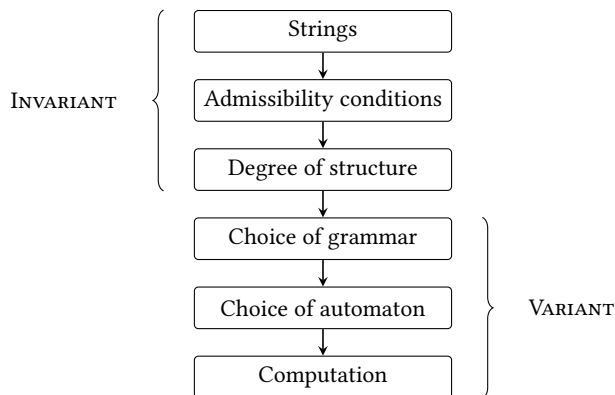


Figure 3.2: From string admissibility to computation. Admissibility and degree of structure constrain the choice of representational and computational methods.

4 Formal grammars: describing admissible strings

Given the admissibility conditions of strings in §3.2, it is good to know that the field has been able to develop as a joint effort of linguists, mathematicians and computer scientists string descriptions that allow all and only admitted strings to be studied methodically. Since admissibility (invariant) is concerned with structure rather than content, the notation for the variants, which is known as the **string rewrite system**, is about our ability to describe such structures borne by strings, all of them and only them. It is quite a remarkable achievement that admissible strings of the Chomsky hierarchy (Figure 3.1) can all be described by one rewrite system with varying conditions, rewriting the string α as β , denoted $\alpha \rightarrow \beta$. All conditions boil down to some constraints on α , β or both. There are other methods, but this one has become the lingua franca of structural description. It is the one we cover in the book.

In the next subsections we do that, starting first with context-free language description (§4.1), then regular language description (§4.3), with a brief excursion in between into Swiss German-like data (§4.2). We also look at ways to put the descriptions in more desirable shapes—desirable according to some empirical criteria, respectively in §4.5 and §4.6. After that, in Chapter 5 we look at other methods that follow from admissibility and string rewrite to understand the limits of computation with admissible and describable strings (i.e. with variants), and to prepare them for more efficient computation.

It is remarkable that once we have the notion of **formal grammar**, admissibility in relation to degrees of structure can be seen from a mathematically simpler descriptive perspective which is a bit more difficult to decipher from pumping lemmas of §3.2. Mathematically, a formal grammar is defined as follows.

- (1) Any formal grammar G to describe a language in Figure 3.1 can be defined as a four-tuple, quadruple $G = (V, \Sigma, R, S)$ where
 - a. V is a finite set of grammar placeholders called **nonterminals**.¹

¹Sometimes they are called **grammar variables**—kindly note the standard symbol V used for

4 Formal grammars: describing admissible strings

- b. Σ is a finite set of grammar placeholders, more commonly known as **terminals**, constituting the **alphabet** of G .
- c. R is a finite set of **rules**, also called **productions**. They are all in the form of the **rewrite notation** $\alpha \rightarrow \beta$, which means α rewrites as β (note the direction of the arrow); α and β can be anything we can put together in the form of a string of terminals and nonterminals, that is, $\alpha, \beta \in (V \cup \Sigma)^*$.
- d. S is the start symbol of G . $S \in V$.
- e. Without losing generality, the sets V , R and Σ are considered nonempty and disjoint.

Notice the qualification ‘finite set’ in all the components. A formal grammar may be large but not arbitrarily large, given our current understanding of what these concepts mean. A grammar itself is a finite specification although its language may be arbitrarily large. That is a precondition for us to be able to put our grammars or our theories about them on the table, to represent, for anyone to assess, equally importantly, for anyone to model our proposal.

Now admissibility is manifested very simply as describing degrees of context for any structure in a rewrite system *alone* (i.e. as condition on R only), as follows:

- (2) Let $\alpha \rightarrow \beta$ be any rule of rewrite in grammar $G = (V, \Sigma, R, S)$.
Let $\mathcal{L}(G)$ denote the language of grammar G . It is the set of strings admitted by G . We will define admission by grammar derivations that connect S to terminal strings when we look at different kinds of grammars.
If, for all $\alpha \rightarrow \beta$ rules in R , we have
 - a. $\alpha \in V$, and β is either from one of the sets Σ^*V , $V\Sigma^*$, or from Σ^* , that is, if there is at most one nonterminal on the righthand side, always on the same side, iterating always in the same manner—hence the term **regular**, then $\mathcal{L}(G)$ is regular.² It belongs to the innermost solid circle in Figure 3.1, that is, to the shallowest embedding hierarchy,
 - b. $\alpha \in V$, and β is from the set $(\Sigma \cup V)^*$, that is, if the righthand side can be anything provided the lefthand

them, standing for “variable”—but this term is a bit unfortunate. They are algebraic placeholders; they do not vary over time or space. Even those who use this terminology would not think of differentiating or integrating them over time or space. My advice: When it comes to naming conventions, look at what they do, not their name. Ryle would be pleased; recall (4) from Chapter 2.

²More specifically, R (therefore G) is **right-regular** if all β are of the shape Σ^*V or Σ^* , **left-regular** if all β are of the shape $V\Sigma^*$ or Σ^* . If R is left- or right-regular, then $\mathcal{L}(G)$ is regular.

side is a single nonterminal—hence the term **context-free**, then $\mathcal{L}(G)$ is context-free. It belongs to the second innermost solid circle in Figure 3.1, that is, to arbitrarily hierarchical embedding of structure,

- c. α, β are from the set $(\Sigma \cup V)^*$ such that $0 < |\alpha| \leq |\beta|$,
that is, if the righthand side can be anything provided the lefthand side is no longer than it, then $\mathcal{L}(G)$ is context-sensitive. It belongs to the third innermost solid circle in Figure 3.1, that is, to arbitrarily hierarchical embedding of structure controlled by limited contexts on its left and right, hence the term **context-sensitive**,
- d. α, β are from the set $(\Sigma \cup V)^*$ such that $|\alpha| > 0$,
that is, if the righthand side can be anything provided the lefthand side exists—hence the term **unrestricted**, then $\mathcal{L}(G)$ is recursively enumerable. It belongs to the outmost solid circle in Figure 3.1, that is, to arbitrarily hierarchical embedding of structure controlled by unlimited contexts on its left and right.

For example, the following (a) is a regular grammar; (b) is a context-free grammar; (c) is a context-sensitive grammar, and (d) is an unrestricted grammar.³

- (3) Let $G = (V, \Sigma, R, S)$. The ‘|’ shorthand means alternative righthand sides for the same lefthand side. By convention uppercase letters are nonterminals and lowercase letters are terminals. For example for (a) we have the grammar specification $G = (V = \{S, A\}, \Sigma = \{a, b\}, R, S)$, where R is the set of rules in (a). Recall that a set has no ordering of its elements; R is not ordered, hence the singling out of S as the start symbol.

- a. $R : \begin{array}{l} S \rightarrow aA \mid bS \mid \epsilon \\ A \rightarrow aS \mid bA \end{array} \quad \mathcal{L}(G) = \text{The set of all and only strings in } \{a, b\}^* \text{ with even number of } a\text{'s.}$
- b. $R : \begin{array}{l} S \rightarrow aSb \mid ab \end{array} \quad \mathcal{L}(G) = \{a^n b^n \mid n > 0\}$
- c. $R : \begin{array}{l} S \rightarrow aAbc \mid abc \\ A \rightarrow aAbc \mid abC \\ Cb \rightarrow bC \\ Cc \rightarrow cc \end{array} \quad \mathcal{L}(G) = \{a^n b^n c^n \mid n > 0\}$

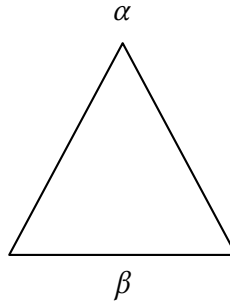
³Soundness and completeness claims for the descriptions (i.e. their adequacy) actually need proof but we skip them; see for example [Sudkamp \(2006\)](#). Take (3b). To show that $\mathcal{L}(G) = \{a^n b^n \mid n > 0\}$, we have to show the equivalence of two arbitrarily large sets. The standard practice is to show that their properties imply each other, that they form subsets of each other, that is, $\mathcal{L}(G) \subseteq \{a^n b^n \mid n > 0\}$ and $\mathcal{L}(G) \supseteq \{a^n b^n \mid n > 0\}$, therefore $\mathcal{L}(G) = \{a^n b^n \mid n > 0\}$, using G as formal capture of (a collection of) properties.

4 Formal grammars: describing admissible strings

$$\begin{array}{llll}
 \text{d. } R : & S & \rightarrow & ACaB \quad aD \rightarrow Da \\
 & Ca & \rightarrow & aaC \quad AD \rightarrow AC \\
 & CB & \rightarrow & DB \quad aE \rightarrow Ea \\
 & CB & \rightarrow & E \quad AE \rightarrow \epsilon
 \end{array}
 \quad \mathcal{L}(G) = \{a^{2^n} \mid n > 0\}^4$$

One thing you will notice from these examples is the distinguishing property of human language strings: All human language expressions appear to have the **constant growth property** (Joshi et al. 1991), by which, if all strings of a human language were put in the increasing order of length, then two consecutive lengths would not differ by an arbitrary amount; in fact they would differ by a constant amount, most likely, 1. We know of no human language in which there are n -word sentences but not $n+1$ -word sentences. This property is maintained not just for 1 but for some constant c up to strictly linear-indexed languages, somewhere between (3b) and (3c) according to Figure 3.1.⁵ Notice the unrestricted grammar in (3d), which is the language in which the number of a 's is always a power of 2. The strings have nonconstant growth.

We can summarize degrees of admissibility as varying conditions on the following configuration where the bottom β is the string leading to structure characterized by α , from simple iteration, (a) above, to context-free embedding, (b), then context-bound embedding, (c), then to no restriction, (d). We can visualize the constraints on R :



Think of this image as the interpretation of the rewrite $\alpha \rightarrow \beta$ in (2). Notice that the triangle stretches (the α bloats), turning α from a summit to a roof as we move up the Chomsky hierarchy (Figure 3.1), going from (2a) to (2d), in the end admitting extremely rigidly constrained structures, like a rectangle, that is, looking less and less hierarchical and more grid-like as we move upward in Figure 3.1. Think of being grid-like as having more and more control on trees *inside*

⁴From Hopcroft & Ullman (1979).

⁵A language is strictly in class A if it is in A but not in a subset of A. For example, (3b) is **strictly context-free** because it is context-free but not regular. That is the essence of drawing Figure 3.1 like that.

β by α ; for example some rules in (3c) and (3d). I appeal to visual analogies to avoid endless pages of proofs that that is what is happening as we go from (2a) to (2d).

Recall from (2) that the rewrite notation $\alpha \rightarrow \beta$, read “ α rewrites as β ” and “ β rewrites as α ” is the descriptive tool for making explicit the structure of an admitted string, any string. It is derived from admissibility, in the case of context-free strings, from §3.2.1 (3). You can think of **rewrite** as a collection of methods to textually build or analyze trees such as the one before.

The idea of **derivation** makes the rewrite process explicit. The term “rewrite” may suggest that only things which can be substituted for can be rewritten, that is, the nonterminals. The concept of rewrite is general enough that any string of terminals and nonterminals can be rewritten:

(4) Let $G=(V, \Sigma, R, S)$ be any grammar.

- a. Then $u \xRightarrow{R} v$ means u **derives** v using the rules in R only, where $u, v \in (V \cup \Sigma)^*$.

If the rule used in R is specific, we write that rule. If what matters is zero or more use of rules but not the rules themselves, we can write $u \xRightarrow{*} v$ to mean u derives v with zero or more rule application, turning the **Kleene star** operator ‘ $*$ ’ to a meta tool.

- b. Define $\mathcal{L}(G)=\{w \in \Sigma^* \mid S \xRightarrow{*} w\}$.

It is the **language of** G described with the **derivability relation**.

It is the set of terminal strings w which are in derivability relation with the start symbol S .

Notice that derivability is a relation, not a function. The same string can be derived from many strings, and one string can lead to many strings. What is substituted in u, v during rewrite depends on the rule, and even when the rule to be applied is definite, the range and domain it covers may be more than one.

This is another way of saying that we are in fact talking about at least two methods rather than one for the derivability relation: (i) Going from u to v in (4a), which forms the basis of **top-down parsing**, and (ii) going from v to u , which forms the basis of **bottom-up parsing**. (They are covered in Chapter 5.) We can see from the definition of $\mathcal{L}(G)$ in (4b) why they are called such: If we start with S on the left of ‘ $\Rightarrow$ ’ it is the top symbol of grammar. If we start with the terminal string w , we start at the bottom and reach S . We will see examples in the next section.

Before we move on, one warning about **category mistakes** that Ryle warns us against in connection with languages and descriptions. A formal grammar is not an algorithm that does things; it is a description of what is done or to be done. What we *do* on strings is done by automata, the topic of the next chapter. You can rest assure that once they are tied to one admissibility, description and computation are two sides of the same coin.⁶ Otherwise their nonarbitrary association in the methods would have to be stipulated.

I mention this because in the literature you will often see terms such as **parsing grammar** (a category mistake, if not loose talk) and **parsing algorithm** quite often (a meaningful combination). Parsing is computation; it works within the ontology of automata, not descriptive grammars. That is one reason why we will define top-down and bottom-up parsing algorithms as automata, rather than ‘ $\Rightarrow$ ’, which is a description of what is done by automata. The distinction is important because if something goes wrong in a formal linguistic analysis, say considering an expression ungrammatical when it is not, or grammatical when it is not, we would know what to blame rather than push all the buttons. It gives a collective understanding of the relation between admissibility, description and computation.

So much for formal clarification. Empirically, having to connect to a start symbol to be in a language is something to think about regarding meaningful hierarchies. It is common in the field to associate meaningfulness which is consequent to grammaticality as arising from a unique interpretation, arising from a hierarchy. Context-free grammars are natural points to start pondering about such structures. Although they are necessary for understanding uniqueness of a structure, they are not sufficient to understand all human structures, as Swiss German data has shown (2). Going from context-freeness to linear indexing is necessary, and quite revealing when studied with a formal grammar for linear-indexed grammars, which we do after the next section.

⁶For formal proofs of the results that for every regular grammar there is a corresponding computation preserving its language (called **finite automaton**), and vice versa—for every regular computation there is a regular grammar, for every context-free grammar there is a corresponding computation preserving its language (called **pushdown automaton**), and vice versa, in fact for every level of admissible strings in **Chomsky hierarchy** of Figure 3.1 (Chapter 3) there is a similar correspondence of description and computation, see [Hopcroft & Ullman \(1979\)](#), [Sudkamp \(2006\)](#).

4.1 Context-free grammars

What makes context-free strings context-free is that at this level of structural description their rewrite needs no left or right context for any rule. Although the righthand side of a rule can be quite complex, the lefthand side is always one nonterminal. Although the general concept of **derivability** (4a) is general enough to rewrite anything, if all rules have one nonterminal on the left and with these rules we try to connect a string to the start symbol to see if it is in the language (4b), then we only rewrite one nonterminal for context-free strings. There is no context which is conditioning the admissibility of the lefthand side or righthand side. Naturally, when it comes to method selection, the *use* of a context-free rule will have these conditions built in.

4.1.1 Definitions

Let $G=(V, \Sigma, R, S)$ be a **context-free grammar (CFG)**. Then all rules in R are of the form $\alpha \rightarrow \beta$ where $\alpha \in V$, $\beta \in (V \cup \Sigma)^*$. No more needs to be said about the conditions on this grammar to assume its *language* to be context-free because we are now describing admitted strings.

A context-free language is a set of terminal strings in which every admitted string can be described as such, being in the language of grammars like G . We have seen that there is a convenient method to make this process explicit. (Recall that we are now in variant territory, so everything will be related to a method.)

- (5) Let $G=(V, \Sigma, R, S)$ be a grammar.
 - a. If G is a context-free grammar, then $\mathcal{L}(G)$ is a context-free language. This is the common reading. When we are interested in its connection to admissibility, we say that G must be a context-free description if $\mathcal{L}(G)$ is a context-free language satisfying context-free admissibility. The wording change may sound like nitpicking; it is more about mathematical thinking of what is a consequence of what, including methods.
 - b. $\mathcal{L}(G)=\{w \in \Sigma^* \mid S \xRightarrow{*} w\}$.

4.1.2 Examples

It suffices for context-free languages to only rewrite nonterminals given their conditions on rule shapes. It does sound like an arbitrary restriction but it is actually one convenient way to realize admissibility conditions of §3.2.1 (3).

Consider the following example of a context-free grammar:

4 Formal grammars: describing admissible strings

- (6) a. $G : \begin{array}{l} S \rightarrow AB \\ A \rightarrow aAb \mid ab \\ B \rightarrow cBd \mid cd \end{array}$
- b. Formal specification of G in the quadruple format of (1) and (3):
 $G = (\{S, A, B\}, \{a, b, c, d\}, \{S \rightarrow AB, A \rightarrow aAb, A \rightarrow ab, B \rightarrow cBd, B \rightarrow cd\}, S).$
- c. $\mathcal{L}(G) = \{a^n b^n c^m d^m \mid n, m \geq 1\}.$

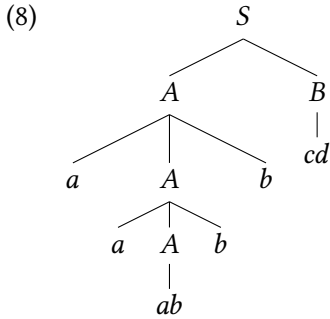
(6a) is the standard notation for grammar description. For some people it is not fully formal. You can point them to (6b). Its language is described in (6c) without proof. (For such proofs take a look at footnote 3).

This is not the only grammar for this language, but its logic is simple enough to foresee that it is one of the correct ones for the language, at least in the weak sense that it does describe all and only the strings in the language. It decomposes a/b dependencies and c/d dependencies since they do not depend on each other (n and m can be different). A describes the first dependency and B the second dependency. Since the number of A s and the number of B s can be different, the number of a/b 's and the number of c/d 's can be different. Within themselves they preserve order, since no rule has b preceding a or d preceding c . Neither n nor m can be zero because every A and B describes a nonempty string, therefore so does S .

From the following derivations (a–b) we can see *how* these strings in $\mathcal{L}(G)$ can be described to belong to it. In (c–e) we can see that these strings are provably ungrammatical given G ; they cannot be connected to S .

- (7) a. $S \xRightarrow{S \rightarrow AB} AB \xRightarrow{A \rightarrow aAb} aAbB \xRightarrow{A \rightarrow aAb} aaAbB \xRightarrow{A \rightarrow ab} aaabbbB \xRightarrow{B \rightarrow cd} aaabbbcd.$
- b. $S \xRightarrow{S \rightarrow AB} AB \xRightarrow{A \rightarrow aAb} aAbB \xRightarrow{A \rightarrow ab} aabbB \xRightarrow{B \rightarrow cBd} aabbcBd \xRightarrow{B \rightarrow cd} aabbccdd.$
- c. $S \xRightarrow{?} A \xRightarrow{A \rightarrow ab} ab.$
- d. $S \xRightarrow{?} BA \xRightarrow{B \rightarrow cd} cdA \xRightarrow{A \rightarrow ab} cdab.$
- e. $S \xRightarrow{?} badc.$

We can get a sense of connectedness in grammatical examples by drawing the derivation connecting a start symbol to a string as a tree in which every local branch is formed by a rule, for example for (7a):



If all leaves are terminal when the top symbol is S in a connected (i.e. admissible) tree, then we have (4b) satisfied to be in the language. Notice that we cannot do that for (c–e). The parsing methods that are discussed in the next chapter are methods for traversing or constructing such trees; that is, they compute, not describe.

Consider the following example of a context-free grammar. This time we spare you the humdrum of full formal specification of grammar when it can be inferred easily from (9a).⁷

$$(9) \quad \begin{array}{ll} \text{a. } G : & S \rightarrow aB \mid bA \mid \epsilon \\ & A \rightarrow aS \mid bAA \\ & B \rightarrow bS \mid aBB \end{array}$$

$$\text{b. } \mathcal{L}(G) = \{w \in \{a, b\}^* \mid \text{there are same number of } a\text{'s and } b\text{'s in } w\}.$$

The language of this grammar may be more difficult to see, let alone its context-freeness.⁸ It is based on a simple logic. A keeps track of the number of a 's, and B keeps track of the number of b 's. If we encounter a b when things are equal (i.e. if we are at S) we call on A to keep track of a 's. If we encounter an a when things are equal we call on B to keep track of b 's. If a -tracking encounters another b then we would need not one but two A 's since we have come to this rule thinking we have to equalize a 's having seen one b . (Look at the connectedness of A and S .) If we find the a in a -tracking, then *one* A has been taken care of, the others if any

⁷Once you get the idea it is kind of boring. Until then though, practice makes intentions clear in $G=(V, \Sigma, R, S)$. For example, is ϵ in the righthand side of one rule in Σ ? Is $S \rightarrow \epsilon$ a rule in R ?

⁸Proving that the language in (9b) is strictly context-free may look equally tedious; but, it is made easier by using **closure properties** of family of languages. For example, context-free languages are closed under intersection with regular languages. That is, intersecting a context-free language with a regular language is always context-free. If we intersect $\mathcal{L}(G)$ of (9b) with the regular language $\{a, b\}^*$, the result is $\{a^n b^n \mid n \geq 0\}$. This language is much easier to prove to be context-free, by being nonregular and by having a much simpler context-free grammar to prove as its grammar.

taking care of themselves, so we can go back and start tracking all over. Similar logic applies to B and b -tracking.

Consider the following example of a context-free grammar.

- (10) a. $G : \begin{array}{l} S \rightarrow AB \\ A \rightarrow aA \mid \epsilon \\ B \rightarrow bB \mid \epsilon \end{array}$
 b. $\mathcal{L}(G) = \{a\}^*\{b\}^*$.

Its language is context-free, also, regular. In other words, a regular language is described context-freely. It shows that the **Chomsky hierarchy** of Figure 3.1 is about admissibility, not description. We can write a much restricted description for it, which we do in §4.3. We cannot do that for (9b) because it is strictly context-free: there is no regular grammar for it. These are points to keep in mind when we connect with the big picture—putting things together, about what kind of semantic dependencies can we associate with syntactic dependencies. This will affect our choice of methods in Chapter 6.

We close this section with an example to make one structure explicit in description. In the very beginning of Chapter 3 we mentioned the problem of matching parentheses. Recall that it is the structure where every opening parenthesis must close, and close only after nested open parentheses close. It does not say that all parentheses must be nested; they may be iterated. We can write the following grammar for it.

- (11) $G : S \rightarrow (S)S \mid \epsilon$

That is one impressively compressed description. Nesting is taken care of S inside the parentheses. Iteration is taken care of the S outside the parentheses, all in one righthand side, that is, one rule; see if you can derive $()()()$ with it.

Notice how deceptively simple a description can be. The dependencies in this language are not regular. They are context-free. Our choice of automata to compute such strings (and their meanings, in the big picture) must keep that in mind.

4.2 An excursion to linear-indexed grammars

The idea of **indexed languages** posits that there seems to be a formally identifiable class of dependency structures in languages that we can call **indexed**. The mathematics of that was worked out by Aho (1968), whose motivation was to identify a subclass of dependencies within the context-sensitive languages of

Figure 3.1. Their empirical import to linguistics started with [Gazdar \(1988\)](#), most notably with an even more restricted use of the idea of index.

So what is the idea of an **index**? It is the idea of keeping track of dependencies in the grammatical description in a way that transcends what strings themselves can do when they appear in **derivations** such as (4a). If such derivations involve a mixture of terminals *and* nonterminals, what properties of the nonterminal can be explored to give us a better understanding?⁹ That is the idea.

For example, the context-free dependency in the rule $S \rightarrow (S)S$ of (11) does not distinguish whether the S 's on the righthand side are nesting or iterating. Using an index we can for example keep track of iterations, for example $S[i..] \rightarrow (S)S[.. i]$ where $[]$ is the index. We shall see what they can be.

Notice that they leverage understanding of more meanings in meaning-bearing elements, and that through form. Think of this as an attempt to come up with a better description (and hopefully, computation) of strings whose admissibility can be narrowed down further compared to the class of context-sensitive languages of Figure 3.1. Is this relevant to understanding natural languages? Recall the Swiss German data and its discussion around (2) of Chapter 3, repeated here for convenience:

- (12) Jan säit das mer d'chind em Hans es Huus lönd hälfe
 Jan says that we₁ children-ACC₂ Hans-DAT₃ house-ACC let₁ help₂
 aastriiche Swiss German
 paint₃
 'Jan says that we let the children help Hans paint the house.'

To follow the semantics given in the gloss, we mark who is doing what to whom (i.e. **dependencies**) in syntactic structure using indices, giving the dependency sequence $s_1 s_2 s_3 v_1 v_2 v_3$ where s is 'subject' and v is 'verb', and s_1 means it is the subject of v_1 etc. The important point of the proof by [Shieber \(1985\)](#) is that Swiss German speakers can do this indefinitely. Such dependencies are not context-free, and apparently, we do not have to move up to context-sensitive languages of Figure 3.1 to admit more strings than necessary for natural language dependencies such as the ones above.

This is the idea of **linear-indexed languages**. It is important for understanding the mathematical reality of dependencies that we can capture (i.e. the invariant), consequently, for the choice of methods for model building and selection (i.e. the variants).

⁹As usual, this exploration comes with some philosophical questions attached to it. For example, what is the nature of *structure* we now have access to? Is it semantic or syntactic?

4.2.1 Definitions

The formal apparatus for linear-indexed languages has been more or less the following since their introduction by [Hopcroft & Ullman \(1979\)](#).

- (13) Let $G=(V, \Sigma, R, S)$ be a formal grammar, as before.
- The nonterminals of G are names plus one of the following index associations: (i) $A[]$, meaning $A \in V$ with empty index, (ii) $A[i \dots]$, meaning A with top index i , (iii) $A[\dots]$, meaning A with some index $\dots$, and (iv) A , meaning no index. Indices can be anything, numbers, names, etc.; their use in brackets means ‘index’.
 - Rules in R are context-free rewrites with the understanding of V in (a). There can be three kinds of rules in R : (i) $A[i \dots] \rightarrow \beta[\dots]$, in which the index is same between A and β except the top i , $\beta[\dots]$ being a shorthand for saying that the index is on *one* of the nonterminals in β , (ii) $A[\dots] \rightarrow \beta[i \dots]$, meaning the same as (i) but index coming from the righthand side, (iii) $A[] \rightarrow \beta$, meaning A must have an empty index list to license the righthand side, and (iv) $A \rightarrow \beta$, which is the context-free case for dependencies in A .
 - Conditions on (i–ii) of (b) are the reason they are called *linear*, passing an index around without distributing its contents, without searching the index, just using the top index.
 - The language of such G ’s is a linear-indexed language:
 $\mathcal{L}(G)=\{w \mid S[] \xRightarrow{*} w\}$. Read the other way, if a language is linear-indexed, its strings have a formal description conforming to the conditions above. That is the implicit reading we make when we choose methods, given admissibility. Notice the ‘[]’ condition, that we connect the start symbol to a string with indices exhausted, that is, empty. It is subtly different than the condition for context-free languages (5b).

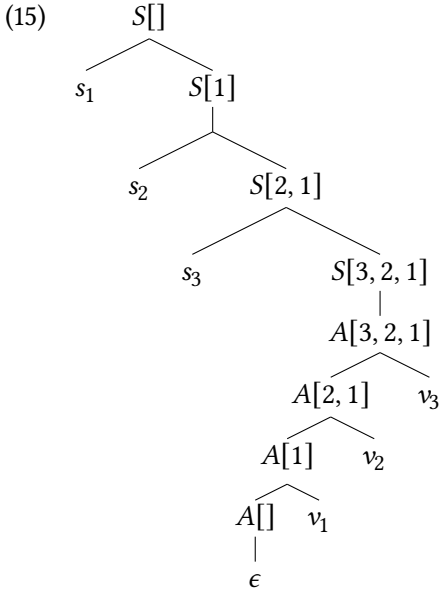
4.2.2 Examples

Let us write the Swiss German dependencies with these definitions in mind. Abstracting the dependencies of who does what to whom in (12) as $s_1 s_2 s_3 v_1 v_2 v_3$, we can write the following linear-indexed description, where the index is keeping track of s and v dependency (recall that s_1 and v_1 means s_1 and v_1 depend, for example ‘we’ and ‘let’ in (12) etc.):

$$\begin{aligned}
 (14) \quad G : \quad S[.] &\rightarrow s_i S[i..] \mid A[.] \\
 A[i..] &\rightarrow A[..] v_i \\
 A[] &\rightarrow \epsilon
 \end{aligned}$$

Read this grammar as follows: The first rule says if we encounter an s_i in the string, we put its index in $S[i..]$. The second rule of S says we pass its index to A . The first rule for A says if there is index i appearing on top, then in the string we must associate the *last* v as having the v_i dependency.¹⁰ The second rule for A says that if there is no index left, we do not expect any more dependency.

The ' $\Rightarrow$ ' notation is deceptively linear—it is actually structural; from a tree diagram it is easier to see that we have the following dependencies unfolding from the description:



From the tree we can see that s_i depends on v_i for any i because this is what the v -rules (i.e. A) *locally* see.

The level of admissibility that allows us to write linear-indexed descriptions does not mean that every such description constitutes a natural dependency, a dependency that we observe in a natural language. It is a necessary structural level not to leave out any natural dependency such as that of Swiss German. What is attestable in the admissible ones needs an **empirical theory** of variants working with a mathematical invariant. For example, the dependencies in $\{a^n b^b c^n d^n \mid n \in \mathbb{N}\}$ seems quite unnatural (it is also not context-free; just like Swiss German dependencies), yet there is a linear-indexed grammar description of its dependencies:

¹⁰Why last? Because the final s we encounter *in the string* is the argument of last v , the penultimate s is for the penultimate v etc.; the subsequent tree will make that explicit.

4 Formal grammars: describing admissible strings

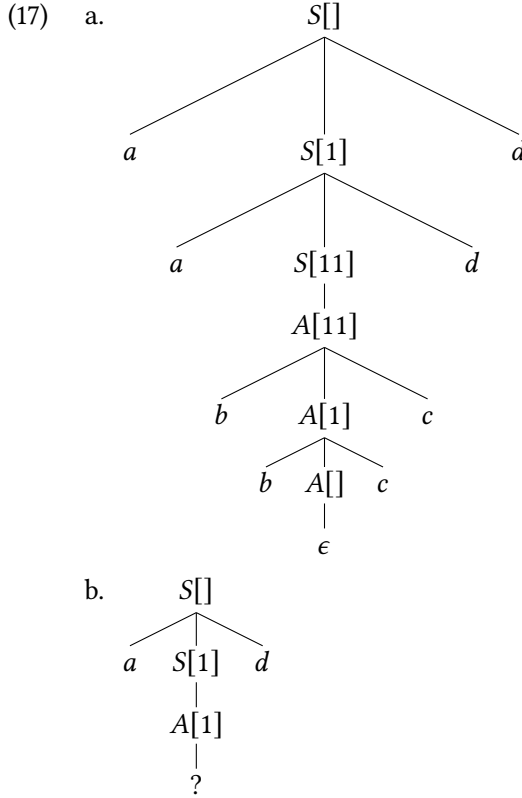
(16) A grammar for $\{a^n b^b c^n d^n \mid n \in \mathbb{N}\}$.

$G : S[.] \rightarrow aS[1..]d \mid A[.]$

$A[1..] \rightarrow bA[.]c$

$A[] \rightarrow \epsilon$

For example for the string *aabbccdd* we have the following derivation in (a) (also note that for example *ad* is not derivable as shown in (b), given the language definition of linear-indexed strings):

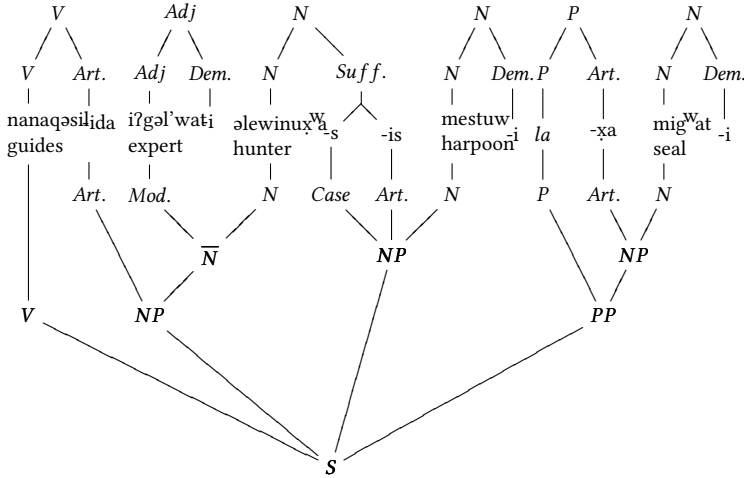


Admissibility puts the following Montagovian view of natural languages in a different perspective: “There is in my opinion no important theoretical difference between natural languages and the artificial languages of logicians.” (Montague 1970:222). Every admissible artificial language of logicians constitutes a formal language. Not every formally admissible stringset constitutes a natural language. That is what **empirical theories** are for, to explain gaps in the data and to do **model selection**, not just **model verification** and construction as he had conceived of modeling to be (see Figure 1.1 for the modeling cycle.).

4.3 Regular expressions (grammars)

The linguistic curse on regular grammars is that they are first and foremost associated with one aspect of **morphology**, the word structure, although they are not enough for that and they are useful in many other domains of linguistics.

For the first point consider the structures in the Kwakw'ala clause in Figure 4.1.



‘An expert hunter guides the seal with his harpoon.’

Figure 4.1: Organization of a Kwakw'ala sentence, from [Anderson \(1992\)](#); Fig. 1.5.

The tree on top is the morphological structure according to Anderson. The tree at the bottom is the syntactic structure of the clause. You will notice that bound elements such as affixes, articles, prepositions, demonstratives are marked not on the word that bears a semantic relation with them but on the previous word. For example the demonstrative that sits on the word for ‘expert’ is actually the demonstrative of the next word, ‘hunter’. Similarly, the case affix and the article on ‘hunter’ are for ‘harpoon’. Kwakw'ala is strictly verb-initial so that any nominal can be safely regarded as having a previous word. It so happens to be the phonology of the language that in such morphology the phonological host is the previous word. The phonology-morphology distinction crossing the formal grammar lines has been pointed by other researchers, for example [Culy \(1985\)](#).¹¹

¹¹The debate continues. It has been pointed out for example by [Manaster-Ramer \(1988\)](#) that some

If we were to use regular expressions exclusively for understanding word structure, then for Kwakw'ala we would have to extend these expressions to cover in principle syntactic phrases, as the tree at the bottom shows. Doing syntax-semantics with regular expressions for the sake of morphology or doing morphology with transcontext-free grammars (since we know that in principle linear-indexed dependencies are possible) because of phonology, are probably not the formally inevitable choices that are put on the table for linguists to empirically study these structures.

Relatively understudied languages are not alone in showing this behavior, even well-studied languages with relatively simple morphology does things with bound elements wanting another perspective in morphology in relation to semantics such as the possessive over phrases not just words:

- (18) (Every man I know)'s taste in wallpaper is appalling. English; Anderson et al. (2006)

If word morphology were only about word forms without a concern for their meanings, we could do well with regular expressions, but seeing where the morphemes belong demands that we do something about this, which we will take on again as part of the big picture of putting things together and insisting on transparency in doing that, in §6.4.

The implicit use of regular grammars for the orthographic treatment of morphology can be seen in the use of regular operators of **concatenation**, **iteration** and option as the most widely used tools in **language technology** when the system goes over the form for a bit of abstraction before syntax and semantics is added to it. This usually takes place with the convenient notation of **regular expressions**, which are very easy to textualize and streamline in programming of string matching. They are very effective too because their descriptive power is same as **regular grammars**.

A regular grammar is basically a **context-free grammar** in which every rule has at most one nonterminal on the righthand side, and always consistently on one edge, the left edge of the righthand side or the right edge. In Chapter 5 we shall see that this restriction on description translates to very simplified computation eschewing the need for memory bookkeeping, and that by itself is a remarkable methodological contribution.

of the operations that Culy employ for morphology of Bambara are not quite morphological. Senuma & Aizawa (2024) argue from Ainu that operations which are clearly morphological can be more than what regular expressions can do.

So instead of giving a formal definition of regular grammars, in this section we concentrate on the regular expressions.¹²

4.3.1 Definitions

A regular expression is a string-oriented notation for describing the sets in the class of languages that we call **regular languages**, the innermost (and least dependency nesting) class of languages in Figure 3.1. We define it as follows:

- (19) Let Σ be a finite set, the alphabet. The regular expressions (most commonly abbreviated to **regex**) on Σ are given by a **recursive definition** (that word again; see §1.5):
- a. ϵ is a regex.
 - b. $\emptyset$ is a regex.
 - c. a for all $a \in \Sigma$ is a regex.
 - d. If α and β are regexs, then so is $\alpha\beta$, the concatenation of two regexs. The notation $\alpha \circ \beta$ is possible but not very common.
 - e. If α and β are regexs, then so is $\alpha \cup \beta$, the union of two regexs. The notation $\alpha + \beta$ is more common in computer science.
 - f. If α is a regex, then so is α^* , the **Kleene star** of α .
 - g. Nothing else is a regex.

Regexs describe **regular sets**, sets of strings with regular admissibility (§3.2.2). The mapping from the regex to language sets is the following (see (2) for ‘ $\mathcal{L}$ ’):

- (20) Let α be a regex. Then
- a. $\mathcal{L}(\epsilon) = \{\epsilon\}$.
 - b. $\mathcal{L}(\emptyset) = \emptyset$.
 - c. $\mathcal{L}(a) = \{a\}$, for all $a \in \Sigma$.
 - d. $\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha)\mathcal{L}(\beta)$.
 - e. $\mathcal{L}(\alpha \cup \beta) = \mathcal{L}(\alpha) \cup \mathcal{L}(\beta)$.
 - f. $\mathcal{L}(\alpha^*) = \mathcal{L}^*(\alpha)$.

¹²See Hopcroft & Ullman (1979) for the proofs that for every **right-regular** and **left-regular** grammar in the sense defined above, there is a regular expression, and vice versa. In that source and in many earlier textbooks these grammars were called **right-linear** and **left-linear**. Linear grammars are empirically interesting in their own right so we use the recent terminology for regular grammars to distinguish them.

4.3.2 Examples

The first example shows the convenience of the regexp notation for describing fairly long informal descriptions (a) with precision (b):¹³

- (21) a. Let A be the language of all and only the strings in $\{a, b, c\}^*$ in which every third symbol is b .
 b. Let α be the regexp $((a \cup b \cup c)^2 b)^* (a \cup b \cup c \cup \epsilon)^2$. Then $A = \mathcal{L}(\alpha)$.

The finite concatenation of for example u^2 means uu . The description in (a) does not say that all strings in A have lengths multiple of three, so the first outer parenthesis in the regexp of (b) ensures that we have b at every third position, then because of the second outer parenthesis we can have anything up to two symbols after that. The ‘up to’ part is described by iterating on ϵ . The informal description keeps it implicit but the regexp is precise in claiming that strings shorter than three are in the set since no one can argue that their third element is not b . Such is the nature of admissibility constraints.

The precision and compression are quite tempting, and we have to keep the larger perspective in mind when we use them. For example, we could write a regexp for one microcosm of English morphology as $(un \cup \epsilon)(happy \cup fair)$, to compress descriptions of four words *unhappy*, *unfair*, *happy*, *fair*. When we start thinking about including *do/undo* in the description, being a verb or not, not just being a predicate or not, begins to make a difference: *undoes* is fine, *unhappies* is probably not for an adult.¹⁴ Methods are useful, more useful when we keep the big picture in mind when we choose them.

Consider another example. The way the language is described in (a) appears to suggest that we must use set difference in its formal capture, but the fact that we can do it without set difference in (b) shows that our understanding can rise above descriptive vocabulary.

- (22) a. Let A be the language of all and only the strings in $\{0, 1, 2\}^*$ in which there is no substring 01.
 b. Let α be the regexp $(1 \cup 2 \cup 0^+ 2)^* (0 \cup \epsilon)^*$. Then $A = \mathcal{L}(\alpha)$.

Here, ‘+’ is not a new regexp operator; it is short for uu^* for any u , that is, one more more, rather than zero or more: $u^+ = uu^*$. If you think this is confusing,

¹³As before, we skip the proof that what the regexp captures has these properties; see footnote 3.

¹⁴This form could be admissible though, if we can think of a child uttering something like it. We see once more that **admissibility** is not a reified concept. We can think of it independent of methods of description and computation.

having seen ‘+’ used interchangeably with ‘∪’ in regexps, the formal context would hopefully disambiguate its function.

Notice that there is no set difference, consequently no set intersection operator in (19). We shall see the computational relevance of that in Chapter 5.

One final example about regexps and the language they describe. If we change the condition in (22a) to choosing strings out of one less element, say $\{0, 1\}^*$, then the formal description (regexp) would change in a way that requires rethinking of the structure rather than simplifying a larger solution; it can be as simple as 1^*0^* . Now try a language in which there are even number of 0’s and odd number of 1’s in every string. Informal description is easy, but formal description is nontrivial. We shall see in Chapter 5 how to obtain it in a systematic and simple way by thinking computationally.

4.4 An excursion to linear grammars

It is tempting to think of a “balanced morphology” for natural languages much like the formal problem of matching parentheses. But we must keep in mind that studying the idea of encoding the dependencies by structure (i.e. **admissibility**) assumes that we worry about that structure *because* it makes a difference in meaning therefore in **meaningfulness**. That is one reason why some calls for transregular morphology, which are loosely based on meaning related to form, have been not considered very convincing; see footnote 11.

A hypothetical natural language with balanced morphology in its word structure might have for example prefixes depending on suffixes, and doing that indefinitely in the formal pattern of $\{a^nbc^n \mid n \geq 1\}$, where *a*’s would be prefixes and *c*’s would be suffixes, *b* being the stem or root. As far as we know, there is no such language. Navajo comes close to doing that extensively for verb forms on affix interdependency (Faltz 1998), with verb root at the very end of the word, but not indefinitely.

The following example in (23a) is from Aronoff & Fudeman (2011), where the morphological process in question is **compounding**, which is a special case of **morphological derivation** (creating more words from a word), by employing two or more **lexemes** in doing derivation, which is another linguistic abstraction standing for one of the commonsense conceptions of the concept of **word** (see Aronoff & Fudeman 2011).

- (23) a. She’s a high voltage electricity grid systems supervisor.
English; Aronoff & Fudeman 2011:112–113

- b. Q: Which electricity grid systems supervisor did you see?
A: ?The high voltage one.
- c. Q: Which systems supervisor did you see?
A: *The high voltage electricity grid one.
- d. Q: Which supervisor did you see?
A: *The high voltage electricity grid systems one.

As they argue, (23a) is one noun not only because it can be substituted by one noun—phrases can be substituted by one word as well, but also because semantically speaking its referential properties do not embed like syntax, as can be seen from (b–d). The dependencies in that noun may appear to be iterating, but (b–d) also suggests that that may not be the big picture.

So let’s emancipate **morphology** from the sandbox of iterate-and-substitute and think about such dependencies from a larger perspective. Recall from §3.3 that we have this gut feeling that **embedding** *ought to be* different than **iteration** so that we worry about different conditions for admissibility. If morphology does more than iteration, then we are tasked with understanding our urge that although morphology and its semantics can embed, they do not seem to embed *like* syntax and its semantics—that seems to have a lot to do with limited reference, and we try to do that by looking at **admissibility**, that is, before any reification.

We seem to need another level of admissibility to understand this unexpected niche of a dependency microcosm, unexpected given our standard understanding of the solid lines in Figure 3.1. That has been my motivation for including **linear admissibility** (§3.2.3) in an introductory textbook. The languages of the world are faced with new challenges, especially in understanding and modeling of these differences in the technologically advanced world of making sense of questions and answers. The creators of this technology need us the linguist as much as we need them. If we just do the ‘compute talk’ with them, they have already heard it, for a long time.

English is not alone in delivering embedded albeit restricted relations of **form** and **reference** over and above iteration that one would expect from **regular dependencies**. In the following example (24a) we see a Turkish compound, which can be shown to manifest similar superiterative dependencies, which is more like the $\{a^n b^n \mid n \geq 1\}$ dependency we have assumed to be nonexistent at word-level morphology:

- (24) a. Kadıköy belediye-si vergi daire-si çöp vergi-si
K municipality-3s tax department-3s garbage tax-3s

yatırma gişeler-i bina-sı.
deposit desk-3P building-3s

‘The building of Kadıköy municipality garbage tax collection desk of
the tax department’ Turkish

We can do this indefinitely. The meaning can be decomposed into the following to make sense of its embedding: The building belongs to the municipality, the desks belong to the tax department, the desks are for garbage tax, for its deposit. Displaying the same example with these dependencies may reveal its embeddedness:

- (25) a. Kadıköy belediye-si_i vergi daire-si_j çöp vergi-si_k
K municipality-3s tax department-3s garbage tax-3s
yatırma_k gişeler-i_j bina-sı_i.
deposit desk-3P building-3s
‘The building of Kadıköy municipality garbage tax collection desks of
the tax department’ Turkish
- b. Bu ne binası? ‘Which building is that?’
c. *Bu ne dairesi? ‘Which department is that?’
d. *Bu ne vergisi? ‘Which tax is that?’

Again, the embeddings are not iterating. They are not like syntax either, as briefly shown in (b–d). Syntactic embeddings can reveal deeply embedded reference unless there are for example **island constraints**, for example in the following *syntactic construction* (26) also involving the possessive.

- (26) Reng-i-ni kim seç-ti adam-ın arkadaş-ı-nın
Color-POSS.3S-ACC who choose-PERF man-GEN.3S friend-POSS.3S-GEN.3S
araba-sı-nın?
car-POSS.3S-GEN.3S
‘Who picked the color of the man’s friend’s car?’ Turkish

Therefore we have good reasons to have a **linear grammar description** vocabulary for **linear admissibility**.

4.4.1 Definitions

Let $G=(V, \Sigma, R, S)$ be a **linear grammar**. Then all rules in R are of the form $\alpha \rightarrow \beta$ where $\alpha \in V$, $\beta \in \Sigma^*(V \cup \emptyset)\Sigma^*$. No more needs to be said about the conditions on this grammar to assume its *language* to be linear because we are now describing

admitted strings. Notice that, unlike **context-free description** of (5), there can only be one nonterminal on the righthand side, in any position. If the position is consistently left edge or right edge in all rules of R , then we would be describing a regular language (19). These are not arbitrary restrictions on grammar shapes. They arise from different admissibility conditions.

A **linear language** is a set of terminal strings in which every admitted string can be described as such, being in the language of grammars like G . (Recall that we are now in variant territory, so everything will be related to a method.)

(27) Let $G=(V, \Sigma, R, S)$ be a grammar.

- a. If G is a linear grammar, then $\mathcal{L}(G)$ is a linear language. This is the common reading. When we are interested in its connection to admissibility, we say that G must be a **linear description** if $\mathcal{L}(G)$ is a linear language satisfying linear admissibility. The wording change may sound like nitpicking; it is more about mathematical thinking of what is a consequence of what, including methods.
- b. $\mathcal{L}(G)=\{w \in \Sigma^* \mid S \xRightarrow{*} w\}$.

4.4.2 Examples

The dependencies we have been looking at in this section, which we can abstractly as $\{a^n b^n \mid n \in \mathbb{N}\}$, can be described by the following linear grammar in (a):

- (28) a. $G : S \rightarrow aSb \mid \epsilon$
 b. $\mathcal{L}(G) = \{a^n b^n \mid n \geq 0\}$.

Again, it may be tempting to think that all admitted **linear strings**, that is, strings with linear admissibility, may lead to natural descriptions. That would be a very tight admissibility indeed for a **variant**, but the modern role of **model selection** in sciences is there for a better understanding of admissibility without premature concerns for it in descriptions. The following description is linear too, and its language, the language of **palindromes**, strings that read the same forward and backward, has never been heard of in natural languages.

- (29) a. $G : S \rightarrow aSa \mid bSb \mid a \mid b \mid \epsilon$
 b. $\mathcal{L}(G) = \{w \in \{a, b\}^* \mid w \text{ is a palindrome}\}$.

The following example too shows the formidable expressive power of linear descriptions in presenting nontrivial dependencies.

$$(30) \quad \begin{aligned} \text{a. } G : \quad S &\rightarrow 0S1 \mid A \\ &\quad A \rightarrow aAb \mid \epsilon \\ \text{b. } \mathcal{L}(G) &= \{0^n a^m b^m 1^n \mid n, m \geq 0\}. \end{aligned}$$

Try to write a linear description for something that is superficially similar, say $\{0^n 1^n a^m b^m \mid n, m \geq 0\}$. Start thinking about why you instinctively switched to another admissibility after a few energetic trials, or considered decomposing the problem.

4.5 Methods for context-free grammars

In this section we look at some methods to improve grammatical description, in simplicity and adequacy, and methods to harness the improvements. We have to keep in mind that we do this given the admissibility of context-free languages (§3.2.1).

4.5.1 Structure-preserving and stringset-preserving CFGs

It is one thing to have a **weakly equivalent description** of a **stringset**, which means recognizing all and only the strings we study but not necessarily their structure, another thing to have **strongly equivalent description**, recognizing the stringset as well as its structure. The idea of admissibility is abstract enough to allow for these possibilities *in a description*, that is, in a variant.¹⁵ It means that we must be able to take a description and revise it so that it is stronger in this sense.

Here is a case study. Consider the problem of ordinary arithmetic expressions: Four operations on numbers. For the sake of keeping the description small, let us assume we do arithmetic on one-digit numbers. We can in principle describe this stringset with the following context-free grammar. (We shall see subsequently why we started writing a context-free description for it.)

$$(31) \quad \begin{aligned} G : \quad S &\rightarrow S + S \mid S - S \mid S \times S \mid S \div S \\ S &\rightarrow (S) \mid -S \mid 0 \mid 1 \mid \dots \mid 9 \end{aligned}$$

Even without proof we can see that this grammar describes all and only well-formed arithmetic expressions on one-digit numbers. For the ‘only’ part for example, $93 \div$ is not well-formed (i.e. ungrammatical), therefore not meaningful

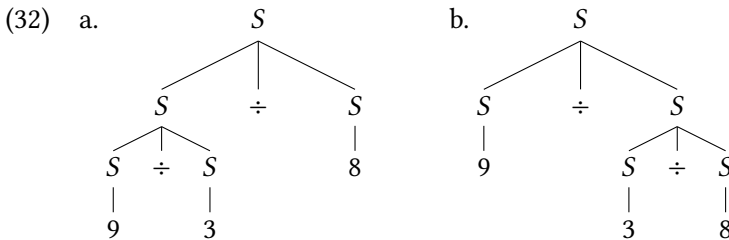
¹⁵Recall that **admissibility** already captures their invariant including their structure; we are now trying to see how well we can describe that so that further methods down the stream for example their computation and model selection can be helped in an earlier stage before testing our ideas in a later stage or communicate them better to others in the current stage.

4 Formal grammars: describing admissible strings

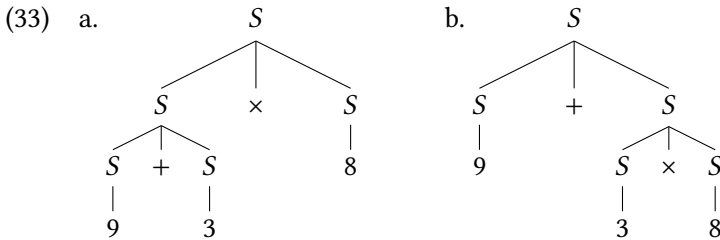
arithmetic. $(1 \times)2$ is not well-formed either. And our grammar cannot derive them connecting their derivation to S , that is, for them $S \Rightarrow^* 9 \ 3 \div$, and $S \Rightarrow^* (1 \times)2$.

Assuming the ‘all’ part without proof, we still have things to worry about for grammatical strings, two to be exact.

First, in ordinary arithmetic, given say $9 \div 3 \div 8$, we know how to do two divisions and when, commonly as $((9 \div 3) \div 8)$, that is, first divide 9 by 3, then divide that by 8. Nothing in (31) says that that must be the case. Here are two interpretations of the expression using trees as graphic illustrations of the ‘ $\Rightarrow$ ’ relation:¹⁶ The tree on the right is not what we would have in mind to get the correct result.



Second, in ordinary arithmetic, given say $9 + 3 \times 8$, without parentheses, we would interpret that as multiplication “binding stronger”, that is, done first, so it would be 9 plus 3 multiplied by 8 first, not 9 added to 3 first, times 8. Nothing in (31) says that that must be the case. Here are two trees for the expression. The one on the left is not how we interpret the structure of the expression, yet it is describable by (31).



These two problems, associativity (the first problem) and precedence/priority (the second problem), are about **meaningfulness**. That is, they assume grammat-

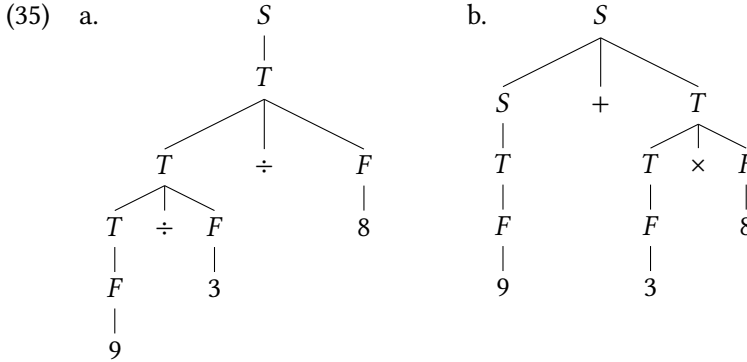
¹⁶Can you map every subtree to ‘ $\Rightarrow$ ’ notation on the grammar of (31)? The idea is simple: the symbol on lefthand side of a rule would be the local parent, and the symbols on its righthand side would be its immediate children.

icality. Can we improve on the description so that meaningfulness needs no unnecessary means to avoid overinterpretation if we can?¹⁷

The following is another attempt to formally describe the same arithmetic.

$$(34) \quad \begin{aligned} G : \quad S &\rightarrow S + T \mid S - T \mid T \\ T &\rightarrow T \times F \mid T \div F \mid F \\ F &\rightarrow (S) \mid -S \mid 0 \mid 1 \mid \dots \mid 9 \end{aligned}$$

We can see that this description avoids wrong associativity in (32b) and wrong precedence in (33a). It achieves these because left-associativity is now made a structural property in the grammar by left-recursive rules (35a). Precedence is also a structural property in this description because we cannot get to F without going through T first (i.e. innermost operator tree has precedence, and T has local access to $\times$ and $\div$), structurally unfolding for example (33b) as (35b):



Good descriptions are better reflections of **admissibility** in the sense that they manifest less reliance on extra assumptions to communicate ideas clearly to others and to constrain processes further down the stream, for example computation. We shall see that clearly in the next chapter.

4.5.2 Normal forms and binary CKY derivation

A **normal form** for a descriptive vocabulary is a pattern or shape in which all admissible properties can be described for the level we are studying. For context-

¹⁷One thing we have learned in science is that, if descriptions are relaxed for the sake of easier depiction *knowing that* further down the stream model selection would do the work of disambiguation, it may make the life of model selection harder and harder, and unnecessarily complicated. Yes some people say complicated or not it does the job, but there is always the risk of making *extra* assumptions in that complication further down the stream so that if testing goes wrong we would be in trouble diagnosing what to blame. Yes some people also say that good descriptions are just that, descriptions, but **oversimplified descriptions** are more than lack of rigor or disregard to detail.

free admissibility for example, a normal form for its descriptive language means a pattern in which all context-free grammars can be expressed. If N is a normal form for context-free grammars, it means that all context-free languages can be described in the form of N , and any given context-free grammar can be converted to a shape in N without changing its language. In that sense it is an **extensional** idea because although the stringset that is recognized would not change, the way we recognize the strings in it may change depending on the conversion. Because a normal form is about *form* and not the idea, it is clear that we are in the domain of variants when we come to normal forms for description.

We present without proof two normal forms for context-free grammars.¹⁸ The Greibach normal form (36a) is good for many things for example for elimination of **left-recursion** as it is sometimes needed in **top-down parsing**, which is one computational aspect of context-free description, which we cover in the next chapter. It does say something about the invariant too, that any context-free language has the **constant growth property** for its strings because every rule in a Greibach grammar introduces one terminal only if the language does not include the empty string. This point calls for some reflection on the invariant, back from context-free description of human strings, also, linear-indexed description, which has the same property of constant growth (not shown here). Structure building in them can be made constant per structure, and described as such.

- (36) Let $G=(V, \Sigma, R, S)$ be a context-free grammar. We have the following:
- a. G is in **Greibach normal form** if all rules in R are of the form $A \rightarrow aA_1A_2 \dots A_n$, for $n \geq 0$ and for some $a \in \Sigma, A, A_i \in V$. If ϵ is in the language, then we only have $S \rightarrow \epsilon$ in R .
 - b. G is in **Chomsky normal form** if all rules in R are of the form $A \rightarrow A_1A_2$ or $A \rightarrow a$, for some $a \in \Sigma, A, A_1, A_2 \in V$. If ϵ is in the language, then we only have $S \rightarrow \epsilon$ in R .

Chomsky normal form (36b) shows that any context-free structure can be made **binary branching**. We have known that for the mathematical invariant in the general case since Schönfinkel (1920/1924) (see §1.4 (33)). This is its adoption for the variants, that is, for description. (An empirical theory, known as **X-bar theory** starting with Chomsky (1970), has been a manifestation of both the invariant and the descriptive variant.)

One nice property of Chomsky normal form is that it shows how any context-free language can be recognized by a very simple process of deriving its strings.

¹⁸See for example Sudkamp (2006) for proofs.

It is called the **Cocke-Kasami-Younger (CKY)** algorithm, also known as CYK, named after its three inventors.¹⁹

CKY assumes a grammar in Chomsky normal form and decides whether a given string is **derivable** by that grammar, always combining two constituents at a time, and always locally looking at results of earlier stages. These results can be kept very conveniently in a two-dimensional upper-triangular matrix. Convenience and simplicity come from the fact that, if we always combine two things, then finding what to combine can be made very easy and efficient. The algorithm is short but very loopy; it is a textbook classic, see for example [Hopcroft & Ullman \(1979\)](#).

Its logic can be graphically described as in Figure 4.2. The figure is one instance of the **CKY chart** for a string of 4 elements, but it can be generalized to any length. Its point is that, given any length, we only look at binary combinations of strings of length x and strings of length y , from a position onwards, never backwards. At any point during the **derivation** such binary possibilities are very simply read from earlier results, as the combinations in every box show. Thus the process will terminate after exactly n rows for a string of length n , with just one box at the bottom covering the entire string, therefore we can always tell from one box at the bottom whether the string will have been derived by the rules of a Chomsky normal form grammar. If it contains a derivation onto S , the answer is yes, otherwise no. In other words, this is indeed a **decision algorithm**.

The rows are quite intuitive given the Chomsky normal form. We assume without loss of generality that if the input is empty, there is no point building the CKY chart. In that case, if we have $S \rightarrow \epsilon$ then the answer would be yes, otherwise no. So we build the chart for rules with nonempty strings only. The chart is upper-triangular because at any row j there can be no derivations of length less than j due to Chomsky normal form. The top row is for rules of type $A \rightarrow a$ because only they have derivations of length one. It is best we do the rest over an example. Consider the context-free grammar in (9a) of same number of a 's and b 's in every string, in any order, repeated below as (37a). It is not in the Chomsky normal form but it can put in that form quite easily as in (37b).

¹⁹There is also the fourth inventor. It has been invented earlier but had not made it to common literature, by [Sakai \(1961\)](#). I drop these notes occasionally because remembering good old ideas for new problems needs remembering forgotten folk who created them. I have been told many times by the new generation of linguistics students that historical backdrop of ideas sometimes appeal more to them than why they came to be the current states of affairs. CKY is an idea whose time had come in a historical setting of then-burgeoning formal language theory.

4 Formal grammars: describing admissible strings

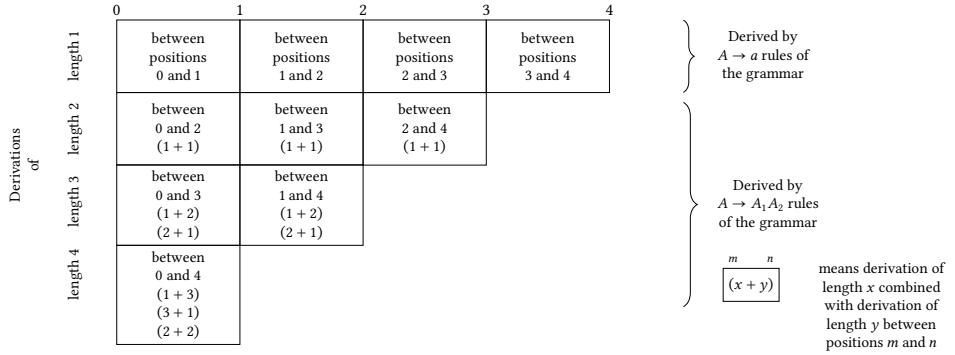


Figure 4.2: CKY derivations for a four-element string.

$$\begin{aligned}
 (37) \quad a. \quad G : \quad S &\rightarrow aB \mid bA \mid \epsilon \\
 A &\rightarrow aS \mid bAA \\
 B &\rightarrow bS \mid aBB \\
 b. \quad G' : \quad S &\rightarrow A'B \mid B'A \mid \epsilon \\
 A &\rightarrow A'S \mid B'A'' \\
 B &\rightarrow B'S \mid A'B'' \\
 A' &\rightarrow a \\
 B' &\rightarrow b \\
 A'' &\rightarrow AA \\
 B'' &\rightarrow BB \\
 c. \quad G'' : \quad S &\rightarrow A'B \mid B'A \mid \$ \\
 A &\rightarrow A'S \mid B'A'' \\
 B &\rightarrow B'S \mid A'B'' \\
 A' &\rightarrow a \\
 B' &\rightarrow b \\
 A'' &\rightarrow AA \\
 B'' &\rightarrow BB
 \end{aligned}$$

A closer inspection of (37b) shows that, although it is in Chomsky normal form, it is not best suited for CKY because the start symbol is recursive, that is, it appears on many righthand sides, therefore the $S \rightarrow \epsilon$ rule would always be in the chart because at any cell we would have one, two, many ϵ 's for any length of derivation. The empty righthand side is really there to say that this must be the end of the string. By using a special symbol for that, something which cannot be in a string, for example '\$', we can remedy this problem.²⁰ We get (c) from (b). If the language of G' is L , then the language of G'' is $L\{\$\}$.

Now we check for example for the following string in (38a) whether it is in the language of the revised grammar. One way to see that it is in the language is given in (38b), in the **derivation** notation, which is descriptive. The task of CKY

²⁰Such methods are known in science and engineering as instances of **problem relaxation**. In such methods, in order to solve the problem easier, we relax it in the sense of admitting more instances than we should, for example dealing with strings such as $s\$$ rather than s where there is room for '\$' in the data. This helps us put the grammar in a better shape, as we did, then solve the CKY problem easier, *then* restore the original **admissibility**.

is to do that algorithmically, that is, showing where and when we use ‘ $\Rightarrow$ ’ and how.²¹

- (38) a. Question: Is the string *baab* in the language of G'' ? Equivalently, can we show that $S \Rightarrow^* baab\$$?

$$\begin{aligned} \text{b. } S &\xRightarrow{S \rightarrow B'A} B'A \xRightarrow{B' \rightarrow b} bA \xRightarrow{A \rightarrow A'S} bA'S \xRightarrow{A' \rightarrow a} baS \xRightarrow{S \rightarrow A'B} baA'B \xRightarrow{A' \rightarrow a} \\ &baaB \xRightarrow{B \rightarrow B'S} baaB'S \xRightarrow{B' \rightarrow b} baabS \xRightarrow{S \rightarrow \$} baab\$. \end{aligned}$$

CKY is the algorithmic way of doing (38b). It is shown in Figure 4.3. This chart is so clean that one may get the impression CKY is a **deterministic algorithm**, one that makes one decision at every step and never backtracks. It is not. The chart is clean because the grammar it uses is a **deterministic context-free grammar**. CKY itself is complete in the sense that it will give us all derivations onto the start symbol. If the grammar is **ambiguous**, meaning allowing multiple derivations for the same string, we see more entries per cell in the chart, as in Figure 4.4.

1	$B' \rightarrow b$ 1	$A' \rightarrow a$ 2	$A' \rightarrow a$ 3	$B' \rightarrow b$ 4	$S \rightarrow \$$ 5
2				$B \rightarrow B'S$ 6 = 4, 5	
3			$S \rightarrow A'B$ 7 = 3, 6		
4		$A \rightarrow A'S$ 8 = 2, 7			
5	$S \rightarrow B'A$ 9 = 1, 8				

Figure 4.3: CKY chart for *baab\$* using G'' of (37). Numbers inside cells identify chart items and their combinations. Empty cells mark spans for which no derivation is found.

²¹A bit of crisscrossing terminology here. In **computer science**, mathematically speaking an **algorithm** is an **automaton**, which computes; it is not a description. Grammars describe and automata compute what they describe. That is the theory of **parsing**. In the literature you may hear “grammar parsing” or “parsing grammars,” which make some computer scientists lose hair.

1	$S \rightarrow n$ 1	$P \rightarrow +$ 2	$S \rightarrow n$ 3	$T \rightarrow *$ 4	$S \rightarrow n$ 5
2		$R \rightarrow PS$ 6 = 2, 3		$Q \rightarrow TS$ 7 = 4, 5	
3	$E \rightarrow SR$ 8 = 1, 6		$S \rightarrow SQ$ 9 = 3, 7		
4		$R \rightarrow PS$ 10 = 2, 9			
5	$S \rightarrow SR$ 11 = 1, 10 $S \rightarrow SQ$ 12 = 8, 7				

Grammar

$S \rightarrow n$	$R \rightarrow PS$
$P \rightarrow +$	$Q \rightarrow TS$
$T \rightarrow *$	$S \rightarrow SR$
	$S \rightarrow SQ$

Figure 4.4: CKY chart for $n+n*n$. Empty cells indicate spans for which no derivation is found. The final chart item contains two derivations, corresponding to the two possible bracketings of the expression.

4.6 Methods for regular expressions

The current understanding of regular expressions is that if they are simple enough we write them down. If the pattern is too complicated we leave it to **automata** to **compute** the complexity, which we can turn into a workable **description** if we need it. This pattern of regexp use has become increasingly popular in linguistics because of the need for **language annotation**, which is typically done in textual forms, just like regexps. We cover the relation of expressions to computation in the next chapter. Let us wrap up this chapter by looking at what we can do to simplify **regexps** (19) in their own formal niche.

4.6.1 Some descriptions are stylistically simpler than others

One common simplification of regexps is factoring the common parts, which is typically done to reduce to size of a regexp in word-based morphological processors that use them over an orthography. Even the change of terminology may give you the impression that we have moved from **admissibility** to its description, then to simplification of description, to practicalities.

Take for example, $uv \cup uw$, where u, v, w are regexps. It can be factored to $u(v \cup w)$. Things can be more complicated, for example $u(u \cup v)^* \cup v(u \cup v)^*$, which can be simplified to $(u \cup v)^+$. Recall that this plus is not a new operator, it is itself simplified notation as u^+ for uu^* . The idea in such simplifications is that we preserve the language of the regexp. Since regular structures are more **iterating** than **embedding** (see §4.4), such **weak equivalence**, equivalence of two sets rather than structures, is useful.

Some of the regular expression identities in the sense of their weak equivalence are listed for convenience in the following set (without proofs).

(39) Assume that u, v are regexps. We have

- a. $u^* = (u^*)^*$
- b. $(uv)^*u = u(vu)^*$
- c. $(u \cup v)^* = (u^* \cup v^*)^* = u^*(vu^*)^* = (u^*v)^*u^*$.

For example we can show the following simplification in style. Recall that simple description does not entail simple computation. The inverse does not hold either. These are notions of simplicity applied to two different domains of **variants**, so they are hardly surprising.

(40) To see that $u(u \cup v)^* \cup v(u \cup v)^* = (u \cup v)^+$:

- a. Apply (39c) to the left to get $u(u^*v)^*u^* \cup v(u^*v)^*u^*$.²²
- b. Factor to get $(u \cup v)(u^*v)^*u^*$.
- c. Apply (39c) to get $(u \cup v)(u \cup v)^*$,
- d. which is equivalent to $(u \cup v)^+$.

4.6.2 Regular relations

It was **Johnson (1972)** who first noticed that the phonological patterns in the now-classic **Chomsky & Halle (1968)** (SPE) are in fact **regular relations**, regular patterns of one string related to another under constraints. The SPE notation is lingua franca in **phonology**:

(41) $A \rightarrow B/C_D$

It means that objects conforming to the pattern A relate to objects conforming to the pattern B provided that A is in the context of C and D .

²²This step is actually redundant; we can directly factorize as in (c). We show it to move in to the comfort zone of having less ‘ $\cup$ ’s for factoring without risk.

We are concerned with phonological objects, therefore these are patterns of **iteration** rather than **embedding**.²³ In terms of **admissibility**, these are regular patterns for **variants**. In the regular relation notation (41) would be one wholistic expression rather than a pattern, replacement, and context: $(C : C)(A : B)(D : D)$. We explain the notation below, but it must be obvious what is coming: C does not relate to something else, it is the context *before* A . D does not relate to something else, it is the context *after* A . B is the replacement, for A . But this notation adds something more: If we write the expression as $(C : C')(A : B)(D : D')$, we can see that C, A, D are in one dimension of values, and C', B, D' are in another dimension of values where $C = C', D = D'$. This is made explicit in the regular relation notation, which is fashioned after regexps ever since [Johnson \(1972\)](#), for example in the comprehensive resource of [Beesley & Karttunen \(2003\)](#).

- (42) Let Σ be a finite set, the **input alphabet**. Let Γ be a finite set, the **output alphabet**. The regular relations on $\Sigma \times \Gamma$ are given by a **recursive definition**:
- $\emptyset : \emptyset$ is a regular relation.
 - $a : b$ for all $a \in (\Sigma \cup \{\epsilon\}), b \in (\Gamma \cup \{\epsilon\})$ is a regular relation. If $a = b$, it is common (and confusing) practice to write $a : b$ as a or b .
 - If α and β are regular relations, then so is $\alpha\beta$, the concatenation of two regular relations. The notation $\alpha \circ \beta$ is possible but not very common.
 - If α and β are regular relations, then so is $\alpha \cup \beta$, the union of two regular relations. The notation $\alpha + \beta$ is more common in computer science.
 - If α is a regular relation, then so is α^* , the **Kleene star** of α .
 - Nothing else is a regular relation.

In other words, a regular relation has *two* languages: the **input language** and the **output language**. That is why it must draw from different kinds of values. It too is a great tool for **language annotation**, just like regexps, *also* for **language transformation**, provided that we can textualize their alphabets. Their consequent computation—computing the strings of *two* languages at once—is a **transducer**, literally an output device, rather than input-only recognition **finite automaton** of regular languages. We shall see the difference in the next chapter.

We can see the expressive and transformative power of regular transformation by reconsidering our earlier formal example, the language in which out of a, b, c every third symbol is b . The regexp from (21) is $((a \cup b \cup c)^2 b)^* (a \cup b \cup c \cup \epsilon)^2$. If we

²³This is not to say that there is no embedding in phonology. It just means that we are here talking about iteration of these embeddings rather than embeddings themselves such as hierarchically conceived syllable structures.

want to recognize that language *and* “delete” all inputs except c , we can write the regular relation $((a:\epsilon \cup b:\epsilon \cup c:c)^2 b:\epsilon)^* (a:\epsilon \cup b:\epsilon \cup c:c \cup \epsilon:\epsilon)^2$. The input-output terminology is very common, but mathematically speaking, nothing is deleted. We just have two languages to put in correspondence rather than recognize only one. According to this regular relation, $aabc : c$ is a meaningful relation whereas $aabc : ac$ is not (its input language fits the bill, but not its output language). $aacc : cc$ is not a relation made meaningful by this regular relation either: Although it maps all non- c ’s to null, it does not do so by recognizing its input language. The uniqueness of the combination of two languages will be made computationally explicit in the next chapter by designing *one* transducer for every regular relation. We know that we must be able to do this because we have one admissibility to understand regular relational structures of grammaticality and consequent meaningfulness.

The idea of regular *relation* puts the discussion of the nature of structure and dependency in linguistic strings in a new perspective. We know that $A = \{a^n b^n \mid n \in \mathbb{N}\}$ is not regular, but $(a:b)^*$ is, which might appear to relate every a with b , just like A . It begs us to avoid committing a **category mistake** that Ryle had warned us against from everyday examples (see Chapter 2, footnote 6). The same *dependency* can be as simple as a relation and complicated as a language. To reiterate my revisiting of the basics in the introduction, relations are *not* failed functions. They are just different species.

We close this chapter with a linguistic example concerning regular relations. Johnson 1972:36 wrote the Sanskrit nasal retroflexion rule in the then-current SPE notation as follows:

$$(43) \quad \begin{bmatrix} +cor \\ +nas \\ \sigma \end{bmatrix} \rightarrow \begin{bmatrix} -ant \\ (\sigma) \end{bmatrix} / \$^* \begin{bmatrix} +cont \\ +cor \\ -ant \\ -distr \end{bmatrix} [-cor]^* \text{ — } [+son] \*$

Earlier in the book, he gives an informal description of what he set out to capture formally (with my emphasis added): “the Sanskrit rule which changes a dental n to η when the n is (a) preceded *somewhere in the same word* by a retroflex continuant without an intervening palato-alveolar, retroflex, or dental consonant, and (b) followed immediately by a sonorant.” Johnson 1972:13.

His formal notation can be translated as follows: The conditioning segment of the word is essentially a retroflex continuant expressed in the form of SPE **features**.²⁴ In Sanskrit these are for example ς , η and r . The σ means other features

²⁴The features are: $\mp cor$: coronal, noncoronal, $+cont$: continuant, $-ant$: not anterior, posterior within the coronal region, $-distr$: nondistributed (tongue tip), $+nas$: nasal, $+son$: sonorant.

of the segment, which are not relevant to the problem to mention. (σ) means we maintain them in the output language. $\* means any number of segments for the rule to apply. If we turn these vectors of features into input and output languages, to some kind of **feature geometry** of correspondences, we can see that he is describing a regular relation, and its domain of application, which is implicit in the formal rule, is the word. These are made explicit by Kaplan & Kay (1994). They did that because they moved from description to computation, which is the topic of next chapter.

Admissibility and description can be liberal in the sense that we communicate these ideas to other fellow human beings, who can fill in the blanks themselves if they feel like something is missing, something we do without noticing in many cases, which we cannot do when we asked a computer to compute the string without **homunculi**; see §1.8.

Computers are there to solve our problems, not their problems, whatever they are, or we their designers (not discoverers) project them to be.²⁵

²⁵This is another way of saying that computers are designed to try out human **intentions**, intention with ‘t’, humans leveraging their ideas in the form of their **intensions** by designing **representations**, including their **encoding** and **decoding**, for intension with ‘s’, so that they can be tried without homunculi. “Emergent consciousness” does not follow from this, and we can avoid the c-word (consciousness) concerning computers (Bozşahin 2018).

5 Automata: computing admissible strings

We have seen that **admissibility** follows from an **invariant** if there is one, but it does not, cannot, fully determine the **variants**. If it were the case then there would be no need for **models** or for their selection. If we want our theories, consequently models, to be more compressed than what they attempt to explain, then this **degree of freedom** is inevitable unless you are in the winner-takes-all and loser-loses-all camp in scientific theorizing (see §2.1).

We do not need to do that for a practical reason as well, because that is what **descriptive methods** are for. These methods by themselves do not give the variants because they still answer the *what* question. To understand *how* they can do that, we design another set of tools that follows from admissibility and description. In formal language theory and linguistics, this toolbox has been **computation**.

The admissibility corresponding to **degrees of structure** in a string that is manifested in Figure 3.1 translates very transparently to **description**, as we have seen in the previous chapter. It also translates very transparently to methods of computation. This translation is much easier to see going from admissibility to descriptions first. That is why this chapter is separated from these concepts and presented later than admissibility (Chapter 3) and description (Chapter 4).

5.1 Pushdown automata

The **pushdown automaton** (PDA) is the computational counterpart of **context-free language** description. Whatever admissibility context-free dependencies can afford can be described by a context-free grammar as we have seen in the previous chapter, and any dependency we can write as a context-free grammar can have its strings computed by the PDA and vice versa.¹ A PDA is a good way to begin to see the difference between description and computation.

¹These are results that have been proven; see for example [Sudkamp \(2006\)](#).

5.1.1 Definitions

A PDA is defined as follows.

- (1) Let M be an **abstract machine**, a machine that shows how abstract computation takes place.² As a PDA we define $M = (Q, \Sigma, \Gamma, \Delta, s, F)$, where
 - a. Q is a finite set of **states**,
 - b. Σ is the **input alphabet**,
 - c. Γ is the pushdown alphabet, more commonly called the **stack alphabet**,
 - d. Δ is the **transition relation** of M . An entry such as $(p, a, A), (q, B)$ means at state p looking at $a \in \Sigma \cup \{\epsilon\}$ and stack top $A \in \Gamma \cup \{\epsilon\}$, pass over a , pop A off the stack, change to state q and push $B \in \Gamma \cup \{\epsilon\}$ onto stack.
 - e. The state s is the **start state** of M , where all computations must start.
 - f. F is a finite subset of Q where the machine entering these states can **accept** the input. We shall define **acceptance criteria** subsequently.

Abstract they may be, but they are machines nevertheless, therefore just like any machine they have **configuration**. With this concept we can define accepting configurations. The most common definition is (2c).

- (2) Let $M = (Q, \Sigma, \Gamma, \Delta, s, F)$ be a PDA. Then
 - a. $[q, \alpha, A]$ is a **configuration** of M , $q \in Q, \alpha \in \Sigma^*, A \in \Gamma^*$. It represents a complete snapshot of the machine at some point in **computation**, all the information necessary to advance the computation by M .
 - b. $[s, w, \epsilon]$ is the **starting configuration**, that is, the stack is empty and input w is ready to be processed, machine at start state s .
 - c. $[f, \epsilon, \epsilon]$ is the **accepting configuration**, $f \in F$, at which point the machine is assumed to have computed the input string. It is acceptance by accepting state, with nothing left in the input and the stack.
 - d. $[q, ax, Ay] \vdash [p, x, By]$ is **one-step computation** by M if there is the relation $(q, a, A), (p, B)$ in Δ , the transition relation. It means that if the machine is in state q looking at $a \in \Sigma \cup \{\epsilon\}$ in the remainder of the input, it will move to state p , pass over a , pop A off the stack and push B into the stack. A and B can be empty too.
 - e. The language of M is $\mathcal{L}(M) = \{w \in \Sigma^* \mid [s, w, \epsilon] \vdash^* [f, \epsilon, \epsilon], f \in F\}$.
It is the set of all and only the strings that M can compute starting

²An **abstract computation** is a computation by an abstract machine that does not need the **encoding** and **decoding** relations that all **physical computations** require; see §1.8.

from its input configuration and ending in one of its accepting configurations in zero or more steps of computation.

Compare (2e) with (5b) of Chapter 4. The descriptive terminology of ‘ $\Rightarrow$ ’ has been replaced by the procedural terminology of ‘ $\vdash$ ’; it refers to states and memory, the stack, which is limited memory in the sense that only the top is visible. Consequently, its language has a procedural definition involving contents of stack and what is left in the input. If you begin to get the feeling that these further **degrees of freedom** in dealing with **admissibility** in the **variant** (what to put in the stack, how many states do we need) must simplify **computation** to be more than just mere application of **description**, then we are moving in the right direction. We now look at these aspects.

5.1.2 Examples

We start with something we have described before as a language, (6a) of Chapter 4. We can define the computation of its strings as follows.

- (3) Let the PDA $M = (Q=\{s, p_1, p_2, q_1, q_2\}, \Sigma=\{a, b, c, d\}, \Gamma=\{A, C\}, \Delta, s, F=\{q_2\})$, where Δ is the following set of transition relations.

$$\text{a. } \frac{\Delta}{\begin{array}{lll} (s, a, \epsilon), (p_1, A) & (p_1, a, \epsilon), (p_1, A) & (q_1, c, \epsilon), (q_1, C) \\ & (p_1, b, A), (p_2, \epsilon) & (q_1, d, C), (q_2, \epsilon) \\ & (p_2, b, A), (p_2, \epsilon) & (q_2, d, C), (q_2, \epsilon) \\ & (p_2, c, \epsilon), (q_1, C) & \end{array}}$$

- b. Then $\mathcal{L}(M) = \{a^n b^n c^m d^m \mid n, m \geq 1\}$.³ For example, $aabbcd$ is computed (recognized) because, starting with it, we can go the end with an empty stack:

$$\text{c. } [s, aabbcd, \epsilon] \vdash [p_1, abbcd, A] \vdash [p_1, bbcd, AA] \vdash [p_2, bcd, A] \vdash [p_2, cd, \epsilon] \vdash [q_1, d, C] \vdash [q_2, \epsilon, \epsilon].$$

For example in step 1, we are at state s , we are transitioning to p_1 with empty stack, the input “consumed” or passed over is a , stack now contains A .

³We skip the proofs, as before. The plural is for a reason: The result requires *two* proofs: (i) $\mathcal{L}(M) \subseteq \{a^n b^n c^m d^m \mid n, m \geq 1\}$, and (ii) $\mathcal{L}(M) \supseteq \{a^n b^n c^m d^m \mid n, m \geq 1\}$. This is how the equivalence of two **arbitrarily large** sets is shown. The second one involves more or less what we do here, that it must be shown that if the string has the property, then it can be recognized by M . The first one is more tricky: We must show that if M accepts a string then it will have certain properties. These are tedious but necessary and doable proofs.

We could not do that for example for $aabbc$, which is not in the language:

- d. $[s, aabbc, \epsilon] \vdash [p_1, abbc, A] \vdash [p_1, bbc, AA] \vdash [p_2, bc, A] \vdash [p_2, c, \epsilon] \vdash [q_1, \epsilon, C]$. Although we are at the end of the input, the stack is not empty, and q_1 is not an accepting state. Think about the string $aabb$:
- e. $[s, aabb, \epsilon] \vdash [p_1, abb, A] \vdash [p_1, bb, AA] \vdash [p_2, b, A] \vdash [p_2, \epsilon, \epsilon]$. Although the input is exhausted and the stack is empty, p_2 is not an accepting state.

A pushdown automaton does not “understand” a dependency. It merely possesses exactly enough memory to verify one.

Here is another example, from [Hopcroft & Ullman 1979](#):111, slightly modified for our purposes because the original has different acceptance criteria.

- (4) Let the PDA $M = (Q=\{s, q_1, q_2\}, \Sigma=\{0, 1, c\}, \Gamma=\{R, B, G\}, \Delta, s, F=\{q_2\})$, where Δ is the following set of transition relations.

a. Δ

$(s, \epsilon, \epsilon), (q_1, R)$	$(q_1, 0, R), (q_1, BR)$	$(q_1, 1, R), (q_1, GR)$	$(q_1, 0, B), (q_1, BB)$
$(q_2, \epsilon, R), (q_2, \epsilon)$	$(q_1, 1, B), (q_1, GB)$	$(q_1, 0, G), (q_1, BG)$	$(q_1, 1, G), (q_1, GG)$
	$(q_1, c, R), (q_2, R)$	$(q_1, c, B), (q_2, B)$	$(q_1, c, G), (q_2, G)$
	$(q_2, 0, B), (q_2, \epsilon)$	$(q_2, 1, G), (q_2, \epsilon)$	

- b. Then $\mathcal{L}(M) = \{wcw^r \mid w \in \{0, 1\}^*\}$. Here α^r for any string α means the **reversal** of α ; for example, $(abcd)^r = dcba$. For example for 01c10 we have
- c. $[s, 01c10, \epsilon] \vdash [q_1, 01, c10, R] \vdash [q_1, 1c10, BR] \vdash [q_1, c10, GBR] \vdash [q_2, 10, GBR] \vdash [q_2, 0, BR] \vdash [q_2, \epsilon, R] \vdash [q_2, \epsilon, \epsilon]$.

We cover nontrivial dependencies such as those in these examples because whether we could go from description to PDA or from PDA to description is not always easy to see, until of course we see (in §5.3.1) that there in fact an algorithm for doing all this, precisely because **string admissibility** is **equational** in character. It is best to get an appreciation of a carefully designed abstract machine before we leave it to an algorithm. Notice that this one has less states than the previous one (and a different stack design) although the dependency seems a bit more complex. Descriptive complexity does not always correlate with computational concerns.⁴

The final example is the balanced parentheses problem with which we started Chapter 4 and formally described in (11) of that chapter, except that to avoid

⁴One more reason to resist reifications in understanding a problem not just the solution.

notational clutter we use a for ‘(’ and b for ‘)’. The magic of the solution will reveal itself in §5.3.1.

- (5) Let the PDA $M = (Q=\{s, q\}, \Sigma=\{a, b\}, \Gamma=\{S\} \cup \Sigma, \Delta, s, F=\{q\})$, where Δ is the following set of transition relations.

- a.
$$\frac{\Delta}{(s, \epsilon, \epsilon), (q, S) \quad (q, a, a), (q, \epsilon) \quad (q, b, b), (q, \epsilon) \quad (q, \epsilon, S), (q, aSbS) \quad (q, \epsilon, S), (q, \epsilon)}$$
- b. Then $\mathcal{L}(M)$ = every a is balanced by b . For example for $aabb$ we have
- c. $[s, aabb, \epsilon] \vdash [q, aabb, S] \vdash [q, aabb, aSbS] \vdash [q, abb, SbS] \vdash [q, abb, aSbSbS] \vdash [q, bb, SbSbS] \vdash [q, bb, bSbS] \vdash [q, b, SbS] \vdash [q, b, bS] \vdash [q, \epsilon, S] \vdash [q, \epsilon, \epsilon].$

5.2 Finite automata

A **finite automaton (FA)** is a PDA with no memory. Just from this property we can deduce that it cannot compute some dependencies, namely **strictly context-free dependencies**. However, it can compute all strings with **regular dependencies**. These have been very useful dependencies in cbemlinguistic annotation, mainly because *regular expressions* that describe them are very text-oriented tools to be employed in such environments.⁵ It will turn out that we learn something else from a method such as finite automaton that seems to reflect back on the nature of such dependencies: Although not every **context-free language** has a **deterministic PDA**, *every regular language* has a **deterministic FA**. (A **deterministic** abstract machine has exactly one option to follow in any **configuration**.) Granted that in common practice it is much easier to write a **nondeterministic finite automaton (NFA)** for any regular language, all nondeterminism in an NFA can be eliminated.

This is a point to ponder. Regular dependencies are so simple that their strings can be computed very deterministically.⁶ It is not inadequacy of methods for regular dependencies or context-free dependencies. This is about the nature of structure in their **admissibility** (Chapter 3).

⁵There is a mathematical reason why nonregular dependencies have been difficult to streamline in such environments with equal ease: They address more structured dependencies.

⁶That is not the case for **context-free languages**. Some of them are **inherently ambiguous** in the sense that there is no deterministic grammar or deterministic PDA for them, for example: $\{a^n b^j c^m d^k \mid n, m, j, k \geq 1, n = j, m = k \text{ or } n = k, m = j\}$.

5.2.1 Definitions

- (6) Let M be an **abstract machine**. As a **finite automaton (FA)** we define $M = (Q, \Sigma, \Delta, s, F)$, where
- Q is a finite set of **states**,
 - Σ is the **input alphabet**,
 - Δ is the **transition table** of M . If Δ is a **function**, then M is a **deterministic finite automaton (DFA)**. If it is a **relation** then M is a **nondeterministic finite automaton (NFA)**. An entry such as $(p, a), (q)$ means at state p looking at $a \in \Sigma \cup \{\epsilon\}$, pass over a and change to state q . Other notations are common as well; this one keeps the difference from PDAs to a minimum.
 - The state s is the **start state** of M , where all computations must start.
 - F is a finite subset of Q where the machine entering these states can **accept** the input. We shall define **acceptance criteria** subsequently.

Because FA's have no memory, their **configuration**, therefore **acceptance criteria**, are relatively simpler:

- (7) Let $M = (Q, \Sigma, \Delta, s, F)$ be an FA. Then
- $[q, \alpha]$ is a **configuration** of M , $q \in Q, \alpha \in \Sigma^*$. It represents a complete snapshot of the machine at some point in **computation**, all the information necessary to advance the computation by M .
 - $[s, w]$ is the **starting configuration**, that is, the input w is ready to be processed, machine at start state s .
 - $[f, \epsilon]$ is the **accepting configuration**, $f \in F$, at which point the machine is assumed to have computed the input string. It is acceptance by accepting state, with nothing left in the input.
 - $[q, \alpha] \vdash [p, x]$ is **one-step computation** by M if there is the relation $(q, a), (p)$ in Δ , the transition relation. It means that if the machine is in state q looking at $a \in \Sigma \cup \{\epsilon\}$ in the remainder of the input, it will move to state p , and pass over a .
 - The language of M is $\mathcal{L}(M) = \{w \in \Sigma^* \mid [s, w] \vdash^* [f, \epsilon], f \in F\}$.
It is the set of all and only the strings that M can compute starting from its input configuration and ending in one of its accepting configurations in zero or more steps of computation.

5.2.2 Examples

The following first example is very easy to understand in terms of FA, somewhat elaborate to write as **regex**. This is all the more interesting considering the fact that it is actually a DFA. It shows once more that **description** and **computation** are different but related species.

- (8) Let the FA $M = (Q=\{s, q_1, q_2, q_3\}, \Sigma=\{a, b\}, \Delta, s, F=\{q_2\})$, where Δ is the transition table in Figure 5.1. For example, we have $(s, b), (q_1)$ and $(q_2, a), (q_1)$ in Δ . Notice that it is a **function**, therefore M is a DFA.
- Then $\mathcal{L}(M) = \{w \in \{a, b\}^* \mid \text{there are odd numbers of elements from the alphabet in any string}\}$. For example for $bbab$ we have
 - $[s, bbab] \vdash [q_1, bab] \vdash [s, ab] \vdash [q_3, b] \vdash [q_2, \epsilon]$, and $q_2 \in F$.
 - For $aab \notin \mathcal{L}(M)$, we have $[s, aab] \vdash [q_3, ab] \vdash [s, ab] \vdash [s, b] \vdash [q_1, \epsilon]$, and $q_1 \notin F$.

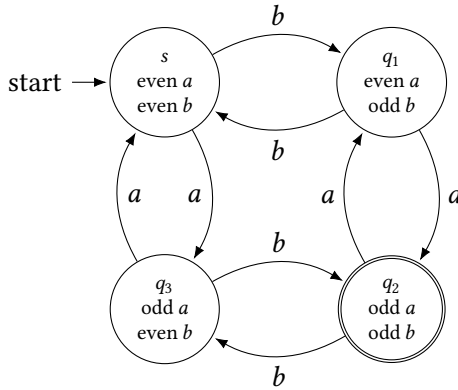


Figure 5.1: Deterministic finite automaton (DFA) for strings with odd numbers of both a 's and b 's. Accepting states are double circled; others are nonaccepting.

The second example which we write just as a graphic transition table in Figure 5.2 is also a DFA. It is not all too obvious that its **transition function** is from something we have seen earlier to describe just as easily, (21) of Chapter 4. That is what good mathematical methods we shall soon see are good for, to connect these seemingly unrelated **variants**.

Finally, consider the example in Figure 5.3. Its transition table is worth writing mathematically to see that it is indeed not a function. It is an NFA.

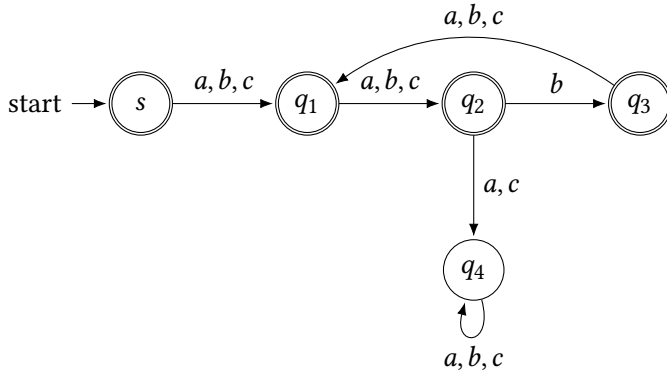


Figure 5.2: A deterministic finite automaton (DFA) for computing all and only strings out of $\{a, b, c\}^*$ where every third symbol is b . Accepting states are double circled; others are nonaccepting. One **regexp** for it is $((a \cup b \cup c)^2 b)^* (a \cup b \cup c \cup \epsilon)^2$.

- (9) Let the FA $M = (Q=\{s, q_1, q_2, q_3\}, \Sigma=\{a, b, c\}, \Delta, s, F=\{s, q_1, q_2, q_3\})$, where Δ is the transition table in Figure 5.3.
- Then $\mathcal{L}(M) = \{w \in \{a, b, c\}^* \mid \text{starting with } b\text{'s and } c\text{'s, there is no } bc \text{ after any } a\}$. Notice that all states are accepting but not all strings are accepted; For example for $bcabc \notin \mathcal{L}(M)$ we have
 - $[bcabc] \vdash [s, cabc] \vdash [q_1, abc] \vdash [q_2, bc] \vdash [q_3, bc] \vdash [q_3, c]$, and we cannot get to the end of the input.
 - For $baa \in \mathcal{L}(M)$, we have $[s, baa] \vdash [q_1, aa] \vdash [q_2, a] \vdash [q_1, a] \vdash [q_2, \epsilon]$, and $q_2 \in F$.

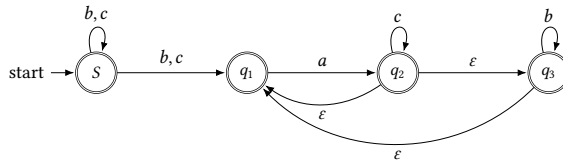


Figure 5.3: An NFA over $\{a, b, c\}$ for strings with an initial numbers of b, c prefix and no bc sequence after any a . Accepting states are double circled; others are nonaccepting. One **regexp** for it is $(b \cup c)^+ (ac^* b^*)^*$.

Its **transition relation** is not a function; for example, $(s, b), (q_1) \in \Delta$ and so is $(s, b), (s)$. In other words, given the same next symbol, Δ provides two ways to

proceed from the same state. Also, it has many **empty transitions**. Since they do not depend on the input, this is also quite nonfunctional.

It must be clear by now that **description** and **computation** are best not to be conflated with each other or with **admissibility** so that we can see what they are trying to do more clearly. We have just seen that we cannot equate acceptability by being in an accepting state. The **acceptance criteria** for every level of linguistic computation is different because they address different **admissibility**. We shall see in §5.4.1 that being able to eliminate *all nondeterminism* in computing such as the one in Figure 5.3 is not a narrative but a consequence of a theorem. The fact that we cannot do this for linguistic dependencies that are strictly nonregular shows that we are trying to come to grips with different kinds **empirical problems**, *consequently* employing different kinds of descriptions and computations.

We shall see through methods that just as an FA is a reduced PDA, reduced to work without a stack, a PDA is a reduced **Turing machine (TMs)**, reduced to work with one stack. These are not reductions for reduction's sake; they show that **structure building** really depends on what kinds of **dependencies** we can keep and for how long. You may have seen depictions of TMs with just one long tape and an FA. That is all there is to it. Even with such impressive memory we can see that whatever we can do with a tape we can also do with two stacks because if we have two stacks, they make *two* sides of a structured string workable at two tops, no more, much like a serial tape. And knowing that any **computable function** can be couched as a TM (known as the **Church-Turing thesis**), these results should not be surprising. There are no more things that we can compute by adding more than a tape or more than two stacks.⁷ Computation as we know it today works very **locally** to achieve great effects. We try to narrow down methods *so that* their connection to **description** and **admissibility** can be seen easily, and the consequent computations would work more effectively.⁸

⁷See for example Lewis & Papadimitriou (1998) for these results.

⁸It follows that there are some **dependencies** in computation that are quite unheard of in **natural language**; for example the **halting problem** of the TM, whether a given TM would halt on any input. We can write a TM for it, but we cannot compute such dependencies in all cases, unlike language. The problem is **semidecidable**. However, we can at least **represent** the dependency, although we may not be able to compute it. There are also problems in TM that we do not know how to represent, let alone compute. (Recall that if we cannot represent, we cannot compute.) For example, what is the next number after π ? Computation in its own admissibility is telling us that computable problems are not that kind of problems. Relating study of **structure** to **representability** is one important contribution from **philosophy of computer science**.

5.3 Methods for pushdown automata

These methods connect context-free computation to context-free description, showing that once we have one, the other one follows, although nontrivially. This is because of **admissibility** having an **equational** character. No mathematics has been worked out better than the mathematics of **equality of values** as we have seen for example in the discussion of **recursive description** versus **recursive values** (§1.5).

5.3.1 Obtaining a PDA from CFG: top-down parsing

The proofs for obtaining context-free grammars for every PDA are somewhat elaborate, but they are there nevertheless if one needs them. Let us concentrate on the other direction, going from **context-free description** (i.e. CFG) to **context-free computation** (i.e. PDA).

Assume that we have a context-free grammar $G = (V, \Sigma, R, S)$. The simplest but not necessarily computationally best translation of the grammar turns every rule into a PDA action and every terminal to something that the PDA can use to get to the point where it can use a rule by seeing the start symbol on top of the stack. A set of theorems shows that all we need is two states in such a PDA, which is quite a normal form for a PDA. It comes from the power of being able to use the stack for translating the rules of grammar into action. It is called **nondeterministic top-down parsing** for this reason, which has subtleties that we will talk about at the end of this subsection.

Mathematically, the **top-down translation** from a CFG to PDA involves going over every component of G in $G = (V, \Sigma, R, S)$. We set up two states, say s and q . The first one captures the **starting configuration** of the emerging PDA $M = (\{s, q\}, \Sigma, \Gamma = V \cup \Sigma, \Delta, s, F = \{q\})$ that we are building; so only q accepts if we can reach the end of the input with an empty stack. We do the following steps:

- (10) a. Set up the PDA action $(s, \epsilon, \epsilon), (q, S) \in \Delta$ where S is the start symbol of the grammar.
- b. Set up a PDA action $(q, a, a), (q, \epsilon) \in \Delta$ for every **terminal** symbol in Σ . These actions effectively take an terminal off the stack at the right context so that we can reach a **nonterminal** on top. In a grammar such moves can be implicit in the minds of interpreters, but PDAs have no such magic powers.
- c. For every rule $\alpha \rightarrow \beta \in R$, set up the PDA action $(q, \epsilon, \alpha), (q, \beta) \in \Delta$, which does not touch the input and replaces the lefthand appearing on top of the stack with its righthand side.

- d. The **starting configuration** is $[s, w, \epsilon]$ for input w . The **accepting configuration** is $[q, \epsilon, \epsilon]$ for $q \in F$.
- e. Then, we have $\mathcal{L}(G) = \mathcal{L}(M)$. That is, we will have created a PDA that computes what the grammar describes, and does it in a top-down manner.

Let us develop a top-down parser for the grammar (37c) of Chapter 4. The translation that we have seen sets up the following transition table:

Δ			
	$(s, \epsilon, \epsilon), (q, S)$	$(q, a, a), (q, \epsilon)$	$(q, b, b), (q, \epsilon)$
(11)	$(q, \epsilon, S), (q, A'B)$	$(q, \epsilon, S), (q, B'A)$	$(q, \epsilon, S), (q, \$)$
	$(q, \epsilon, A), (q, A'S)$	$(q, \epsilon, A), (q, B'A'')$	$(q, \epsilon, B), (q, B'S)$
	$(q, \epsilon, A'), (q, a)$	$(q, \epsilon, B'), (q, b)$	$(q, \epsilon, A''), (q, AA)$
			$(q, \epsilon, B''), (q, BB)$

The **CKY derivation** of the input $baab\$$ that we have seen in Figure 4.3 unfolds as steps of parsing actions as follows (these are nondeterministic choices).⁹

$$(12) \quad [s, baab\$, \epsilon] \vdash [q, baab\$, S] \vdash [q, baab\$, B'A] \vdash [q, baab\$, bA] \vdash [q, aab\$, A] \vdash [q, aab\$, A'S] \vdash [q, aab\$, aS] \vdash [q, ab\$, S] \vdash [q, ab\$, A'B] \vdash [q, ab\$, aB] \vdash [s, b\$, B] \vdash [q, b\$, B'S] \vdash [q, b\$, bS] \vdash [q, \$, S] \vdash [q, \$, \$] \vdash [q, \epsilon, \epsilon].$$

Notice that the parsing actions used here and the rules used in Figure 4.3 are the same. The actions also show explicitly how they can get to a lefthand side on top of the stack. Explicitness is what algorithmic methods are good for. Rules are descriptions for fellow human beings to follow, **computation** is us telling the computer how to do it without our intervention (recall the **no homunculi** dictum of computer science, from §1.8).

There is one subtlety in top-down parsing that we humans have no difficulty coping with but where an algorithm needs some help. If we have a **left-recursive** rule in our grammar, say $A \rightarrow A\beta$ for some nonterminal A and remainder β , the parsing action that is set up in a top-down parser would be $(q, \epsilon, A), (q, A\beta)$,

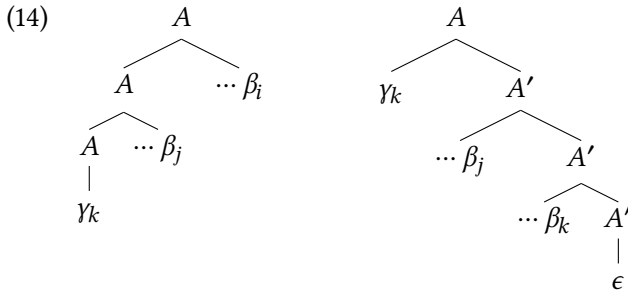
⁹If a language is **deterministically context-free**, then there are deterministic parsing algorithms, both top-down and bottom-up, which forms the basis of theory of **compilers** in computer science. Most **programming languages** of today are *designed* not to be ambiguous. For every well-formed program, they generate one structure, which gets executed in the language of the physical computer (which we design).

Think about the philosophical implications of relaxing that assumption, apart from computational inefficiency of nondeterminism. If the same **program** which is carefully crafted according to the grammar rules of a programming language can have different translations therefore different executions, on what basis can we fix these programs, or, if we allow them to proceed with multiple executions, how do we choose among what they would do?

which means the top of the stack would not change and there would not be an obvious way to stop using that action incessantly since the stack only sees its top in the progress of computation. Luckily, at least for fans of top-down parsing, there is a general method explained below to go from (13a) to (13b), for any grammar. Such grammars adhere to **Greibach normal form** (see (36a) of Chapter 4). Grammars in such a normal form are always top-down parsable because there would be no left-recursive rule in them. The fact that it is a normal form means any context-free grammar can be put in that form, therefore become top-down parsable.

- (13) a. Assume we have **left-recursive** n rules and nonleft-recursive m rules for A :
- $$A \rightarrow A\beta_1 \dots \mid A\beta_n \mid \gamma_1 \dots \mid \gamma_m.$$
- b. The stringset of A can be recognized by **right-recursive** rules by rewriting A as follows:
- $$A \rightarrow \gamma_1 A' \dots \mid \gamma_m A'$$
- $$A' \rightarrow \beta_1 A' \dots \mid \beta_n A' \mid \epsilon.$$

We can visually see that the translation works for every left-recursive structure; there would be one γ at the left edge, with many β 's because of left-recursion. The second tree captures the same stringset right-recursively.



Having to do this translation because of a method is much to the horror of the linguist, because there would a good reason why we would write a **description** as a left-recursive rule, most typically from **semantics** and **scope**, and having the same **stringset** would not be enough; we would want to preserve structure. Bottom-up parsing does not have that problem, as we shall now see.

In principle, **top-down parsing** goes from the start symbol to the string itself in implementing the computation of $S \Rightarrow^* w$. **Bottom-up parsing** goes the other way, from w to S .

5.3.2 Obtaining a PDA from CFG: bottom-up parsing

Assume as before that we have a context-free grammar $G = (V, \Sigma, R, S)$. The following translation turns every rule into a PDA action seeing the righthand side on top of the stack and replacing it with the lefthand side. It is called **nondeterministic bottom-up parsing** for this reason. If we can keep doing this, we will have reached the start symbol if the input string is in the language of G . Two states suffice in such a PDA as well. Again, this comes from the power of being able to use the stack for translating the rules of grammar into action.

Mathematically, the **bottom-up translation** from a CFG to PDA involves going over every component of G in $G = (V, \Sigma, R, S)$. We set up two states, say s and q . The second one captures the **accepting configuration** of the emerging PDA $M = (\{s, q\}, \Sigma, \Gamma = V \cup \Sigma, \Delta, s, F = \{q\})$ that we are building when all rule application will have been done in s ; so only q accepts if we can reach the end of the input with just the start symbol, at which point we can accept. We do the following steps to set up the PDA:

- (15) a. Set up the PDA action $(s, \epsilon, S), (q, \epsilon) \in \Delta$ where S is the start symbol of the grammar.
- b. Set up a PDA action $(s, \epsilon, \beta), (s, \alpha) \in \Delta$ for every rule $\alpha \rightarrow \beta$ in R . These actions effectively replace a righthand side with its lefthand side. Since the lefthand side is *always* one symbol in a context-free grammar but the righthand side can be larger, this is commonly called **reduction**, especially in computer science. The subtlety is that the righthand side β needs to be located *in the stack* looking from the top; it may not always be on top.¹⁰ What is put in the stack goes in to the part of stack that was located as the righthand side.
- c. The **starting configuration** is $[s, \epsilon, w]$ for input w . The input is in the stack. The **accepting configuration** is $[q, \epsilon, \epsilon]$ for $q \in F$.
- d. Then, we have $\mathcal{L}(G) = \mathcal{L}(M)$. That is, we will have created a PDA that computes what the grammar describes, and does it in a bottom-up manner.

Let us develop a **bottom-up parser** for something we have seen done by **CKY derivations** in Figure 4.4 of Chapter 4. The rules in that grammar translate to the following bottom-up parsing actions, here shown simply by the transition relation Δ . Observe that the grammar in the figure is **left-recursive**.

¹⁰There is a way to do this without searching the stack, called **LR parsing**. We leave that to advanced texts in computer science; see for example [Hopcroft & Ullman \(1979\)](#).

$$(16) \quad \frac{\Delta}{\begin{array}{cccc} (s, \epsilon, S), (q, \epsilon) & (s, \epsilon, n), (q, S) & (s, \epsilon, +), (s, P) & (s, \epsilon, *), (s, T) \\ (s, \epsilon, PS), (s, R) & (s, \epsilon, TS), (s, Q) & (s, \epsilon, SR), (s, S) & (s, \epsilon, SQ), (s, S) \end{array}}$$

Recall also from the figure that the chart has multiple entries for the input $n + n * n$, where in one $n + n$ combines first, then with times, and in the other $n * n$ combines first, then with plus. These two interpretations are respectively (17a) and (17b). The steps in bold tell why we get these interpretations.

$$(17) \quad \begin{array}{l} \text{a. } [s, \epsilon, n + n * n] \vdash [s, \epsilon, S + n * n] \vdash [s, \epsilon, SPn * n] \vdash [s, \epsilon, SPS * n] \vdash \\ \quad [s, \epsilon, \mathbf{SR} * n] \vdash [s, \epsilon, S * n] \vdash [s, \epsilon, STn] \vdash [s, \epsilon, STS] \vdash \\ \quad [s, \epsilon, SQ] \vdash [s, \epsilon, S] \vdash [q, \epsilon, \epsilon]. \\ \text{b. } [s, \epsilon, n + n * n] \vdash [s, \epsilon, S + n * n] \vdash [s, \epsilon, SPn * n] \vdash [s, \epsilon, SPS * n] \vdash \\ \quad [s, \epsilon, SPSTn] \vdash [s, \epsilon, SPSTS] \vdash [s, \epsilon, \mathbf{SPSQ}] \vdash [s, \epsilon, SPS] \vdash \\ \quad [s, \epsilon, SR] \vdash [s, \epsilon, S] \vdash [q, \epsilon, \epsilon]. \end{array}$$

5.4 Methods for finite automata

The methods we cover in this section are most useful in **modeling tools** where some reorganization of **regular dependencies** may go a long way in simplifying the task. They also show that we can do these simplifications *because* these dependencies are so simple in structure.

Memorylessness of the methods we cover in this section is not from a practical choice. Before we look at the methods that will look quite “flat” in terms of **structure**, have a brief look at **admissibility** conditions in Chapter 3, of **context-freeness** (3), **regularity** (4), and **linearity** (5). Only the regular one has no **dependency** reaching for both sides of a **center**. The flatness that we shall see in this section arises from this admissibility condition, not from a desire for methodological simplicity. For the other two, observe that a **stack** is good enough to understand their dependency, never needing another kind of memory or more than one stack as the **Turing machine (TM)** does. The result that even a TM does not need more than two stacks to compute any computable dependency shows that the consequent choice of **linguistic admissibility** seems to be on the right track to give us constrained **variants for description and computation**.

5.4.1 Eliminating nondeterminism

We shall see that **nondeterminism** in **regular dependencies** can be completely eliminated. Although the common practice is to continue to write NFAs because

they are easier to conceive, as we shall see in §5.4.2, we know from theorems that when it comes to computing we can eliminate it.

We can see the sources of **nondeterminism** in Figure 5.3. For example, from the start state we can stay in it or move to q_1 if the current symbol is b . Same with c . From q_2 we can ignore input and move to q_3 , and continue to do so to q_1 . These are called **empty transitions**.¹¹ There are no b or c transitions from q_1 . In other words, that FA is not a **function**.

One way to compile all these nondeterminisms is to treat them as equivalences, equivalence of states in not passing over any symbol, equivalence of states in passing over the same symbol, and equivalence of states in not doing anything, that is, having no transition. Since these are the only sources of nondeterminism at this level of **dependencies**, it will take care of every case. This is called the **subset construction**. It is much simpler to see the idea if we start with two definitions before the construction as Sudkamp (2006) does.

(18) Let FA $M = (Q, \Sigma, \Delta, s, F)$. Define for each state q in Q :

- a. **empty-closure** of q : The set of all states we can go in Δ with **empty transitions** only. Since we can stay in any state in doing that, empty closure of a state always includes itself.
- b. **input-transition function** t for any pair of q and $a \in \Sigma$: The set of all states that we can reach from q by passing over *one* a , preceded and succeeded by empty transitions.
- c. Define a DFA $M' = (Q', \Sigma, \Delta', s', F')$ from M as follows:

1. s' is the empty-closure of s .
2. Set up the input transition function t as Q' : 3. Starting with s' , which is now a set of states of M , define for every $a \in \Sigma$ the set of states we can reach from the sets of states we are looking at. That is, form the **set union** of every q in the K of $t(K, x)$, for every $x \in \Sigma$. 4. If we obtain a new set of states, repeat the process until no more new sets of states are produced.

Since Q of M is a finite set, and so is Σ , this process will always terminate.

5. F' is the set of set of states from M which contains an **accepting state**.
- d. From this construction, M and M' provably compute the same **stringset**. That is, $\mathcal{L}(M) = \mathcal{L}(M')$. Δ' is a function, therefore M' is a DFA.

¹¹They are *not* empty as **abstractions**. They are not redundant as computation because they change state. They are empty in content.

Let's apply this method to the NFA of Figure 5.3. The **input transition function** t is the following. Notice that entries in the table are sets. There are as many columns as the size of Σ , and as many rows as the size of Q .

t	a	b	c
s	$\emptyset$	$\{s, q_1\}$	$\{s, q_1\}$
q_1	$\{q_1, q_2, q_3\}$	$\emptyset$	$\emptyset$
q_2	$\{q_1, q_2, q_3\}$	$\{q_1, q_3\}$	$\{q_1, q_2, q_3\}$
q_3	$\{q_1, q_2, q_3\}$	$\{q_1, q_3\}$	$\emptyset$

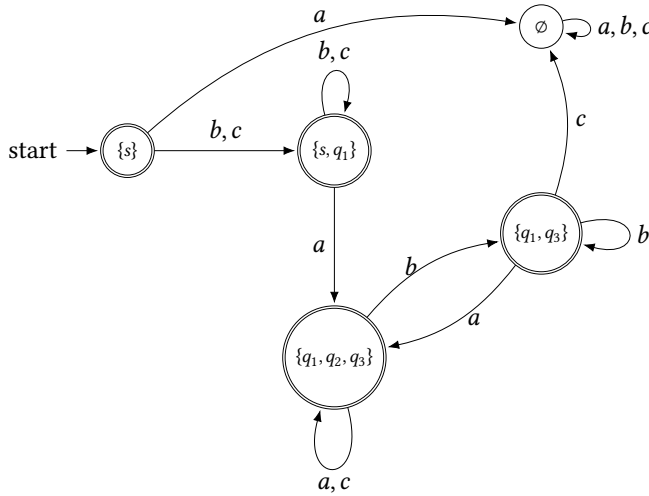


Figure 5.4: Deterministic finite automaton obtained by subset construction. Accepting states are double circled; others are nonaccepting. One **regex** for it is $(b \cup c)^+(ac^*b^*)^*$. This is a DFA version of the NFA in Figure 5.3. The names in sets of states in the machine are from the NFA. That is why it is called the **subset construction**.

For example, $t(q_3, a) = \{q_1, q_2, q_3\}$ because before we pass over an a we can empty-transition to q_3 then to q_1 , 'take' the a , then stay there or empty transition to q_3 then to q_1 *after* a . From s there is no a transition, with or without empty transition, therefore $t(s, a) = \emptyset$. Once we have the empty set, since there is no M state in it, all its transitions will be back to itself.

So we start constructing the DFA by taking the empty closure of the start state of the NFA we started with, and set every transition from that state. For the example **empty-closure**(s)= $\{s\}$. Now we look at $t(\{s\}, a)$, $t(\{s\}, b)$, and $t(\{s\}, c)$, to see if they lead to new sets of states. For example for $t(\{s\}, b) = \{s, q_1\}$ we have a new subset, and we repeat the process for $\{s, q_1\}$. This new set of states

has two members, s and q_1 , so $t(\{s, q_1\}, x)$ is the **set union** of all the states we can go in the table, for every $x \in \Sigma$. For example, for $t(\{s, q_1\}, b)$ is the union $t(\{s\}, b) \cup t(\{q_1\}, b) = \{s, q_1\} \cup \emptyset = \{s, q_1\}$. Similarly, $t(\{s, q_1\}, a) = t(\{s\}, a) \cup t(\{q_1\}, a) = \emptyset \cup \{q_1, q_2, q_3\} = \{q_1, q_2, q_3\}$. Continuing that way for all combinations of new set of states and input symbols we will have set up a **function**. Finally we double circle every set of states that contain an accepting state of Figure 5.3 (s, q_1, q_2, q_3). These are the accepting states of M' . In the end we get the DFA in Figure 5.4.

Notice that the DFA for the same stringset has more states than the NFA for it in Figure 5.3. We cannot equate simplicity and determinism with number of states.

5.4.2 From regular expressions to automata and back

There are many transparent methods to go from **regular expressions** to **regular computations** (i.e. FAs) back and forth. The method we describe here is more linguistics friendly, assuming that we transform our **regular grammars** to **finite automata** first, then tackle the question of relating regular expressions to regular grammars in a transparent way, something we can always do. We look at the logic of this transformation first.

If we have a **regular grammar**, all rules are of the form $A \rightarrow \epsilon \mid a \mid aB$. In terms of FA, we can think of the first two cases as **accepting configurations** because there is no more unknown, that is, a **nonterminal**, to continue. We can think of them as reaching an accepting state starting from A . We make up one accepting state (say Z) in the automaton to lead these continuations from the ϵ and a from a “state A ” to “state Z .” For the third kind of rule, we must continue. The way we continue is determined by the rule, by A , a , and B . In other words, it *depends* on the grammar. (Z depends on the grammar too, but it is not obvious from the grammar rules.) In the translation of such rules to an FA, it is equivalent to going from state A to state B through a . We can do this for all rules since all of them are in these forms. This forms the basis of the algorithm described in [Sudkamp \(2006\):205](#), slightly simplified and modified to fit our notation:

- (20) Let $G = (V, \Sigma, R, S)$ be a regular grammar. Define the NFA $M = (Q, \Sigma, \Delta, s = S, F = \{Z\})$ the following way. Notice that the start symbol of the grammar has become the start state of the NFA.
 - a. The nonterminals of G are the states of M . If there are rules of the form $A \rightarrow \epsilon \mid a$, we add one unique new nonterminal Z , $Z \notin V$, to Q . Since there must be at least one such rule if the grammar has a language, Z is necessary. Since we are building a **nondeterministic FA**, one such Z will be sufficient to cover all such kinds of rules.

- b. In constructing the transition function Δ , add the entry $(A, a), (B)$ for every rule $A \rightarrow aB$. Add the entry $(A, x), (Z)$ for every rule $A \rightarrow x$, $x = \epsilon$ or $x = a, a \in \Sigma$.
- c. Then $\mathcal{L}(G) = \mathcal{L}(M)$. That is, we maintain the **stringset** equivalence of the strings described by the grammar and the strings computed by the automaton.

Let us work through an example. The following example, repeated from §4 (22a), was described there with a regular expression as shown in the following (b), but we shall ignore it for the moment and write a regular grammar for it as well.

- (21) a. Let A be the language of all and only the strings in $\{0, 1, 2\}^*$ in which there is no substring 01.
- b. Let α be the **regexp** $(1 \cup 2 \cup 0^+2)^*(0 \cup \epsilon)^*$. Then $A = \mathcal{L}(\alpha)$.

Assume that we come up with the following grammar for it. Notice that it is a **regular grammar**. Its equivalence to the regular expression in (21b) we will develop as we proceed.

- (22) Let $G = (Q = \{S, A\}, \Sigma = \{0, 1, 2\}, R, S)$ where R is

$$\begin{aligned} S &\rightarrow 1S \mid 2S \mid 0A \mid \epsilon \\ A &\rightarrow 0A \mid 2S \mid \epsilon \end{aligned}$$

From the $S \rightarrow \epsilon$ and $A \rightarrow \epsilon$ rules we set up two transitions $S \xrightarrow{\epsilon} Z$ and $A \xrightarrow{\epsilon} Z$ according to the algorithm in which S and Z are now states not nonterminals. Recall that Z is going to be the only accepting state. From the remaining S rules we get $S \xrightarrow{1} 1$, $S \xrightarrow{2} 2$, and $S \xrightarrow{0} A$. From the remaining A rules we get $A \xrightarrow{0} 0$ and $A \xrightarrow{2} S$. Combining all of them (because “same name” now means “same state”) we get the automaton in Figure 5.5.

As expected from a grammar, (22) does not tell us when strings violate the conditions described by the grammar. This is something for automata to do. The NFA does that. It can be made deterministic by applying the algorithm in (18). The result is shown in Figure 5.6. Notice that it is more compressed about when strings violate the conditions of the language.

What if we do not have a regular grammar; we have a **regexp** instead? We can turn the regular expression to a regular grammar and apply the algorithm described in (20). In computer science it is more common to relate regular expressions and FA directly. It is important in linguistics to realize that what we do in the process is translating a grammatical description to its recognition; that is,

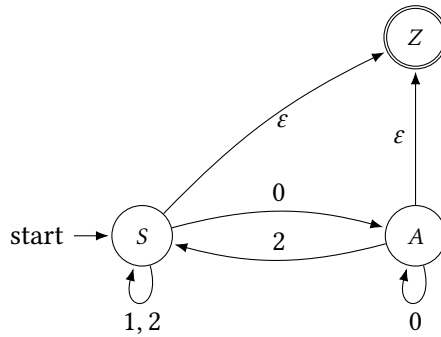


Figure 5.5: The NFA derived using the algorithm in (20) for the regular grammar in (22).

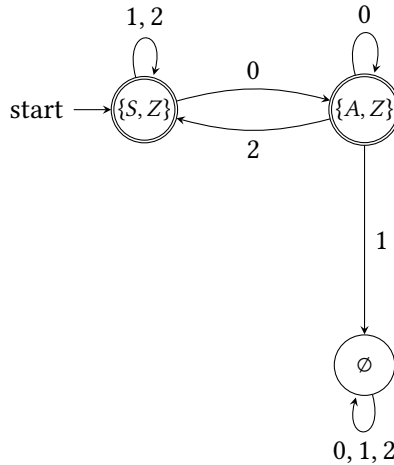


Figure 5.6: The DFA derived from Figure 5.5 by the algorithm in (18).

we are dealing with a grammar, with **regular dependencies**, but dependencies nevertheless.

And what if we want to go the other way, to derive a regular expression from a finite automaton? Here again we can see the benefit of having all these methods even though we know that **nondeterminism** at this level of complexities in dependencies is not necessary. It is much easier to derive a regexp from an NFA first compared to a DFA. Since we know how to convert any NFA to a DFA, this simplification of the task is nontrivial for the working linguist.

Let us say we have the FA $M = (Q, \Sigma, \Delta, s, F)$. It does not matter whether M is deterministic or nondeterministic. Make up a new state, say $Z \notin Q$. From every

accepting state in F define an empty transition to Z , that is $(f, \epsilon), (Z)$, for every $f \in F$. We make all the states of the developing machine nonaccepting except Z . Now we definitely have an NFA, say M' . We are working our way backwards so that we can recycle the algorithm in (20) for regular grammar construction, then moving on to regular expression construction. Now:

- (23)
- a. Every state in M' except Z becomes a **nonterminal**.
 - b. Every transition $A \xrightarrow{a} B$ in M' becomes the **right-regular** rule $A \rightarrow aB$.
 - c. Every transition $A \xrightarrow{\epsilon} Z$ in M' becomes the right-regular rule $A \rightarrow \epsilon$.
 - d. There are no $A \xrightarrow{a} Z$ transitions in M' because of the way we designed the special state Z ; therefore nothing needs to be done about them.
There will be no $A \rightarrow a$ rules in the developing grammar.

We can apply the algorithm (20) to the result grammar to verify that we will get the NFA we started with. We have not changed the language, the stringset. Let us do that with Figure 5.6. First we apply (23a) to get the NFA in Figure 5.7 where Z' is the new state that is made up, to avoid confusion.

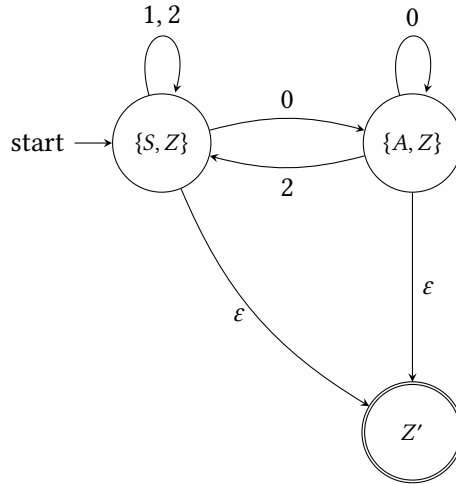


Figure 5.7: The relaxed NFA from the DFA of Figure 5.6. The $\emptyset$ state of the DFA is eliminated because there can be no transitions from it to Z' . Consequently its transitions are eliminated as well.

Now make up some names for the sets $\{S, Z\}$ and $\{A, Z\}$ for the states in the figure. Seven rules we obtain from the figure, one for each transition, are precisely the same as the rules in (22).

Since we are now going in the other direction compared to the NFA construction after (22), to establish a regular expression, we need to translate the regular grammar we obtained to a **regexp**. Recall that a regular expression can be formed by three operations, (i) concatenation, (ii) union, and (iii) Kleene star. There are also base cases by which every empty string or member of the alphabet is a regular expression. Case (i) translates $A \rightarrow aB$ and $B \rightarrow a$ to ab . Case (ii) translates $A \rightarrow a \mid b$ to $a \cup b$. Case (iii) translates $A \rightarrow aA \mid \epsilon$ to a^* . Since every regular expression just refers to terminals because there are no nonterminals, more complex expressions are built from these base constructions. (Recall that having no **nonterminals** has made regular expressions the textual tools of choice in **language annotation** and language resource maintenance in **language technology**. They are indeed very textually transparent.)

Let us apply the idea to what we have obtained again going in the other direction, (22), repeated for convenience:

$$(24) \quad \begin{aligned} S &\rightarrow 1S \mid 2S \mid 0A \mid \epsilon \\ A &\rightarrow 0A \mid 2S \mid \epsilon \end{aligned}$$

Every regular grammar can be converted into an equivalent regular expression. The general conversion is algorithmic but not especially enlightening, often yielding expressions that are more complicated than necessary. We therefore rely on intuition and algebraic simplification in the example.

Group all righthand sides of every nonterminal into one, which yields every righthand side as an option, as in (24). Think of every such rule list with the same lefthand side as a parenthesized expression. Every such rule list translates to something like $(r_1 \cup \dots \cup r_n)$ where r_i is one righthand side. The one for S is going to be the overall regular expression.

To keep matters short and intuitive, we can think of our S rule in (24) as something like $\sigma = (1 \cup 2)^* 0\alpha$ where α is the regular expression for the A rules, and σ is our regular expression for S . The Kleene star is because of $S \rightarrow \epsilon$. It is *followed* by 0α because of being derived from a regular grammar, that is, it is something we recognize *after* we are done with this use of S . Now α itself is something like $\alpha = 0^* 2\sigma \mid \epsilon$ because A refers back to S after it is done with this loop of A . This is where the tedious algorithms can help us solve these recurrence relations, which then we would have to simplify algebraically (see §4.6.1) because they tend to be large, very large, but we keep it intuitive for this example.¹² Substituting for α in

¹²It fares no better than directly relating regular expressions and automata, which tend to create many states or subexpressions depending on which direction we work. The relief is that all of that nondeterminism can be eliminated as we have seen, and written back in textual expression in a simpler way. We are able to do all of this because at this level of structural expressivity (§3.2.2), **admissibility** is so iterative the methods can be made completely deterministic.

σ in the first recurrence step we reach $\sigma = (1 \cup 2)^* 0(0^+ 2\sigma \mid \epsilon)$. Intuitively continuing, we can see that it allows repetition of 1, 2 and $0^+ 2$, not necessarily ending with one of these (because of ϵ). It may end with 0's. This is what is described by the regular expression in (21b): $(1 \cup 2 \cup 0^+ 2)^* (0 \cup \epsilon)^*$.

5.4.3 Transducers and regular relations

Moore machines and **Mealy machines** are abstract machines that are called **transducers**, things that output something. They were designed to transform input to output, not as language recognizers. They can be repurposed for linguistics by turning them into recognizers of one language *and* producers of another language. This need was motivated by the recognition of their relevance to **morphology** and **phonology** through **regular relations** (§4.6.2), for example by Kaplan & Kay (1994), Beesley & Karttunen (2003). For example, we can design an abstract language of **meta phonemes**, or underspecified phonemes standing for some abstraction. H for example could mean “high vowel”, to be realized as *i*, *ɪ*, *u*, *ü* depending on the phonological context. We can then condition the regular relations H:i, H:ɪ, H:u and H:ü using the structural power of transducers. Starting with Koskeniemi (1983), this has been used extensively in the field for **morphological analyzers** and **phonological analyzers**.

Moore machines produce output by being in a state. Mealy machines produce output by making a transition between states. A Moore machine can be transformed to a Mealy machine and vice versa. Rather than give full formal definitions of these machines with two languages, we look at their definitions of languages and operating conditions. Mealy machines seem to be more intuitive for morphology and phonology, so we use them for description.

- (25) Let $M = (Q, \Sigma_i, \Sigma_o, \Delta, s, F)$ be a Mealy machine. Σ_i, Σ_o are respectively input vocabulary and output vocabulary. F, s are same as FAs. Δ is the transition relation. It has entries such as $(p, a : b), (q)$ meaning in state p it scans input a , output b , and ends in state q . In terms of **regular relations**, we may consider this transition to relate a and b .
- The **starting configuration** of $M = [s, w_i : \epsilon]$ for input w_i .
 - The **accepting configuration** of $M = [f, \epsilon : w_o], f \in F$.
 - The input language of $M = \mathcal{L}_i(M) = \{w_i \mid [s, w_i : \epsilon] \vdash^* [f, \epsilon : \alpha]\}$, for some $\alpha; f \in F$.
 - The output language of $M = \mathcal{L}_o(M, w_i) = \{w_o \mid [s, w_i : \epsilon] \vdash^* [f, \epsilon : w_o]\}$, $f \in F$.

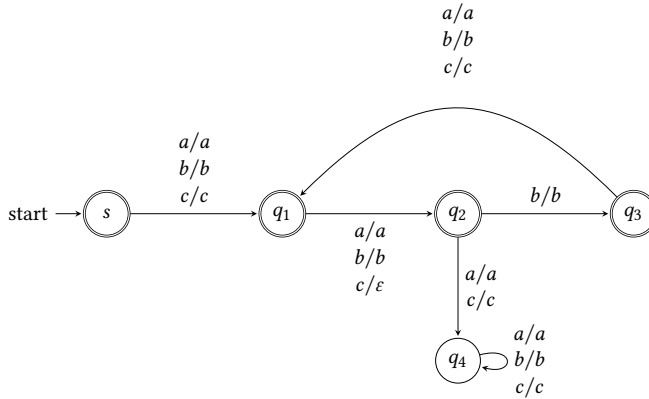


Figure 5.8: A Mealy machine for accepting the input language out of $\{a, b, c\}$ where every third symbol is b , and output language of deleting from accepted strings c 's if there is one just before that b . It is drawn in the standard notation used in the literature. In our terminology, for example the transition c/ϵ from q_1 to q_2 is $c:\epsilon$ as a **regular relation**. One regular relation for the machine is $((a:a \cup b:b \cup c:c)(a:a \cup b:b \cup c:\epsilon)b:b)^*(a:a \cup b:b \cup c:c \cup \epsilon:\epsilon)^2$.

An example is shown in Figure 5.8.

Summarizing the stage we reached in the book, we have arrived at Figure 5.9

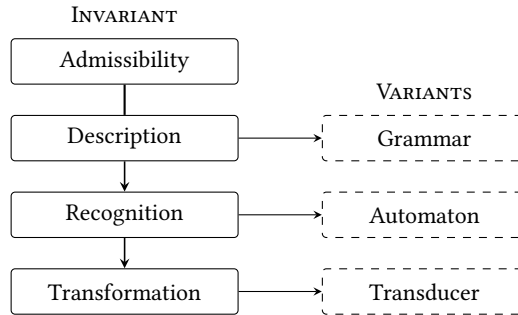


Figure 5.9: From admissibility to computation. Grammars describe admissible strings, automata recognize them, and transducers map them systematically to outputs.

6 Mathematics of linguistic embedding and delineation

We have come to the point summarized in Figure 6.1, ready to take on the bigger picture, of putting things together. It should come as no surprise that that is going to involve **interpretation** for natural languages. We are **situated** beings.

The first thing to note about embedding in natural language is that **syntax** can structurally embed indefinitely (although it does not have to, which is the claim for Pirahá—[Everett 2005](#)). **Semantics** can embed indefinitely too (and that seems to be not contested by anyone), yet the two embeddings are not reducible to each other but obviously connected. Embedding here is used in the mathematical sense of structure-preserving inclusion. These two domains of linguistics, syntax and semantics, are expected to share the dual principle (embedding, connecting distinctively) in the invariant so that we can see its admissibility (connectedness) realized in a variant. It is going to affect our choice of mathematical methods for the variants.

It will be seen in this chapter that two other domains in connection to form and meaning, **morphology** and **phonology**, have different, nonembedding, invariants because they do not embed *like* syntax and semantics; however; they may (and do) decompose. Consequent differences for the mathematical methods for their variants would then seem natural.

To repeat from the beginning of the previous chapters: The easiest way to conceive the abstraction in **structural embedding** is the problem of matching parentheses: Every opening parenthesis must close, and close only after nested open parentheses close. This “after” is therefore an abstraction, a structural relation. It is not a linear metric of number of left and right parentheses.

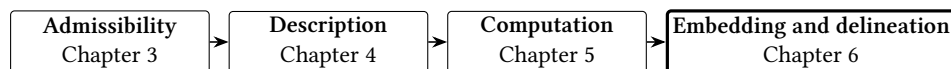


Figure 6.1: The progression of the book from structural admissibility to description, computation, and finally to linguistic embedding and delineation.

6.1 Why are syntax and semantics necessarily embedding?

When it comes to natural problems related to nested structure, the following data shows that the so-called **center-embedding** problem (1a) can be interpreted as a concern for the variant rather than the invariant (i.e. not affecting admissibility or connectedness but very difficult if not impossible to process as nesting deepens). Indefinite embedding in the same language is much easier to interpret as long as the right kind of verbs is involved, cf. (1b) and (1c); and, center-embedding is not problematic in all languages, even preferred, as in (1d).

- (1) a. The man [the dog [the cat chased] bit] laughed. English
- b. I know [you know [she knows [that John believes [that the earth is flat]]]].
- c. *I know you know she likes that John believes that the earth is flat.
- d. Ben [sen-in [ben-im [Ayşe'nin film-i gör-düğ-ü-nü]
I you-2s I-1s A-3s movie-ACC see-COMP-3S-ACC
bil-diğ-im-i] san-dığ-m-ı] düşün-üyor-um. Turkish
know-COMP-1S-ACC consider-COMP-2S-ACC think-PROG-1S
'I think you believe that I knew that Ayşe has seen the film.'

However, a critical concern for both the invariant and the variant is that, although both kinds of embedding can be indefinite when grammatical, syntactic embedding and semantic embedding are not identical, although, obviously, they must be related. The following distinctions have been pointed out from this perspective quite early in the modern era, for example by Chomsky (1965):

- (2) She begins to try to persuade/promise/expect/want him to study. English

Syntactic embedding is same for all the verbs, at least on surface; for example, their bracketing is the same; their syntactic behavior is almost indistinguishable; for example, they passivize more or less with equal ease—perhaps 'promise' showing a bit of gradience, yet their semantics is not identical: With 'persuade' he studies; with 'promise' she studies; 'he' is not semantically an argument of 'expect' or 'want' although it is an argument of both 'persuade' and 'promise'; what 'want' does in him studying (desired propositional state) is not semantically the same as what 'expect' does in him studying (desired epistemic state). Some of these are differences in semantic kind; most of them are differences in semantic embedding, different from syntax. Expect talks about what is likely. Want talks about what is preferred. Syntax does not tell us which. It just transmits what can be made meaningful by forming or interpreting an expression.

6.1 Why are syntax and semantics necessarily embedding?

All languages make such distinctions, but, obviously, not the same way. These facts put the relation between **syntax**, **semantics**, **morphology** and **phonology** in perspective. Looking from the **invariant**, **linguistic variants** manifest themselves as **processes**: Syntax is not only transparent structure, it is transparent transmission of **structured content**. Semantics is not just content, it is structured content, structured by us living in a world of change (NB. §2.2). Morphology is not just words and their operations, it is the beginning of construction of form-meaning correspondences, about what forms go with what meanings. Phonology is not just about sounds and signs, it is about structuring of transmission medium of knowledge of language. We would expect subsequent choice of methods to be affected by these consequences of admissibility conditions.

Negation can affect identical syntactic embeddings differently:

- (3) He doesn't think/know that she left.

With 'think' he thinks that she did not leave; with 'know' he does not know that she did not leave. Syntactic embedding, when grammatical, does not automatically give semantic embedding. If syntax gives us admissibility, and that through verbal **subcategorization** (i.e. syntactic requirements of the verb), semantics needs verbs to give more than subcategorization, a space of interpretation which is *consequent* to grammaticality. Such observations will guide our efforts when it comes to combining methods for syntax and semantics.

We do not need negation, which is known to be problematic in daily inference,¹ to assess the difference in syntax and semantics. In the following example, the clausal argument is syntactically identical, yet semantics of 'manage' entails success whereas semantics of 'try' does not:

- (4) He managed/tried to leave.

And sometimes, it is the same verb that does different things *because* it can take different syntactic kinds of arguments:

- (5) a. He remembered to lock the door.
b. He remembered that he locked the door.

The second example is recall. The first one is not.

In closing this section, we take another look at two examples in the beginning of the section, (1b) and (1c). The first one shows that syntax is sufficiently embedding, and consequently, semantics is sufficiently embedding. The second one shows that it is not without qualification, in this case depending on the nature

¹One revealing study of the problems is Wason & Johnson-Laird (1972).

of clausal complementation, therefore all of this embedding is meaningful when the expression is grammatical. (1d) shows that the phenomenon is not peculiar to English, but its subcategorizations and syntactic configurations, therefore inferential meanings, are. That is why we write different grammars for different languages although we have the feeling that there is probably one invariant of connectedness in any grammar.

Syntactic embedding tells us where one clause sits inside another. Apparently nesting of clausal structure may not be necessary; for example, simple clause chaining in a very flat structure may be possible, but nesting of semantics in such structures still needs explanation. Syntax does not tell us what semantic relation holds between clauses: belief, knowledge, desire, attempt, success, perception, influence and obligation may all appear in very similar syntactic frames. Non-clausal nesting can be indefinite too, for example *the man's book's dust jacket's color*. Again, it is not syntax that tells us the semantic dependencies. Such nesting is not always preferred even when a language allows it, employing other strategies to express the same kind of dependencies. In others words, syntactic admissibility and semantic admissibility, which we can take to be unlimited compositionality, are different species.

Syntax is therefore **transmission** of structure,² not determination of structure, which may be indefinitely embedding. It is because of this property, the transparency of abstraction in linguistic forms, that it must be necessarily nesting in its transmission. Transmission must be understood in its mechanisms, that is, in its variants, to and from simplex structures (words, morphemes—meaning bearing elements) to and from complex expressions. Semantics is consequent meanings of potentially indefinite structures, therefore it too must be necessarily nesting.

6.2 Contingent homomorphisms: putting embeddings together

Let us now see, given the methods we have studied, how **meaningfulness** can be consequent to **grammaticality** in a **variant**.

In the previous chapter we have seen that even in a simple structural problem which is artificially constructed, the problem of matching parentheses, there

²I am here trying to avoid theory-laden terminology which would need an ontological commitment to a **variant**, a reification of grammar and how it operates on that ontology. **Projection**, **logical structure** and **derivational structure** are terms commonly used in reification discourse.

is a consequent sense of meaningfulness. Matching parentheses are meaningful because they are grammatical, not the other way around.

We have seen natural language examples in §2.2. The implication of that for combining the methods we have seen in the previous chapter is that semantics follows syntax. Its method of choice is therefore consequent to the method of choice for syntax. We have seen in §6.1 that although syntax and semantics can embed arbitrarily, they do not embed identically, therefore if we want to study consequent sense of meaningfulness, consequent to grammaticality, we must put these two embeddings in a **correspondence** of linguistic description (for grammaticality) and method (for meaningfulness). This idea has been known for a long time, emanating from two sources depending on inclination, (i) as a formal logical method from Montague (1973), and (ii) as a method for natural languages, from Harris (1946), Wells (1947), Bach (1976), sometimes altogether called the **rule-to-rule-thesis** and **direct compositionality** (Barker & Jacobson 2007).³ We follow a method in which these problems are not worked out independently, because we are trying to see what is *consequent* to grammaticality.

This section is about how to do that for **context-free languages** with standard methods compositionally. The implication is that if want to extend our understanding of grammaticality and consequent sense of meaningfulness with empirically more adequate methods for example those in §4.2, because of Swiss German data, the same move would extend to all human languages; we would make similar choices for putting things together. It is just that starting with less adequate context-free languages makes presentation easier to follow because standard methods are available at that level.

We use the **bottom-up** method for this purpose (§5.3.2). We shall see that this choice of method is not arbitrary. As a start consider the following table relating syntactic description to semantic method. The first column is a description not a method because it does not answer the *how* question; it answers the *what* question for a **variant** (namely, English grammar and its categories). The second column is a method because it does not answer the *what* question by itself although it does have semantic content. Its use is consequent to the employment of the corresponding syntactic description. We must therefore set up a correspondence, which is itself a method, not something about **admissibility**.

³Klein & Sag (1985) suggested further that we do not have to stipulate the rules, that they are predicatable from one recursive relation of semantic interpretation.

(6)	syntactic description	semantic method
	$S \rightarrow NP VP$	$S = \lambda np.\lambda vp.vp(np)$
	$VP \rightarrow V_{acmp} NP PP_{in}$	$VP = \lambda v.\lambda np.\lambda pp.v(np)(pp)$
	$VP \rightarrow V_{tr} NP$	$VP = \lambda v.\lambda np.v(np)$
	$VP \rightarrow VP PP$	$VP = \lambda vp.\lambda pp.vp(pp)$
	$V_{acmp} \rightarrow \text{played}$	$V_{acmp} = \lambda x.\lambda y.\lambda z.iter'(x, z, play'(x, y)) \wedge master'(x, y)$
	$V_{tr} \rightarrow \text{played}$	$V_{tr} = \lambda x.\lambda y.play'(x, y)$
	$NP \rightarrow \text{she}$	$NP = she'$
	$NP \rightarrow \text{the piano}$	$NP = piano'$
	$PP_{in} \rightarrow P_{in} NP$	$PP_{in} = \lambda p.\lambda np.p(np)$

We shall explain the nomenclature of these variants next. But first, observe that the rule-to-rule-thesis (and direct compositionality) are at work as the guiding principles of the **variants**. For example, the syntactic description of *played* as an accomplishment verb (V_{acmp}) takes three arguments on the semantic side, for compositionality, whereas *played* as an activity verb (V_{tr}) takes two. **Preterminal** entries engender these readings (more on that soon), and the syntactic rules in which they are interpreted, respectively the first and the second VP rules, conform.⁴ If we want to study **fundamental questions in linguistics**, the foremost being **language acquisition**, then we better not take this **homomorphism** between nonidentical syntactic and semantic structures as an **axiomatic homomorphism** but **contingent homomorphism** in studying child grammars; we briefly touch upon these issues briefly as we proceed. We are now at this final stage of the book considering the following:

- (7) **Homomorphisms** do not have to be axioms of grammar; they may be empirical hypotheses about how linguistic structures correspond.

The correspondences in (6) are written in such a way that once a righthand side of a syntactic rule (i.e. lefthand side's condition) is satisfied, the semantic method composes the meaning as a *function* of the meanings of elements on the righthand side, from left to right. In the first rule for example, once an NP and a VP are derived, their lambda terms are substituted for meaning assembly.⁵ In

⁴There are ways to make these specifications nonredundant therefore not prone to miscorrespondences, as is done in for example **categorial grammars**, but we leave that to advanced texts. It does not concern **admissibility**. We employ **phrase structure** as lingua franca for syntactic description of variants.

⁵The descriptions in (6) are deliberately simplified for exposure. For example, pronominal reference is simply *she'* for *she*, meaning whatever reference the pronoun engenders. Also, the verb is the main function in semantic methods, for example in the clausal rule. There is no **generalized quantification**, that is, determiners as raised functions taking nominals and verbs as arguments. There are no second-order **case functions** either, functions for arguments that seek argument-takers that subcategorize for them. These are advanced topics.

other words, we assemble the meaning of the lefthand side as a function of the meanings of the elements on the righthand side.

This is best suited for **bottom-up parsing**. By combining these descriptions in a bottom-up parser we can see for ourselves how meaning assembly proceeds one derivational step at a time, to the point that once the expression is found to be grammatical, we also will have built the meaning which is a consequence of that grammaticality, not just a postinterpretation of a general assumption of grammaticality. Here the beauty is in the detail to show that while we are deciding grammaticality we are also incorporating the native speaker's and signer's *choice* of categories to make the expression meaningful for self, for example *play* as accomplishment or activity. In this regard recall §2.2 (6), repeated for convenience:

- (8) a. *She won the competition for an hour.
- b. She won the competition for a year.
- c. She played the piano in a year.
- d. *She played the piano in an hour.

What would make for example (d) meaningful is the category that makes it grammatical in the first place, something like V_{acmp} of (6), *not* V_{tr} of (6). That is because V_{acmp} 's semantics is the one that says she played the piano iteratively (*iter'*), not just one activity of play, and reached a mastery. Only that interpretation gives (d) the *sense* that she must be a genius *for that accomplishment* in such a period, not the others. This sense is consequent to its meaningfulness, itself a consequence of grammaticality. That is why all of these aspects must be put together, as a demonstration of our understanding of these connections. This may get tedious, combining every step of the syntactic derivation relation ($\Rightarrow$, defined §4 (4a)) with semantic action ($\vdash$, defined §5 (2)). That is what **parsers** are for, technologically making modeling of such aspects possible, scientifically making our ideas explicit enough to be transparently assessable. One example would be enough to see the point.

Consider the case in which (8d) is considered grammatical, therefore meaningful. How do we derive both properties? A **bottom-up parser** which can be deduced from (6) using the methods in §5.3.2 is the following.⁶ Instead of giving a full translation of (6) into an automaton, which can be done following

⁶The critical point here is that, once we have the level of admissibility and consequent linguistic description worked out, the method that would compute the correspondence, the **parser**, is deducible, as in (15) of Chapter 5.

§5.3.2 (15), we can concentrate on the abstraction that makes the difference in the computation of consequent meaningfulness of grammaticality. The sides of each translated rule in action are boxed for convenience in the following fragment, the righthand side of the rule in the first box, the lefthand side of the rule in the second box, as described in §5.3.2 (15).

- (9) a. $(s, \epsilon, \boxed{\text{played} : \lambda x.\lambda y.\lambda z.\text{iter}'(x, z, \text{play}'(x, y)) \wedge \text{master}'(x, y)}),$
 $(s, \boxed{V_{\text{acmp}} : \lambda x.\lambda y.\lambda z.\text{iter}'(x, z, \text{play}'(x, y)) \wedge \text{master}'(x, y)})$
 b. $(s, \epsilon, \boxed{\text{played} : \lambda x.\lambda y.\text{play}'(x, y)}),$
 $(s, \boxed{V_{\text{tr}} : \lambda x.\lambda y.\text{play}'(x, y)})$

Depending on which **categorical choice** we make consequent meaningfulness would be different. This is quite a **categorical shift** in the Husserlian sense ((8) of Chapter 2), because the following are not considered categorical options where category mistakes would have been made by the bearer of these categories, and if Ryle is right ((3) of Chapter 2), native speakers never make such “mistakes.” It is the business of the analysts to get our analyses right, and on the right ground.

- (10) a. $(s, \epsilon, \boxed{\text{played} : \lambda x.\lambda y.\text{play}'(x, y)}),$
 $(s, \boxed{V_{\text{acmp}} : \lambda x.\lambda y.\lambda z.\text{iter}'(x, z, \text{play}'(x, y)) \wedge \text{master}'(x, y)})$
 b. $(s, \epsilon, \boxed{\text{played} : \lambda x.\lambda y.\lambda z.\text{iter}'(x, z, \text{play}'(x, y)) \wedge \text{master}'(x, y)}),$
 $(s, \boxed{V_{\text{tr}} : \lambda x.\lambda y.\text{play}'(x, y)})$

6.3 Contingent homomorphisms and fundamental questions

We have come to the philosophical juncture of the book: Why would we study **contingent homomorphisms** of form and meaning rather than their **axiomatic homomorphisms**?

A child does not acquire a grammar by first assuming that syntax and semantics are homomorphic. Rather, the child must discover whatever regularities exist between sound, structure, meaning, and action from finite experience. If those regularities are approximately homomorphic, that is a finding about human language, not a premise of the theory. An axiomatic homomorphism assumes from the outset that syntax and semantics stand in a homomorphic correspondence.

A contingent homomorphism asks whether syntax and semantics exhibit homomorphic properties, and determine empirically how far those properties extend. The first one is a formal, logical question, the second is a natural science question. The role of scientific theory is not to legislate homomorphism but to discover it. The question is not whether elegant mappings can be constructed—they always can—but which **invariants** survive across speakers, signers, languages, developmental stages, and communicative situations. This is consistent with our urge that although we may be rationalizing **adult grammars** *posthoc*, we are prepared to be more open minded when it comes to **child grammars**.

Proceeding this way, **model selection** becomes a most interesting step in understanding the phenomenon because no model would be axiomatically guaranteed to be homomorphic. Consequently, the **modelability** of the empirical proposals becomes crucial so that we can look back on the **invariant** and try to see what makes some models more attractive, Poincaré-style, including a deliberation, a comparison with models which are considered to be axiomatically homomorphic. The important point about modelability is not that it must always facilitate replication and prediction through experiments and thought experiments—as in “hard sciences;” it must help us make things explicit so that the theory can be assessed by believers, nonbelievers and agnostics alike. **Narratives** we can live by, **theories** we must build if we can.

For the next generation of linguists, a mathematical understanding of the invariants seems more pressing than reifying them in psychology, biology or natural logic. That is the most transparent connection to **scientific modeling**.

To summarize some **fundamental questions** in linguistics, we have at least the following, repeated from the very beginning, §1 (1):

- (11) a. Why are children able to acquire language? (**ontogeny**)
- b. What is the relation of **adult grammars** to **child grammars**? (**continuity**)
- c. Why are there so many different languages when **dependencies** they manifest seem to be limited? (**diversity**)

Regarding the second one, (11b), in the context of the current discussion in relation to the other two, one influential research program treats child grammars and adult grammars as developmentally related systems rather than qualitatively different ones (e.g. Crain & Thornton 1998). It is the **continuity hypothesis**.

Whether continuity is complete or only partial is an empirical question. In either case, the transition from child grammars to adult grammars becomes a legitimate object of mathematical inquiry through contingent homomorphisms between grammatical systems. Whether continuity proves complete, partial, or domain-specific is ultimately an empirical question.

From the present perspective, the important point is that the mapping between child grammars and adult grammars is itself contingent and therefore a legitimate object of mathematical inquiry. **Admissibility** would not change about **grammaticality**, but what goes into grammaticality develops with the **social semiotic**, e.g. grasp of **achievements**, **accomplishments**, and **joint attention**; see for example [Hrđy \(2005, 2009\)](#), [Tomasello \(2001, 2009\)](#).

6.4 Why are morphology and phonology sufficiently decomposing and necessarily nonembedding?

We have seen that, mathematically, **embedding** (or nesting) differs from **iteration**, whether or not they are on the same kind of (i.e. **recursive**) values or not. Their **admissibility** conditions are different (Chapter 3). Different admissibility conditions naturally give rise to different preferred mathematical descriptions. It is time we look at the consequence of that on the linguistic **variants**.

We have seen arguments before (§4.4) about differences in embedding of **morphology** and **syntax** emanating from **referentiality**, that is, from **semantics**. We also know that there seems to be no morphemes within a morpheme, no phonemes within a phoneme, although they may have hierarchical structure and iterate. It is best we look at the earlier data once more to see why they must still be decomposing even though they may not embed *like syntax* and its **semantics**.

Consider the example §4.4 (25), repeated in the following. The subscripts denote referential dependencies.

- (12) a. Kadıköy belediye-si_i vergi daire-si_j çöp vergi-si_k
K municipality-3s tax department-3s garbage tax-3s
yatırma_k gişeler-i_j bina-sı_i.
deposit desk-3p building-3s
'The building of Kadıköy municipality garbage tax collection desks of
the tax department' Turkish
- b. Bu ne binası? 'Which building is that?'
- c. *Bu ne dairesi? 'Which department is that?'
- d. *Bu ne vergisi? 'Which tax is that?'

They are not atomic but **decomposing** in the sense that for example if we change the possessive agreement, it will affect the possessor and the possessed although they can be separated quite iteratively (13). They are **nonnesting** referentially, as can be seen from (12c) and (12d).

- (13) Kadıköy belediye-miz_i vergi daire-si_j çöp vergi-si_k yatırma_k
 K municipality-1P tax department-3s garbage tax-3s deposit
 gişeler-i_j bina-(miz)_i.
 desk-3P building-1P
 ‘The building of our Kadıköy municipality garbage tax collection desks of
 the tax department’ Turkish

We could do the same with an inner element of the compound as long as it is referentially encapsulated (14), much like (12c) and (12d). In contrast, Turkish syntax allows nesting of referential dependencies and reaching into them (15).

- (14) Kadıköy belediye-si_i vergi daire-miz_j çöp vergi-si_k yatırma_k
 K municipality-3s tax department-1s garbage tax-3s deposit
 gişeleri-(?miz)_j bina-sı_i.
 desk-1s building-3s
 ‘The building of Kadıköy municipality our garbage tax collection desks of
 the tax department’ Turkish

- (15) Ben-im sen-in tanı-dığ-ı-nı bil-diğ-im çocuk nerede?
 I-1s you-2s acquaint-COMP-2s-ACC know-REL-1s child where
 Where is the child who I know you know? Turkish

Decomposition and **embedding** require different kinds of methods because they belong to different **variants**.

Much like §6.2, we can bring together the aspects we have discussed in this section using standard methods for **regular dependencies**, the **finite automata**. Extending them to superregular dependencies is still under development in the field. They are not covered. One suggestion for self study is to have a closer look at **transducers** to turn regular dependency computation into recognition of form and production of correlative meaning, of the kind we see in stem-bound semantics such as plurals and gender for nouns, tense and aspect for verbs, etc. Their correlational nature has been revealed in statistical investigations as well, for example [Futrell & Hahn \(2025\)](#).

6.5 Mathematics of delineation

Delineation is the idea of drawing a boundary, real or imaginary, personal or public, between a presumably self-sustaining body, concrete or abstract, and the world. We know that all living creatures have some kind of delineation because

they maintain **homeostasis**, that is, a stable system of body. Embedding explains **connectedness**; delineation explains **identity**.

The keyword here for understanding abstractness of this process is **system**, at least for humans when it comes to abstraction.⁷ Although ancients felt deeply about this point, in more recent times it was **Schopenhauer (1819)**, **Husserl (1900)** who insisted that it demands a **representation** of the world, that is, an abstraction in the form of **intension** (with ‘s’ not ‘t’). That is because the concept of system as an abstraction can make delineation workable across the kinds of boundaries mentioned. In other words, they can be variants of one invariant. This forms the basis of modern **systems theory**, leading to **systems science**.⁸

We see this urge to think in terms of systems to arrive at an explanation of complex human behavior (rather than description and enumeration of solutions) in biology (**Păun 2000**), self-organizing human behavior (**Pask 1976**), computer science (**Marr 1977**), psychology (**Gibson 1966**), and anthropology (**Geertz 1973**, **Jaynes 1976**). We have also seen early attempts to use it for natural language, for example **Wiener (1954)**, **Nalimov (1974/1981)**. Wiener saw language as a means of communication and did not worry about the implications of language without communication and communication without language. **Nalimov** was linguistically more insightful, from syntax to semantics. **Fodor (1983)** is perhaps the easiest entry to modern conceptions of language as a system.

Just to wet your appetite for systems as a linguist, it is worth reminding ourselves that linguistic categories⁹ are unlike other categories, say for music, organized action, dance, etc., although we also feel that they must be related somehow if not for anything else then for the fact that one mind does it all without mixing them up. There seems to be some kind of **modularity**, and what that means mathematically may reveal some subtleties if, for the sake of exploration, we can think of ourselves as systems. These are the topics of this section.

So what exactly is a system? A full exposure is not what we need. We will summarize its basic concepts and mathematics from a modern classic in the field, **Dorf & Bishop (2017)**, and from more descriptive **Flood & Carson (1993)**.

⁷Though we should not count our closest cousins or ravens out, among others.

⁸**Cybernetics** was the nowadays abandoned and repurposed name in the early days (**Wiener 1954**), drawing from the wonderful Platonic analogy of **kubernetes**, steering a ship.

⁹At any level, from distributional categories such as nouns and verbs, property versus index, pronoun versus referring expressions, to subject and object, ergative and accusative subsystems, code switches, idioms, speech acts, and the like.

6.5.1 The idea of system as an abstraction

A **system** is an interconnection of elements. A **control system** is a system for a desired **system response**, an altogether behavior of a system as a whole. Keep that word, *desired*, in mind, for language; we will come back to it. Whether we have an expected response or not, the assumption that these elements are considered *interconnected* makes it an abstraction. Nature does not need that assumption. So when we look at nature using systems, we are not saying that we are looking at natural systems, we are studying what we can say about natural things *if* they are systems, and whether that perspective helps us know more about them and ourselves than what we started with. Some system concepts are necessary for such kind of study.

A **sensor** provides measurement of a desired **external signal** (that word again). That is how a system connects with the world. A natural system is assumed to be doing that, and a designed system is designed to do that, knowing what to measure.¹⁰ An **actuator** is something a control system is assumed to use to adjust the environment. Therefore **choice**, **control** and **action** seem to be inextricably related in a system. For example, if you have a sweeping robot, its motor drive is an actuator that turns its electric energy to mechanical torque. The designers of the robot had desired responses under predictable operating conditions, not your robomop.

A **process** is something with input and output. The implication from the name is that it is the *processing* of the external signal within the system along with other things inside that delivers an internal signal as output. (It is not an external signal until that internal signal is put out. Think about a case where your robot prepares enough torque in the right direction but say a rubber band designed for force transfer fails to deliver that on to action. The failure would be inside the system, and that is why you would call the repair service rather than resituate the robot. You might resituate it say on a less inclined surface before the call to check for power insufficiency but after a few trials you would know what to do. You would do that if you know perhaps instinctively that these are designed *systems*—what went wrong inside is for designers and producers to find out.)

Before we cover different kinds of systems and their implications for linguistics, let us have a brief look at the idea of **complexity** from a system's perspective. Flood & Carson (1993) reiterate the following aspects in dealing with complexity:

¹⁰Knowing what to measure is of course not to be taken lightly for meaningful action. For example, in **psychology**, Gibson (1966) treats **perception** as a system doing inference in its own right, rather than just take in "raw data" first then do something about it. This process itself is considered to be **categorical** in nature (Barsalou 1999).

- (16) A system can be called **complex** if
- a. there are large number of components (elements) in it,
 - b. there are large number of interactions in it,
 - c. the system is **nonlinear**,
 - d. the system is **asymmetric**,
 - e. the system is **nonholonomic**.

For many system scientists, (a–b) would indicate a big system, not necessarily a complex system, because it is more of the same thing. A nonlinear system has at least one element that interacts with others in a nonlinear way. For example a tire rolling freely down the hill is a nonlinear system. Such systems can behave quite unpredictably depending on starting points, and become **unstable**. An asymmetric system shows differential growth at least in one aspect. For example, cell division creates more cells, so in one sense it leads to bigger organization of same material, but in the process of development it becomes a different creature, not just collection of parts. This is asymmetry conceived as a *process*, not a label. Nonholonomy means some parts of the system can be temporarily outside control and go their own ways. The keyword here is *temporarily*, because if it were permanent, then we were not looking at a control system anyway.¹¹ **Holonomy** is the system concept that is most appealed to in linguistics in the form of **modularity** as a noncomplex system (Fodor 1983).¹² There is a nuance here, that **modularity** can be about organs-as-systems that are holonomic, having **procedural modularity**, or it can be about **compositionality** and **encapsulation** of interactions between **processes**, that is, systems with unique, holonomic, limited **interfaces**, which is called **representational modularity**; see Crain & Steedman (1985), Fodor & Lepore (1998) for some discussion.

If **human grammars** can be conceived as systems, that is, if the **invariant** of **connectedness** can lead to **variants** which can themselves be conceived as systems, then our understanding that a grammar is more than a list of its parts and their interactions, no matter how many, may indicate a complex system, most likely in the sense of asymmetry, and maybe more. However, before moving to **asymmetry** and **holonomy**, it is worth looking closer to the relation between **complexity**, **structure**, and **growth**.

¹¹It is the opposite of being **holonomic**, that is, having wholes (holos) with laws (nomos). The term holonomy has another use in mathematics, which we do not cover.

¹²System scientists tend to eschew the m-word (modularity)—just look at the subject index of textbooks on systems science, knowing that it is hopelessly overloaded. If it helps, you can think of a nonholonomic system as a system whose modularity is sacrificed temporarily.

There is a connection that has to be made for understanding the relation between an apparently **discrete** domain like language, discrete in the sense that its **functions** are discrete, summable and subtractable but not integrable or differentiable, and apparently **nondiscrete** domains like time-varying systems, nondiscrete in the sense that their functions are differentiable and integrable (but functions nevertheless). The mathematics of the connection goes back to Euler and Laplace. It was a profound discovery by Euler, symbolized in (17a), which is perhaps the most famous equation in mathematics, connecting five cherished constants and two cherished concepts of mathematics in one identity: 1, 0, π , i (imaginary number where $i^2 = -1$), e (the base of the natural logarithm), + and =.

- (17) a. $e^{i\pi} + 1 = 0$, (Euler's identity)
 b. which we obtain from $e^{ix} = \cos x + i \sin x$ by letting $x = \pi$.

Its relation to **structure** is as follows: Growth in size is best symbolized with an exponent of the natural base: e^x . When moved to imaginary dimension as ix in (17b), growth stops growing and starts rotating. The circle symbolizes the structure of growth. Roughly, e is manifestation of growth, i is manifestation of rotation, and π is manifestation of structure.

To see the connection between discrete structures and continuous structures, the connection between two kinds of systems, let us start with the idea of **power series** in (18). This series is **discrete** because although the range of f , the coefficient function, can be real, its domain is the countable $\mathbb{N}$. Such power series are therefore **sequence encodings** because the coefficients are symbols encoded as numbers, and the exponent records the position. Some well-known coefficient functions (f 's) are listed.

- (18) Let $P(f, x) = \sum_{n \in \mathbb{N}} f(n)x^n$ be the discrete power series with coefficient f , value x , and power n .

- a. We have

$$\sum_{n \in \mathbb{N}} x^n = 1 + x + x^2 + \dots = \frac{1}{1 - x}$$

for $|x| < 1$. We have the unit coefficient $f(n) = 1$ for all n .

- b. $e = \lim_{n \rightarrow \infty} (1 + \frac{1}{n})^n$.

- c. We have

$$\sum_{n \in \mathbb{N}} \frac{1}{n!} x^n = 1 + 1 + \frac{1}{2!} + \frac{1}{3!} + \dots = e^x$$

Here we have $f(n) = \frac{1}{n!}$.

We can think of (a) where the coefficient is unity as the positional encoding of the idea of **arbitrarily large**. The behavior of the number e follows from such series too, (b–c). It is important because e is the **invariant** of the operation of continuous **growth**. It is the only value for which the derivative of exponentiation is equal to itself: $\frac{d}{dx}(e^x) = e^x$. Here, $f(n) = \frac{1}{n!}$. That is very discrete.

Natural languages may be discrete. However, if we are to continue pursuing the idea of natural-systems-as-structured-sequence-manipulators further, the system in which language is couched as a **variant**, us, would be time-varying (so is linguistic reference, if you think of it as something occurring in time). It is natural for us to seek different but related **invariants** for them. This time we seek derivatives and integrals rather than differences and sums because we are encoding continuous sequences. These are the objects of inquiry in continuous systems, systems evolving over time.

Time is a real thing, so we will be looking at power series over x^t in something like (18), rather than x^n , t being **time**. We would not have e^{ix} of (17b) where only x is real. And, compared to (18), our f 's are no longer countable. From the equation $x = e^{\ln x}$, if x is less than unity, for the series to converge to a value, then its natural logarithm, $\ln x$, would be negative. We have $x^t = e^{(\ln x)t}$. Take $-s = \ln x$, we have $x^t = e^{-st}$, that is, $x = e^{-s}$. In a power series such as (18), having e^{-s} for x in x^t means (now that we have time t instead of discrete n) we should integrate rather than sum because we are now in a continuous domain (our f is no longer countable for t). We now have the **Laplace transform** (19). It is the profound connection of a continuous domain like t to a discrete domain like s . It also has an inverse, that is, a function going from time to discrete domain, which we do not need.

- (19) Let $f(t)$ be a function of time. It is the coefficient of the continuous growth. We have the discrete function $F(s)$.¹³ (Laplace transform)

$$F(s) = \int_{0^-}^{\infty} f(t)e^{-st} dt$$

Euler's exponential function e^x appears in the Laplace transform because it is the **invariant** of continuous change. Instead of x^n in (18) of a discrete function, we have e^{-st} . If $f(t)$ is the coefficient of continuous growth, $f(t)e^{-st}$ and its integral is the time-sequence **encoding** of the **continuous structure**. In other words, as an encoding, it is representable discretely. How discrete is $F(s)$? Here are some examples. They are time-invariant.

¹³ 0^- means the integral includes discontinuities such as $t = 0$.

$$(20) \quad \begin{array}{cc} f(t) & F(s) \\ \hline 0(t=0), 1(t \geq 0) & \frac{1}{s} \\ e^{-at} & \frac{1}{s+a} \\ t^n & \frac{n!}{s^{n+1}} \end{array}$$

The **invariant** in a **linear** continuous system is that its **transfer function**, which represents the relationship describing the **dynamics** of the system, is the ratio of the Laplace transform of the output to Laplace transform of the input. This is a very compressed representation of **dynamics of constantly growing structure**. The transfer function is input-output description of system's **behavior**. It does not say anything about the internal structure of the system. The internal structure, however, matters a lot in linguistics. Let us peek into it.

6.5.2 Open systems, semantics, and choice

It is time we connect with what we started in §2.2, the ontogeny of grammaticality. Viewed as systems making choices not just decisions, human language acquirers must be **situated** in an environment. Figure 6.2 is one way to conceive it as a system. Let us go through the terminology in the figure. **Desired response** is an expected outcome for some **observable**. It is added (+) to the system. The **feedback**, which feeds the output of the system back to its input, is **negative feedback** (-), meaning the difference between it and desired outcome would be considered *error* to be corrected. The feedback is not raw output; it goes through a **sensor** to be considered measured. (These are also **feedforward** systems, where the output is passed on to *another* system as input.) Crucially, for our purposes, the **controller** is not just a decision maker. It may choose some desirable input, have a policy. In other words, the system as a whole chooses to be in a certain niche of the world, much like shifts of grammaticality did in §2.2. It does not even have to send all its output as feedback. Such is the nature of **open systems**, which are systems nevertheless.

It is one way to incorporate linguistic choices made, choices because *for some expressions* we may choose an expression to be **grammatical**, consequently **meaningful**, when under other circumstances we wouldn't do that, making a **categorical jump**, an intuition jump in a Husserlian way. Such systems dealing with **semantics** in addition to **syntax** ought to be **referential**, to serve sequence-encoding functions of discrete and continuous kinds we have looked at in (18) and (19).

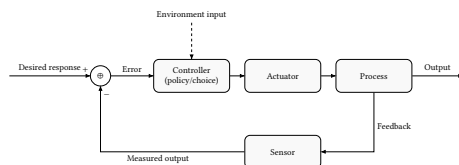


Figure 6.2: An **open-loop** system. Its response is not adjusted fully from feedback. Besides feedback, the controller receives independent information from the environment. Whether or not it uses it in policy making or choice making is open. A closed-on-input system is obtained by removing the dashed environmental input and replacing the controller by a fixed policy.

6.5.3 Closed systems, semantics, and decision-making

A **closed system** is an **open system** closed on all inputs.¹⁴ That is, it has no **degrees of freedom**. Everything depends on what we may call **desired output response**, the problem of ‘desired’ already solved for the system. This is more or less what **Montague (1973)** did for **semantics**, *assuming* we know that **syntax** and semantics are **homomorphic**, and no degrees of freedom were left in conceiving **possible worlds (models)** in which an expression may be true. There is no further choice-making and reference adjustment during this process.

We can see that in systems terminology, by conceiving the models to be built from systems like those in Figure 6.3, adapted from Figure 6.2. Notice one crucial difference between the figures. In Figure 6.2 we do have a degree of freedom to choose reference, therefore a **linguistic category**, whereas in Figure 6.3 that choice is assumed to have been made earlier; we only want to find out what truth worlds are carved out by the choice. In summary, the homomorphism of syntax-semantics in Figure 6.2 would be **contingent**—the data and the environment ought to come in to the picture; in Figure 6.3 it is **axiomatic**.¹⁵ **Systems science** is one way to make that explicit and express what would be involved in a nonreductionist attempt.

This is another way of saying that it is essential in linguistics to study linguistic homomorphisms contingently as well as axiomatically to arrive at a better under-

¹⁴There is one more dimension of open systems. Its discussion would take us too far afield. In thermodynamics, open systems can not only exchange information but also *material* with the environment.

¹⁵Incidentally, if it helps matters to be put in a broader perspective, **Cisek (1999)** suggests that even if we the humans are conceived as systems as much as the things we designed, the computers, are also conceived as systems, we would be like Figure 6.2, not like Figure 6.3, and the computer would be the other way around. Whether computers one day will be like Figure 6.2 is not what philosophers and scientists call a thought experiment; it is just bold speculation.

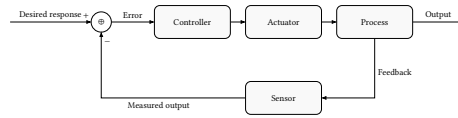


Figure 6.3: A **closed-loop** system. There is no policy or choice, no degree of freedom. The controller takes the difference between desired response and measured response and adjusts based only on that.

standing of the connection between **syntax** and **semantics**, with a better understanding of **morphology**'s and **phonology**'s role in building them and carrying them on, allowing it—the connection—to diversify within predictable boundaries as we see in **typology**. **Systems theory** tells us that the **complexity** of the overall task need not be daunting; it can be kept under control under certain **admissibility conditions**, mathematics of which is good to know.

In this endeavour, model building is your friend, at least not your enemy or rival. **Modelability** of linguistic ideas to the extent of making them explicit enough for others to try is not an option anymore. The task is too complex to be done alone or within isolated communities. It does not mean that we all have to be modelers, but it does mean that our ideas must be modelable, whether we are theoretically inclined or practically inclined. It is the linguistic survival asset in the technologically advanced world.

Let me end the book the way I ended Chapter 2. Mathematics is a language for discovering problems, rather than merely solving them. It can be made transparent to the extent that it facilitates wider community effort. Mathematics does not replace linguistic inquiry. It tells us what kinds of inquiry remain possible. In that sense, mathematics is less a language of certainty than a language of disciplined curiosity. Best of luck with it for the challenges in Figure 6.4 and for tackling the **fundamental questions** in (11).

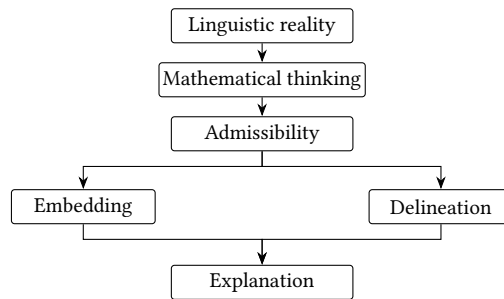


Figure 6.4: From linguistic reality to explanation through mathematical thinking, admissibility, embedding, and delineation.

References

- Abend, Omri, Tom Kwiatkowski, Nathaniel Smith, Sharon Goldwater & Mark Steedman. 2017. Bootstrapping language acquisition. *Cognition* 164. 116–143.
- Aho, Alfred. 1968. Indexed grammars—an extension of context-free grammars. *Communications of the Association for Computing Machinery* 15. 647–671.
- Aho, Alfred V. & Jeffrey D. Ullman. 1972. *The theory of parsing, translation, and compiling*, vol. I. New Jersey: Prentice-Hall.
- Aitchison, Jean. 1987. *Words in the mind: An introduction to the mental lexicon*. Oxford: Basil Blackwell.
- Amdahl, Gene M, Gerrit A Blaauw & Frederick P Brooks. 1964. Architecture of the IBM System/360. *IBM Journal of Research and Development* 8(2). 87–101.
- Anderson, Stephen R. 1992. *A-morphous morphology*. Cambridge Univ. Press.
- Anderson, Stephen R., Lea Brown, Alice Gaby & Jacqueline Lecarme. 2006. Life on the edge: There's morphology there after all! *Lingue e Linguaggio* 1(1). 33–47.
- Arnheim, Rudolf. 1954. *Art and visual perception*. Berkeley: Univ. of California Press.
- Aronoff, Mark & Kirsten Fudeman. 2011. *What is morphology?* 2nd ed. Chichester, UK: Wiley-Blackwell.
- Bach, Emmon. 1976. An extension of classical Transformational Grammar. In *Problems in linguistic metatheory: Proceedings of the 1976 conference at michigan state university*, 183–224. Lansing, MI: Michigan State University.
- Bach, Emmon. 1989. *Informal lectures on formal semantics*. Albany, NY: State University of New York Press.
- Barendregt, Hendrik. 1981. *The lambda calculus*. Amsterdam: North Holland.
- Barker, Chris. 2022. Rethinking scope islands. *Linguistic Inquiry* 53(4). 633–661.
- Barker, Chris & Pauline Jacobson (eds.). 2007. *Direct compositionality*. Oxford University Press.
- Barsalou, Lawrence. 1999. Perceptual symbol systems. *Behavioral and Brain sciences* 22(4). 577–660.
- Beesley, Kenneth R. & Lauri Karttunen. 2003. *Finite state morphology*. Stanford, CA: CSLI Publications.

References

- Bird, Steven & T. Mark Ellison. 1994. One-level phonology: Autosegmental representations and rules as finite automata. *Computational Linguistics* 20(1). 55–90.
- Birdsong, David. 1989. *Metalinguistic performance and interlinguistic competence*. New York: Springer-Verlag.
- Bozşahin, Cem. 2018. Computers aren't syntax all the way down or content all the way up. *Minds and Machines* 28(3). 543–567.
- Bozşahin, Cem. 2026. Recursion: Conceptual necessity or convenience? cause, effect, or by-product? In Aristotelis Panagiotopoulos & Antonios Achilleos (eds.), *Proc. of the 15th PanHellenic logic symposium*. July, Athens, Greece.
- Burns, Samuel R. 2009. The problem of deduction: Hume's problem expanded. *Dialogue* 52(1). 26–30.
- Cann, Ronnie. 1993. *Formal semantics: An introduction*. Cambridge University Press.
- Chierchia, Gennaro & Sally McConnell-Ginet. 1990. *Meaning and grammar: An introduction to semantics*. Cambridge, MA: MIT Press.
- Chomsky, Noam. 1959. On certain formal properties of grammars. *Information and Control* 2. 137–167.
- Chomsky, Noam. 1965. *Aspects of the theory of syntax*. Cambridge, MA: MIT Press.
- Chomsky, Noam. 1970. Remarks on nominalization. In R. Jacobs & P. Rosenbaum (eds.), *Readings in English transformational grammar*, 184–221. reprinted in Chomsky 1972. Waltham, MA: Ginn.
- Chomsky, Noam. 1972. *Studies on semantics in generative grammar*. The Hague: Mouton.
- Chomsky, Noam. 1955/1975. *The logical structure of linguistic theory*. Chicago: University of Chicago Press.
- Chomsky, Noam & Morris Halle. 1968. *The sound pattern of English*. New York: Harper & Row.
- Cisek, Paul. 1999. Beyond the computer metaphor: Behaviour as interaction. *Journal of Consciousness Studies* 6(11-12). 125–142.
- Clark, Eve. 1982. The young word maker: A case study of innovation in the child's lexicon. In Eric Wanner & Lila Gleitman (eds.), *Language acquisition: The state of the art*, 390–425. Cambridge University Press.
- Comrie, Bernard. 1989. *Language universals and linguistic typology*. 2nd. Oxford: Blackwell.
- Crain, Stephen & Mark Steedman. 1985. On not being led up the garden path: The use of context by the psychological parser. In David Dowty, Lauri Karttunen & Arnold Zwicky (eds.), *Natural language parsing: Psychological, computational and theoretical perspectives*, 320–358. Cambridge: Cambridge University Press.

- Crain, Stephen & Rosalind Thornton. 1998. *Investigations in universal grammar*. Cambridge, MA: MIT Press.
- Culy, Christopher. 1985. The complexity of the vocabulary of Bambara. *Linguistics and Philosophy* 8(3). 345–351.
- Dorf, Richard C. & Robert H. Bishop. 2017. *Modern control systems*. 13th edition. Essex: Pearson.
- Dowty, David, Robert Wall & Stanley Peters. 1981. *Introduction to Montague semantics*. Dordrecht: Reidel.
- Ellis, Willis D. 1938. *A source book of gestalt psychology*. London: Routledge & Kegan Paul.
- Everett, Daniel. 2005. Cultural constraints on grammar and cognition in Pirahã: Another look at the design features of human language. *Current Anthropology* 46. 621–646.
- Faltz, Leonard M. 1998. *The Navajo verb*. University of New Mexico Press.
- Flood, R.L. & E.W. Carson. 1993. *Dealing with complexity*. 2nd ed. Plenum Press.
- Fodor, Jerry. 1983. *The modularity of mind*. Cambridge, MA: MIT Press.
- Fodor, Jerry & Ernie Lepore. 1998. The emptiness of the lexicon: Reflections on James Pustejovsky's *the generative lexicon*. *Linguistic Inquiry* 29. reprinted in **Fodor & Lepore 2002**:89–119, 269–288.
- Fodor, Jerry & Ernie Lepore. 2002. *The compositionality papers*. Oxford: Oxford University Press.
- Frege, Gottlob. 1879. Begriffsschrift. In Peter Geach & Max Black (eds.), *Translations from the philosophical writing of Gottlob Frege*. 1966. Oxford: Blackwell.
- Futrell, Richard & Michael Hahn. 2025. Linguistic structure from a bottleneck on sequential information processing. *Nature Human Behaviour*. 1–12.
- Gallier, Jean. 2011. *Discrete mathematics*. New York: Springer.
- Galovich, Steven. 1989. *Introduction to mathematical structures*. San Diego: Harcourt Brace Jovanovich.
- Ganesalingam, Mohan. 2013. *The language of mathematics*. Berlin: Springer.
- Gazdar, Gerald. 1988. Applicability of indexed grammars to natural languages. In Uwe Reyle & Christian Rohrer (eds.), *Natural language parsing and linguistic theories*, 69–94. Dordrecht: Reidel.
- Geertz, Clifford. 1973. *The interpretation of cultures*. New York: Basic Books.
- Gibson, James. 1966. *The senses considered as perceptual systems*. Boston, MA: Houghton-Mifflin Co.
- Gleitman, Lila & Henry Gleitman. 1997. What is a language made out of? *Lingua* 100(1–4). 29–55.
- Hadamard, Jacques. 1945. *An essay on the psychology of invention in the mathematical field*. Dover.

References

- Harris, Zellig. 1946. From morpheme to utterance. *Language* 22. 161–183.
- Harrison, Michael. 1978. *Introduction to formal language theory*. Reading MA: Addison-Wesley.
- Haspelmath, Martin. 2011. The indeterminacy of word segmentation and the nature of morphology and syntax. *Folia linguistica* 45(1). 31–80.
- Hawkins, John. 2003. *Efficiency and complexity in grammars*. Oxford: Oxford University Press.
- Heidegger, Martin. 1927. *Sein und zeit [being and time]*. Tübingen: Max Niemeyer Verlag.
- Heim, Irene & Angelika Kratzer. 1998. *Semantics in generative grammar*. Oxford: Blackwell.
- Heisenberg, Werner. 1958. *Physics and philosophy*. New York: Harper.
- Hopcroft, John E. & Jeffrey D. Ullman. 1979. *Introduction to automata theory, languages and computation*. Reading, MA: Addison-Wesley.
- Horsman, Clare, Susan Stepney, Rob C Wagner & Viv Kendon. 2013. When does a physical system compute? *Proc. of the Royal Society A* 470(20140182).
- Householder, Fred W. 1973. On arguments from asterisks. *Foundations of Language* 10(3). 365–376.
- Hrdy, Sarah Blaffer. 2005. Comes the child before man: How cooperative breeding and prolonged postweaning dependence shaped human potential. In Barry Hewlett & Michael Lamb (eds.), *Hunter gatherer childhoods*, 65–91. New Brunswick, NJ: Aldine/Transaction.
- Hrdy, Sarah Blaffer. 2009. *Mothers and others*. Cambridge, MA: Belnap/Harvard University Press.
- Hume, David. 1739. *A treatise of human nature*. 1739–1740. London: John Noon.
- Husserl, Edmund. 1900. *Logical investigations*. 1970 trans. by J. N. Findlay [Original German edition, 1900–1901.] New York: Humanities Press.
- Hyams, Nina & Robyn Orfitelli. 2018. The acquisition of syntax. In Eva M. Fernández & Helen Smith Cairns (eds.), *The handbook of psycholinguistics*, 593–614. Wiley.
- Jacobson, Pauline. 2014. *Compositional semantics*. Oxford University Press.
- Jaynes, Julian. 1976. *The origin of consciousness in the breakdown of the bicameral mind*. New York: Houghton Mifflin Harcourt.
- Johnson, C. Douglas. 1972. *Formal aspects of phonological description*. The Hague: Mouton.
- Johnson-Laird, Philip N. 1983. *Mental models*. Cambridge, MA: Harvard University Press.

- Joshi, Aravind, K. Vijay-Shanker & David Weir. 1991. The convergence of mildly context-sensitive formalisms. In Peter Sells, Stuart Shieber & Tom Wasow (eds.), *Processing of linguistic structure*, 31–81. Cambridge, MA: MIT Press.
- Kallmeyer, Laura. 2010. *Parsing beyond context-free grammars*. Springer.
- Kaplan, Ron & Martin Kay. 1994. Regular models of phonological rule systems. *Computational Linguistics* 20. 331–378.
- Katz, Jerrold J. 1985a. An outline of Platonist grammar. In Jerrold J. Katz (ed.), *The philosophy of linguistics*, 172–203. in [Katz 1985b](#). Oxford University Press.
- Katz, Jerrold J. (ed.). 1985b. *The philosophy of linguistics*. Oxford University Press.
- Keenan, Edward & Bernard Comrie. 1977. Noun phrase accessibility and universal grammar. *Linguistic Inquiry* 8. 63–99.
- Keenan, Edward L. & Leonard M. Faltz. 1985. *Boolean semantics for natural language*. Dordrecht: D. Reidel.
- Kiraz, George Anton. 2001. *Computational nonlinear morphology: With emphasis on Semitic languages*. Cambridge: Cambridge University Press.
- Klein, Ewan & Ivan Sag. 1985. Type-driven translation. *Linguistics and Philosophy* 8. 163–201.
- Knuth, Donald E. 1996. *Selected papers on computer science*. Cambridge University Press.
- Koestler, Arthur. 1959. *The sleepwalkers: A history of man's changing vision of the universe*. London: Hutchison.
- Koffka, Kurt. 1936. *Principles of gestalt psychology*. London: Kegan Paul.
- Köhler, Wolfgang. 1938. *The place of value in a world of facts*. New York: Liveright.
- Koskenniemi, Kimmo. 1983. *Two-level morphology: A general computational model for word-form recognition and production*. Department of General Linguistics, University of Helsinki. (Doctoral dissertation).
- Kroeger, Paul. 1993. *Phrase structure and grammatical relations in Tagalog*. Stanford CA: CSLI Publications.
- L.T.F. Gamut. 1990. *Logic, language, and meaning, vol. 1: Introduction to logic*. Chicago: University of Chicago Press.
- L.T.F. Gamut. 1991. *Logic, language, and meaning, vol. 2: Intensional logic and logical grammar*. Chicago: University of Chicago Press.
- Lenneberg, Eric H. 1967. *The biological foundations of language*. New York: John Wiley & Sons.
- Lewis, Harry R. & Christos H. Papadimitriou. 1998. *Elements of the theory of computation*. 2nd ed. New Jersey: Prentice-Hall.
- Li, Charles N. & Sandra A. Thompson. 1981. *Mandarin Chinese: A functional reference grammar*. Berkeley: Univ of California Press.

References

- Linnebo, Øystein. 2017. *Philosophy of mathematics*. New Jersey: Princeton University Press.
- Lobina, David J. 2014. What linguists are talking about when talking about... *Language Sciences* 45(10). 56–70.
- Mac Lane, Saunders. 1971. *Categories for the working mathematician*. Berlin/New York: Springer.
- Mackie, John L. 1965. Causes and conditions. *American Philosophical Quarterly* 2(4). 245–264.
- Mallinson, Graham & Barry Blake. 1981. *Language typology*. Amsterdam: North Holland.
- Manaster-Ramer, Alexis. 1988. Book reviews: The formal complexity of natural language. *Computational Linguistics* 14(4). 98–103.
- Marr, David. 1977. Artificial intelligence: A personal view. *Artificial Intelligence* 9. 37–48.
- Matthews, P.H. 1974. *Morphology: An introduction to the theory of word-structure*. Cambridge: Cambridge University Press.
- Mills, Jonathan W. 2008. The nature of the extended analog computer. *Physica D: Nonlinear Phenomena* 237(9). 1235–1256.
- Montague, Richard. 1970. Universal grammar. *Theoria* 36. Reprinted as Thomason 1974:222–246, 373–398.
- Montague, Richard. 1973. The proper treatment of quantification in ordinary English. In J. Hintikka & P. Suppes (eds.), *Approaches to natural language*. Dordrecht: D. Reidel.
- Moravcsik, Edith A. 1974. Object-verb agreement. *Working papers on language universals* 15. 25–140.
- Moravcsik, Edith A. 1978. Agreement. In Joseph H. Greenberg, Charles A. Ferguson & Edith A. Moravcsik (eds.), *Universals of human language IV: Syntax*, 331–374. Stanford Univ. Press.
- Morrill, Glynn. 2011. *Categorical grammar: Logical syntax, semantics, and processing*. Oxford: Oxford University Press.
- Nalimov, Vasili Vasilevich. 1974/1981. *In the labyrinths of language*. English edition in 1981 (edited by R.D. Colodny). Philadelphia: ISI Press.
- Neurath, Otto. 1983. *Philosophical papers 1913–1946*. Dordrecht: Springer.
- Partee, Barbara, Alice ter Meulen & Robert Wall. 1990. *Mathematical methods in linguistics*. Dordrecht: Kluwer.
- Pask, Gordon. 1976. *Conversation theory: Applications in education and epistemology*. Amsterdam: Elsevier.
- Păun, Gheorghe. 2000. Computing with membranes. *Journal of Computer and System Sciences* 61(1). 108–143.

- Pearl, Judea & Dana Mackenzie. 2018. *The book of why: The new science of cause and effect*. Basic books.
- Penrose, Roger. 2006. *The road to reality*. Random house.
- Penrose, Roger. 2013. *Cycles of time: An extraordinary new view of the universe*. Random House.
- Platero, Paul R. 1974. The Navajo relative clause. *International Journal of American Linguistics* 40(3). 202–246.
- Poincaré, Henri. 1905. *Science and hypothesis*. London: Walter Scott Publishing.
- Popper, Karl R. 1945. *The open society and its enemies, vol. I & II*. London: Routledge.
- Russell, Bertrand. 1927. *An outline of philosophy*. London: George Allen & Unwin.
- Ryle, Gilbert. 1931. Systematically misleading expressions. In *Proceedings of the Aristotelian society*, vol. 32, 139–170.
- Ryle, Gilbert. 1937. Categories. *Proceedings of the Aristotelian Society*. reprinted in [Ryle 1971](#).
- Ryle, Gilbert. 1971. *Collected papers, vol. II 1929–1968*. New York: Barnes & Noble.
- Sakai, Itiroo. 1961. Syntax in universal translation. In *Int. conf. on machine translation and applied analysis*, 594–608.
- Salmon, Merrilee H, John Earman, Clark Glymour, James H Lennox, Peter Machamer, J E McGuire, John D Norton, Wesley Salmon & Kenneth F Schaffner. 1992. *Introduction to the philosophy of science*. Reprinted in 1999 by Hackett Publishing. Englewood Cliffs: Prentice-Hall.
- Sandra, Dominiek. 1998. What linguists can and can't tell about the human mind: A reply to Croft. *Cognitive Linguistics* 9. 361–378.
- Sapir, Edward. 1924/1949. The grammarian and his language. In David G. Mandelbaum (ed.), *Selected writings of Edward Sapir in language, culture, and personality*. Originally published in *American Mercury* 1: (1924) 149–155. Berkeley: University of California Press.
- Schönfinkel, Moses Ilyich. 1920/1924. On the building blocks of mathematical logic. In Jan van Heijenoort (ed.), *From Frege to Gödel*. Prepared first for publication by H. Behmann in 1924. Harvard University Press, 1967.
- Schopenhauer, Arthur. 1819. *Die welt als wille und vorstellung [the world as will and representation]*. Leipzig: Brockhaus.
- Schütze, Carson T. 1996. *The empirical base of linguistics*. Chicago: Univ. of Chicago Press.
- Senuma, Hajime & Akiko Aizawa. 2024. Computational complexity of natural morphology revisited. *Transactions of the Association for Computational Linguistics* 12. 649–663.

References

- Sherman, Janet Cohen & Barbara Lust. 1993. Children are in control. *Cognition* 46(1). 1–51.
- Shieber, Stuart. 1985. Evidence against the context-freeness of natural language. *Linguistics and Philosophy* 8. 333–343.
- Simmons, Harold. 2011. *An introduction to category theory*. Cambridge University Press.
- Sipser, Michael. 2006. *Introduction to the theory of computation*. 2nd ed. Boston, MA: Thomson Course Technology.
- Stanojević, Miloš & Mark Steedman. 2021. Formal basis of a language universal. *Computational Linguistics* 47. 9–42.
- Sudkamp, Thomas A. 2006. *Languages and machines: An introduction to the theory of computer science*. 3rd ed. Reading, MA: Pearson Addison Wesley.
- Thomason, Richmond (ed.). 1974. *Formal philosophy: Papers of Richard Montague*. New Haven, CT: Yale University Press.
- Tomasello, Michael. 2001. Perceiving intentions and learning words in the second year of life. In Melissa Bowerman & Stephen Levinson (eds.), *Language acquisition and conceptual development*, 132–158. Cambridge: Cambridge University Press.
- Tomasello, Michael. 2009. *Why we cooperate*. Cambridge, MA: MIT Press.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan Gomez, Łukasz Kaiser & Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*, 5998–6008.
- Wang, Hao. 1974. *From mathematics to philosophy*. Routledge & Kegan Paul.
- Wason, Peter & Philip Johnson-Laird. 1972. *Psychology of reasoning: Structure and content*. Batsford.
- Weizenbaum, Joseph. 1976. *Computer power and human reason*. New York: W.H. Freeman & Company.
- Wells, Rulon. 1947. Immediate constituents. *Language* 23. 81–117.
- Wertheimer, Max. 1924. *Gestalt theory*. English translation published in Ellis 1938.
- Wiener, Norbert. 1954. *The human use of human beings: Cybernetics and society*. Boston: Mifflin Co.
- Willie, MaryAnn. 1989. Why there is nothing missing in Navajo relative clauses. In Eung-do Cook & Keren D. Rice (eds.), *Athapaskan linguistics: Current perspectives on a language family*, 407–437. Walter de Gruyter.
- Winter, Yoad. 2016. *Elements of formal semantics: An introduction to the mathematical theory of meaning in natural language*. Edinburgh University Press.

Mathematics of Linguistic Reality

add description to localmetadata.tex